

基于模糊测试的 PLC 竞态漏洞检测 方法研究

(申请清华大学工学硕士学位论文)

培 养 单 位： 计算机科学与技术系

学 科： 计算机科学与技术

研 究 生： 陈 张 萌

指 导 教 师： 张 小 平 研究员

二〇二五年十一月

Research on PLC Race Condition Vulnerability Detection Methods Based on Fuzzing

Thesis submitted to

Tsinghua University

in partial fulfillment of the requirement

for the degree of

Master of Science

in

Computer Science and Technology

by

Chen Zhangmeng

Thesis Supervisor: Researcher Zhang Xiaoping

November, 2025

关于学位论文使用授权的说明

本人完全了解清华大学有关保留、使用学位论文的规定，即：

清华大学拥有在著作权法规定范围内学位论文的使用权，其中包括：（1）已获学位的研究生必须按学校规定提交学位论文，学校可以采用影印、缩印或其他复制手段保存研究生上交的学位论文；（2）为教学和科研目的，学校可以将公开的学位论文作为资料在图书馆、资料室等场所供校内师生阅读，或在校园网上供校内师生浏览部分内容；（3）按照上级教育主管部门督导、抽查等要求，报送相应的学位论文。

本人保证遵守上述规定。

作者签名： _____

导师签名： _____

日 期： _____

日 期： _____

摘 要

随着工业互联网与运营技术（Operational Technology, OT）及信息技术（Information Technology, IT）网络的深度融合，可编程逻辑控制器（Programmable Logic Controller, PLC）作为工业控制系统的核心单元，其代码安全问题已成为制约智能制造系统可靠性的关键瓶颈。传统 PLC 代码安全检测方法受限于性能约束，运行效率普遍较低，且由于未能充分考虑 PLC 程序固有的时序敏感性及其与外设交互的动态特性，往往难以有效识别诸如竞态条件等隐蔽型安全漏洞。针对上述问题，本文提出将 PLC 源代码在 x86 架构通用计算平台上重新编译，并在此基础上提出一种动静态协同的基于模糊测试的混合分析框架，以提升 PLC 代码安全分析的执行效率，同时使之具备对竞态条件安全问题的检测能力。

本文的主要贡献包括：首先，将 PLC 源代码在通用计算环境中重新编译，并引入虚拟外设模拟层以仿真实际外设行为，从而显著提升模糊测试的执行性能；其次，提出一种针对 PLC 程序的竞态条件检测方法，通过监控程序在多个运行周期内的状态变化，识别非崩溃型异常行为；第三，为进一步优化测试效率，设计了静态分析与动态测试之间的深度协同机制，借助变量分析为模糊测试过程提供精准的种子输入生成指导；最后，构建了完整的实验验证平台。基于真实工业场景的测试结果表明，本文所提方法在检测效率与漏洞覆盖范围方面均显著优于传统检测手段。

本研究不仅为 PLC 程序的安全保障提供了创新的技术路径，其动静态协同的分析范式也对嵌入式系统、物联网设备等相关领域的软件安全分析具有重要的参考价值。随着工业 4.0 战略的持续推进，本文所提出的方法有望为构建高可信工业控制系统提供有效的技术支撑。

关键词：工业互联网安全；工业控制系统安全；可编程逻辑控制器；模糊测试技术；竞态条件

Abstract

With the deep integration of industrial internet with Operational Technology (OT) and Information Technology (IT) networks, the security of Programmable Logic Controller (PLC) code, as the core unit of industrial control systems, has become a critical bottleneck constraining the reliability of smart manufacturing systems. Traditional PLC code security detection methods are often limited by performance constraints, resulting in generally low operational efficiency. Moreover, due to insufficient consideration of the inherent timing sensitivity of PLC programs and their dynamic characteristics when interacting with peripherals, these methods frequently struggle to effectively identify concealed security vulnerabilities such as race conditions. To address these issues, this paper proposes recompiling PLC source code on general-purpose x86 architecture computing platforms and, based on this, introduces a hybrid analysis framework that combines static and dynamic approaches with fuzzing. This framework aims to enhance the execution efficiency of PLC code security analysis while equipping it with the capability to detect race condition vulnerabilities.

The main contributions of this paper include: First, recompiling PLC source code in a general-purpose computing environment and introducing a virtual peripheral simulation layer to emulate actual peripheral behaviors, thereby significantly improving the performance of fuzzing. Second, proposing a race condition detection method tailored for PLC programs, which identifies non-crash anomalies by monitoring state changes across multiple execution cycles. Third, to further optimize testing efficiency, designing a deep collaboration mechanism between static analysis and dynamic testing, leveraging variable analysis to provide precise seed input generation guidance for the fuzzing process. Finally, constructing a comprehensive experimental validation platform. Results based on real-world industrial scenarios demonstrate that the proposed method significantly outperforms traditional detection approaches in both detection efficiency and vulnerability coverage.

This research not only provides an innovative technical pathway for ensuring PLC program security but also offers valuable insights for software security analysis in related fields such as embedded systems and Internet of Things (IoT) devices through its static-dynamic collaborative analysis paradigm. With the ongoing advancement of the Industry

4.0 strategy, the method proposed in this paper is expected to provide effective technical support for building highly trustworthy industrial control systems.

Keywords: Industrial Internet Security; Industrial Control Systems (ICS); Programmable Logic Controllers (PLCs); Coverage-Guided Fuzzing; Race Condition Detection

目 录

摘 要.....	I
Abstract.....	II
目 录.....	IV
插图清单.....	VII
附表清单.....	VIII
符号和缩略语说明.....	IX
第 1 章 绪论	1
1.1 选题背景与意义	1
1.2 工业控制系统与工业互联网	2
1.2.1 工业控制系统与工业互联网	2
1.2.2 工业互联网安全标准	2
1.2.3 可编程逻辑控制器简介	4
1.2.4 PLC 编程语言的应用趋势	6
1.3 PLC 安全威胁	6
1.3.1 人机界面的安全威胁	7
1.3.2 固件安全威胁	7
1.3.3 网络安全威胁	8
1.3.4 代码安全威胁	9
1.3.5 PLC 的安全威胁小结	10
1.4 PLC 中的竞态条件问题	11
1.5 本工作的主要贡献与创新点	13
1.6 论文结构与安排	14
第 2 章 相关工作与预备知识	15
2.1 静态分析方法	15
2.2 动态分析方法	17
2.2.1 符号执行	18
2.2.2 模糊测试	19
2.3 本章小结	23
第 3 章 PLC 的竞态条件问题安全分析框架设计	26
3.1 工作目标和威胁模型	26

3.2 PLC 的竞态条件问题安全分析框架总体设计	26
3.3 PLC 跨环境重新编译	28
3.4 模拟外设的响应	30
3.5 静态分析与动态分析的协同的安全分析框架	32
3.6 动态运行状态监控与异常检测	33
3.7 PLC 的竞态条件问题模糊测试框架	33
第 4 章 具体实现	36
4.1 PLC 跨环境重新编译	36
4.1.1 跨环境重新编译的相关工作介绍	36
4.1.2 兼容性问题改进	37
4.1.3 C++ 源码到模糊测试插桩	39
4.2 模拟外设的响应	40
4.2.1 模糊测试的外设建模方案	41
4.2.2 融入模糊测试框架	44
4.3 静态分析与动态分析的协同机制	45
4.4 动态运行状态监控与异常检测	52
4.5 本章总结	54
第 5 章 实验评估与结果分析	56
5.1 实验环境和测试用例说明	56
5.2 本工作在性能与漏洞挖掘能力方面的表现	57
5.3 与静态分析相关工作的对比研究	59
5.3.1 静态分析工作的复现	59
5.3.2 对静态分析工作的代码复杂度分析	76
5.3.3 两者性能对比	76
5.4 在模糊测试框架中加入静态分析的作用	77
5.5 与动态分析相关工作的执行效率和覆盖率对比研究	78
5.6 本章总结	79
第 6 章 总结与展望	81
6.1 研究工作总结	81
6.2 主要创新点	82
6.3 未来工作展望	83
参考文献	85
致 谢	89

目 录

声 明.....	91
个人简历、在学期间完成的相关学术成果.....	92

插图清单

图 1.1	PLC 中的竞态条件示例	11
图 3.1	动静态协同的 PLC 安全分析框架	28
图 3.2	虚拟外设仿真层架构图	31
图 3.3	模糊测试执行流程	34
图 4.1	动静态协同的 PLC 安全分析框架的总体流程	54
图 5.1	本工作的模糊测试的执行效率	59
图 5.2	ICSFuzz 的模糊测试效率 ^[37]	60
图 5.3	竞态问题检测时间和执行次数	60
图 5.4	和静态分析的对比结果	77
图 5.5	静态分析对比测试用例	78
图 5.6	程序覆盖率情况	79

附表清单

表 4.1	PLC 数据映像区接口类型分类	42
表 5.1	测试用例信息	58
表 5.2	不同规模 PLC 系统的库所数量与状态空间复杂度	76

符号和缩略语说明

ICS	工业控制系统 (Industrial Control Systems)
PLC	可编程逻辑控制器 (Programmable Logic Controller)
SCADA	监控管理层数据采集与监控 (Supervisory Control and Data Acquisition)
DCS	分布式控制系统 (Distributed Control System)
HMI	人机界面 (Human-Machine Interface)
LD	梯形图 (Ladder Diagram)
IL	指令列表 (Instruction List)
FBD	功能块图 (Function Block Diagram)
SFC	顺序功能流程图 (Sequential Function Chart)
ST	结构化文本 (Structured Text)
MITM	中间人攻击 (Man-in-the-Middle Attack)
DoS	拒绝服务攻击 (Denial-of-Service Attack)
OT	操作技术 (Operational Technology)
IEC	国际电工委员会 (International Electrotechnical Commission)
DES	离散事件系统 (Discrete Event Systems)
CPU	中央处理器 (Central Processing Unit)
PII	输入过程映像区 (Process Image Input)
PIO	输出过程映像区 (Process Image Output)
LLM	大语言模型 (Large Language Model)
DPI	深度包检测 (Deep Packet Inspection)

第 1 章 绪论

1.1 选题背景与意义

工业控制系统（Industrial Control Systems, ICS）作为国家关键信息基础设施的核心组成部分，在能源、交通、水利、电力及石油化工等关键行业中发挥着不可替代的作用。随着工业 4.0 战略的持续推进，工业互联网已成为工业控制系统技术演进的重要方向^[1]。工业互联网通过部署具备感知能力的智能终端设备，构建泛在互联的通信网络架构，深度融合物联网传感技术、分布式计算框架及大数据分析平台等现代信息技术^[2]。凭借其显著的性价比优势、灵活的部署方式及良好的系统适应性，工业互联网实现了对工业生产流程的实时监测、智能决策与自动化管控，从而显著提升了制造效能并优化了生产资源配置^[1]。

然而，工业互联网^[1]所实现的“人一机一物”深度互联，在提升生产效率的同时，也带来了严峻的网络安全挑战。研究表明，传统信息安全威胁在工业互联网环境中呈现出独特的攻击特征，而系统自身的异构性和实时性要求等特性又引入了新型攻击维度^[1]。需要指出的是，工业控制系统开发过程中存在的代码复用现象以及开发人员安全意识不足等问题，导致工业互联网系统漏洞持续增长^[3]。这一现状亟需建立针对工业互联网架构的纵深防御体系，为关键基础设施提供多层次的安全保障机制^[1]。

可编程逻辑控制器（Programmable Logic Controller, PLC）作为工业控制系统架构的核心控制单元，其硬件架构通常由中央处理器（CPU）、电源管理模块、输入/输出（I/O）接口模块、存储单元及功能扩展模块构成。PLC 采用专用的控制逻辑编程语言，旨在监控和管理工业自动化过程。值得注意的是，部分工业设备供应商存在认知误区，认为物理隔离的网络环境能够确保 PLC 代码的绝对安全性。然而，工业互联网的深度融合使得 PLC 系统面临多维安全威胁，包括但不限于随机网络攻击、内部人员操作威胁和外部安全入侵威胁等。实证研究表明，由于开发规范缺失或人为失误，PLC 程序可能包含逻辑缺陷、语法错误、作用域冲突、隐蔽跳转指令和冗余代码等安全隐患^[4]。这一现状表明，工业控制程序的代码安全已成为保障关键基础设施安全运行的重要研究课题。

1.2 工业控制系统与工业互联网

1.2.1 工业控制系统与工业互联网

工业控制系统作为国家关键信息基础设施的核心组成部分，在能源、电力、石油化工、水利枢纽及智能交通等关乎国计民生的重要领域发挥着支柱性作用。典型的工业控制系统采用分层架构设计，其逻辑层次可划分为：

1. **现场设备层：**由执行单元（包括伺服电机、调节阀、变频器等）和传感单元（如温度传感器、压力变送器、流量计等）构成，旨在实现对物理过程的直接控制与状态采集。
2. **现场控制层：**以可编程逻辑控制器为核心，配备远程终端单元和分布式 I/O 模块，负责执行实时控制算法并处理现场总线通信。
3. **控制网络层：**通常采用具备实时通信能力的工业以太网协议（如 Profinet、EtherNet/IP）以及现场总线技术（如 Modbus/TCP、PROFIBUS-DP），构建连接现场控制层的 PLC、过程监控层的分布式控制系统（Distributed Control System, DCS）与监控管理层的数据采集与监控（Supervisory Control and Data Acquisition, SCADA）系统之间的高速传输通道。
4. **过程监控层：**部署数据采集与监控系统、人机界面（Human-Machine Interface, HMI）以及历史数据库服务器，实现生产过程的集中可视化管理与控制。

随着工业 4.0 战略的深入实施，现代工业控制系统正经历深刻的数字化转型。这一演进过程主要体现在：通过标准化的 TCP/IP 协议栈（Transmission Control Protocol/Internet Protocol）和工业以太网技术，传统封闭的工业控制系统与公共互联网实现了深度互联。这种互联互通在显著提升生产系统协同效率的同时，也打破了传统工业控制系统的“物理隔离”安全边界。攻击面的扩大使得工业互联网体系面临全方位的安全挑战，网络世界中的风险同样会影响整个工业生产过程，并可能导致制造企业遭受重大经济损失。工业互联网系统不仅使传统安全问题表现出攻击向量泛化的特征，其开放架构特性更衍生出数据篡改、通信劫持等新型攻击模式。此外，由于编程人员的技术水平差异和开源代码复用等问题，工业互联网中出现了越来越多的漏洞。这种现状凸显了在工业互联网环境下构建纵深防御体系的紧迫性，需要从协议安全、代码审计和运行时防护等多个层面建立综合防护机制^[1]。

1.2.2 工业互联网安全标准

为增强工业互联网技术体系的安全性，需为其关键系统提供相应的安全保障。现有的工业物联网系统安全评估主要涵盖三个方面：功能安全（Functional Safety）、

物理安全（Physical Safety）和信息安全（Information Security）^[5]。

功能安全（Functional Safety），也称为机能安全，其概念在国际标准 IEC 61508 《电气/电子/可编程电子安全相关系统的功能安全》中定义，该标准规定了安全相关系统的设计、部署及维护方法体系。功能安全指安全相关系统在正常工况下保持预期功能，并在发生故障时能够通过预设的保护机制进入安全状态的能力。其核心原则是，任何安全相关系统都必须以可预测的方式正确运行或失效^[6]。基于 IEC 61508 的安全生命周期管理，功能安全要求在工程流程中尽可能消除设计错误和遗漏；并且考虑随机硬件失效概率范围内的异常对安全性的影响。虽然无法实现绝对零风险，但必须将风险降低至可接受的水平。只有在整个安全生命周期内系统性地解决这些问题，才能实现最优且经济高效的安全方案。例如，为马达加装温度传感器，当温度超过阈值时能自动停止运转，该保护机制符合功能安全设计要求。功能安全标准针对特定行业或应用存在不同的衍生标准。例如，汽车、铁路、流程工业、发电厂和机械制造等领域均制定了行业专用的安全标准^[7]。

物理安全（Physical Safety）主要指通过机械防护、电气隔离、应急停机装置等工程措施，保障工业设施本身的完整性和可用性，确保其在异常能量释放（如过电流、机械碰撞）情况下的人员安全与设备防护。

信息安全（Information Security）的核心目标包括以下三个基本属性，通常被称为 CIA 三要素^[8]：

- **机密性（Confidentiality）**：确保信息不被泄露给未经授权的个人、实体或流程。
- **完整性（Integrity）**：在数据的整个生命周期内，维护和保证数据的准确性与一致性，防止数据以未授权方式被篡改或破坏。
- **可用性（Availability）**：确保授权用户能够及时、可靠地访问数据和使用系统。

传统的工业控制系统安全主要聚焦于功能安全，即通过高可靠性的安全相关系统和设备设计，确保在系统失效时仍能维持生产设备和人员的安全状态。然而，随着工业控制系统信息化程度的不断提高以及工业互联网的快速发展，系统安全边界日益模糊，单纯依靠功能安全已无法应对新的安全挑战。目前，工业互联网系统的安全体系仍存在明显的结构性失衡——虽然在“异常情况下的系统安全”（即功能安全维度）方面建立了较为完善的保障机制，但在主动防御外部攻击（即信息安全维度）方面的研究与实践仍显不足^[6]。

1.2.3 可编程逻辑控制器简介

可编程逻辑控制器是一种专为工业环境设计的数字运算操作电子系统。它采用可编程的存储器，用于其内部存储程序，执行逻辑运算、顺序控制、定时、计数与算术操作等面向用户的指令，并通过数字式或模拟式的输入/输出控制各种类型的机械或生产过程。与传统的继电器控制系统相比，PLC 不仅能够实现逻辑控制和顺序控制等基本功能，还具备模拟信号处理和高速工业网络通信（如 EtherCAT, Profinet）等先进特性。PLC 硬件采用模块化设计，通常由以下几个核心模块组成：

- **中央处理器（CPU）**：CPU 是 PLC 的控制核心，通常由运算器、控制器和寄存器组成，负责执行用户程序、处理输入数据、进行逻辑与算术运算，并生成相应的输出指令。CPU 通过系统总线（包括数据总线、地址总线和控制总线）与存储器及输入/输出接口电路相连。
- **输入/输出（I/O）模块**：I/O 模块是 PLC 与工业现场设备之间的接口桥梁。**输入模块**接收来自传感器、按钮、限位开关等现场设备的信号，并进行光电隔离、滤波等信号调理，将其转换为 PLC 内部 CPU 可处理的数字信号。输入信号包括数字量和模拟量。**输出模块**将 CPU 处理后的控制信号进行功率放大，驱动接触器、电机、电磁阀、指示灯等执行机构动作。输出类型同样包括数字量输出和模拟量输出。
- **存储器**：存储器用于存储系统程序、用户程序和数据。PLC 的存储器通常分为系统存储器和用户存储器。**系统存储器**采用只读存储器或闪存等非易失性存储介质，用于固化存放厂商预置的系统管理程序（如运行时序控制、用户程序编译、模块诊断等）。**用户存储器**（通常为随机存取存储器，并配有电池备份）用于存储用户根据控制需求编写的应用程序以及程序运行时的中间数据。

PLC 采用循环扫描的工作方式，其工作过程可分为输入采样、用户程序执行和输出刷新三个阶段。PLC 在运行时，CPU 以特定的扫描周期重复执行上述三个阶段^[9]。

在**输入采样阶段**，PLC 在每个扫描周期的开始，按顺序读取所有输入端子上的状态信号，并将其存入输入过程映像区（Process Image Input, PII）。在此阶段，即使外部输入信号状态发生变化，PII 中的值也保持不变，直至下一个扫描周期的输入采样阶段。随后进入**用户程序执行阶段**。PLC 的 CPU 根据 PII 中的输入信号状态，按照用户程序指令的先后顺序（通常为从左到右、从上到下）进行扫描执行。程序执行过程中，涉及的所有输入状态均从 PII 中读取，所有的输出状态则写入输出过程映像区（Process Image Output, PIO）。用户程序执行完毕后，进入**输出刷新**

阶段。CPU 将 PIO 寄存器中的状态数据一次性传送至实际的输出锁存电路，驱动外部负载动作。

当一个扫描周期内的上述三个阶段全部完成后，CPU 将进入下一个扫描周期，如此周而复始。这种循环扫描的工作机制使得 PLC 能够对工业过程实现持续的监控与控制。扫描周期的长短主要取决于用户程序的大小和 CPU 的运算速度，典型值在毫秒级范围内。

PLC 因其高可靠性、强抗干扰能力、灵活的扩展性和便于维护等特点，被广泛应用于制造业、能源、交通、楼宇自动化等众多工业领域^[9]。目前，其应用已从传统的重工业（如冶金、化工、电力）延伸至高端装备制造（如数控机床、生产线）、公共设施（如智能楼宇、交通信号控制）以及环境监控等各个方面。

国际电工委员会制定的 IEC 61131-3 标准是工业自动化控制领域的重要规范，它定义了五种标准化的编程语言范式。这些语言专为工业自动化控制系统的逻辑实现而设计。^[10] 各语言的特点如下：

1. **梯形图 (Ladder Diagram, LD)**: 一种基于继电器接触器控制逻辑的图形化编程语言，采用类似于电气原理图的符号（如常开/常闭触点、线圈）来表达控制逻辑。梯形图程序由左、右电源轨和中间的逻辑“梯级”构成，其执行顺序通常为从左到右、自上而下扫描求值。该语言直观易学，广泛应用于离散逻辑控制。
2. **指令列表 (Instruction List, IL)**: 一种在语法和结构上类似于汇编语言的低级文本语言。它基于累加器栈的操作模型，代码密度高，执行效率高，通常用于对执行速度有苛刻要求或需要直接操作硬件的底层程序开发。
3. **功能块图 (Function Block Diagram, FBD)**: 一种基于数据流编程思想的图形化语言。通过将预定义或用户自定义的功能块（如 AND, OR, 定时器, 计数器, PID 控制器等）进行互连，形成信号处理流程。该语言支持严格的类型检查，适合描述过程控制系统的信号流。
4. **顺序功能图 (Sequential Function Chart, SFC)**: 一种专用于描述顺序控制过程的图形化语言。其核心思想源于 PetriNet 和有限状态机，主要由“步”、“转换”和有向连接弧构成。每一步可关联若干“动作”，当转换条件满足时，系统从一个步转移到下一个步。顺序功能图特别适用于具有清晰工艺顺序的流程控制。
5. **结构化文本 (Structured Text, ST)**: 一种语法结构类似于 Pascal 的高级编程语言。它支持丰富的数据类型（如数组、结构体）、表达式、条件语句（IF...THEN...ELSE）、循环语句（FOR, WHILE）和函数调用，非常适合实现

复杂的算法、数学计算和数据处理任务^[10]。

1.2.4 PLC 编程语言的应用趋势

在传统的工业生产中，梯形图是一种被广泛使用的编程语言。但是在工业互联网应用趋势背景下，结构化文本相比梯形图展现出显著优势。具体来说，主要体现在安全性、集成能力和未来适应性方面。^[11]

1. **支持复杂安全协议与加密集成** 工业互联网要求 PLC 处理加密通信、身份验证和实时威胁检测，结构化文本的文本结构便于实现高级算法（如 SSL/TLS 集成、数据签名），而梯形图的图形化逻辑难以处理此类复杂计算。结构化文本在数据操作中性能比梯形图高 20-35%^[11]，更适合安全关键任务。结构化文本的模块化特性支持封装安全功能块（如访问控制模块），便于重用和维护，降低漏洞风险。梯形图则依赖大量中间变量，易引入安全盲点。
2. **兼容现代开发与自动化工具** 工业 4.0 和 IoT 驱动下，结构化文本与云平台、AI 工具的集成更顺畅。Kilian Tran 等人^[12]使用大语言模型（Large Language Model, LLM）辅助生成结构化文本代码，提升开发效率；而梯形图在自动化测试和代码分析方面受限。此外，结构化文本可以部署版本控制系统（例如 Git）和持续集成/持续部署（CI/CD），便于团队协作和安全审计。相比之下，梯形图的图形修改难以追踪变化，增加安全管控难度。
3. **适应未来安全标准与法规** 随着工业安全标准的普及，结构化文本的文本格式更易实现形式化验证和合规检查。梯形图缺乏类似支持，难以满足严格审计要求。结构化文本的代码可读性和注释能力有助于文档化安全策略，符合监管要求，而梯形图的视觉逻辑在复杂系统中易导致误解。

总而言之，在工业互联网趋势下，结构化文本凭借其计算效率、工具兼容性和安全适应性，成为应对网络威胁和实现智能自动化的优选。梯形图虽在简单控制中有效，但结构化文本更能满足未来安全关键需求。建议结合混合策略，最大化系统防护能力。

1.3 PLC 安全威胁

随着工业控制系统数字化与网络化进程的加速，可编程逻辑控制器作为工业自动化体系的核心控制单元，其安全性已成为制约工业互联网深入发展的关键瓶颈。在工业 4.0 与智能制造深度融合的背景下，可编程逻辑控制器的功能边界已从传统的逻辑控制扩展至数据采集、云端协同及边缘计算等新兴领域。这种功能演进在为工业生产带来效率提升的同时，也使得原本相对封闭的工控系统暴露于多

维度的安全威胁之下。从系统架构的纵向视角分析，安全风险已渗透至设备层、网络层与应用层等各个环节^[13]。

1.3.1 人机界面的安全威胁

在工业控制系统中，人机界面作为操作人员与 PLC 进行交互的核心枢纽，其安全性直接关系到整个控制系统的可靠运行。人机界面不仅承担着工艺流程监控、参数配置和设备状态反馈等关键职能，更是连接信息管理层与物理设备层的重要桥梁。

人机界面的设计缺陷或安全漏洞可能引发多重风险：内部人员可能通过越权操作篡改控制逻辑或植入恶意软件，干扰输入/输出信号的实时性；外部攻击者则可能利用弱认证机制等漏洞渗透至 HMI，并进一步劫持发往 PLC 的控制指令流，最终导致生产线停摆或设备损毁等严重后果。

随着工业互联网的深入推进，HMI 的广泛联网趋势使其暴露面急剧扩大，传统封闭环境下的安全假设被彻底颠覆。远程攻击者可以借助漏洞扫描工具定位暴露于公网的 HMI 设备，并通过协议逆向工程（例如针对 OPC Classic、Profinet 等工业协议）构造恶意指令包，实现对底层 PLC 的非法操控^[14]。

针对这类威胁，学术界与工业界已开展多项探索性研究。例如，ICS3Fuzzer^[15]通过黑盒模糊测试技术对数据采集与监控系统的监控程序进行动态分析，揭示了人机界面软件在处理异常输入时存在内存溢出、逻辑校验缺失等安全漏洞；而 Singla 等人^[16]的研究则尝试将生物特征识别技术（如虹膜识别）融入人机界面操作权限管理，通过多模态身份认证机制来增强访问控制，抵御越权访问攻击。

这些研究在特定场景下提升了人机界面的安全性，但其在实际工业环境中的实时性、可靠性以及与传统控制系统的兼容性等方面仍面临挑战。因此，如何在强化安全防护的同时保障控制系统的实时性能与可用性，仍是人机界面安全防护体系构建的核心课题。

1.3.2 固件安全威胁

PLC 的固件是其设备运行的核心底层软件，其安全性直接决定了控制逻辑的执行可靠性与系统的抗攻击能力。固件作为 PLC 操作系统的支撑基础，一旦被篡改或植入恶意代码，将导致整个控制系统面临被完全操控的极端风险。攻击者可通过物理接触或网络渗透等手段，对固件进行逆向工程分析和恶意代码注入，例如植入后门程序以绕过安全认证机制，或篡改关键指令集以实现未经授权的操作。

更为隐蔽的威胁源于固件启动阶段可能存在的完整性验证缺陷。当 PLC 缺乏

基于密码学技术的安全启动与固件签名校验机制时，攻击者可利用传输协议中的漏洞，在固件更新过程中将合法文件替换为恶意版本。这使得被篡改的固件在设备重启时被加载，从而长期潜伏于系统中。

与此同时，PLC 用户程序（通常以编译后的二进制形式存在）在从工程站到 PLC 设备的传输环节也面临严峻的安全挑战。攻击者无需获取源代码即可对二进制文件进行逆向工程，通过反汇编、反编译等手段重构部分控制逻辑，并插入恶意指令，随后将其重新封装为看似正常的二进制文件。此类攻击常利用工业通信协议本身加密机制的薄弱性，在数据传输过程中实施中间人攻击，导致被篡改的代码被下载至 PLC 并执行，可能引发生产参数被篡改、设备异常启停等具有严重物理后果的安全事件。

这种基于二进制层的攻击不仅难以被传统的网络防火墙或入侵检测系统所发现，而且由于工业现场 PLC 设备通常具有长生命周期特性，使得此类威胁构成长期、持续的潜在风险。这从另一个层面凸显了在代码编译、传输、加载和运行的全生命周期中加强安全校验机制的迫切性^[13]。

1.3.3 网络安全威胁

随着工业控制系统网络化进程的不断深入，PLC 与工业互联网的深度融合使其面临日益严峻的网络安全威胁。位于 PLC 上层的数据采集与监控系统作为监控中枢，其自身的安全漏洞可能成为攻击者渗透至底层控制网络的跳板。例如，攻击者可以利用未及时修复的 SCADA 软件漏洞植入恶意逻辑，进而间接操纵 PLC 执行非预期指令，最终导致生产参数被篡改或设备异常停机等事故^[17]。

在通信协议层面，Modbus/TCP、PROFINET 等广泛应用的工业通信协议因其设计初衷侧重于实时性与可靠性，普遍缺乏对数据传输的加密、通信双方的身份认证及报文完整性校验等安全机制。这一设计缺陷已被诸多已知的工业网络攻击事件所利用。攻击者可借助专门的协议分析工具构造伪装数据包，实施中间人攻击（Man-in-the-Middle Attack, MITM），劫持 PLC 与上位机（如 HMI、工程站）之间的通信链路，从而窃取关键工艺参数或注入虚假控制命令；而重放攻击（Replay Attack）则通过截获并重复发送合法的历史指令，欺骗 PLC 执行旧有操作，例如在化工生产过程中循环开启已被逻辑关闭的阀门，可能导致物料配比失衡甚至安全事故。此外，针对工业网络的拒绝服务攻击（Denial of Service, DoS）可通过向 PLC 的通信端口发送大量泛洪请求，耗尽其有限的处理资源，导致合法的实时控制信号产生严重延迟或丢失，进而可能触发安全联锁系统的误动作，造成生产中断^[4]。

为应对这些威胁，学术界提出了多维度的防护思路：一方面，致力于对现有工

业协议进行安全扩展，例如为 Modbus 协议增加 TLS 加密层（Modbus Secure^[18]）或实现基于证书的双向身份认证，以增强通信过程的机密性与不可否认性；另一方面，则采用深度包检测（Deep Packet Inspection, DPI）技术对网络流量进行实时监控，并结合机器学习算法构建 PLC 等设备的正常通信行为基线，从而有效识别并告警偏离正常模式的异常流量或隐蔽攻击^{[19] [20]}。然而，工业控制场景对通信实时性与确定性的严苛要求，以及现场设备在计算能力、存储资源方面的异构性与局限性，使得上述安全机制的部署必须在防护强度与系统性能之间审慎权衡。

1.3.4 代码安全威胁

在传统的工业系统中，网络与外界物理隔离，基于这一封闭的网络环境假设，安全研究者普遍认为 PLC 在运行时执行的逻辑代码具有固有的安全性，难以被外部逆向解析或篡改。然而，这一假设在工业互联网时代已不再成立。

一方面，操作技术网络与信息技术网络的深度融合打破了原有的安全边界，PLC 通过各种工业网络协议直接或间接地暴露于企业网络甚至公共网络，使得攻击者能够绕过物理隔离实施远程渗透；另一方面，工程实践中因开发人员安全意识薄弱或编程规范缺失所导致的逻辑设计缺陷，也构成了潜在的攻击面。即便开发人员遵循了基本的编程规范，PLC 应用程序中仍可能因结构冗余、时序耦合、异常处理不当等问题而滋生漏洞^[21]。例如，Sizzle 测试框架的相关研究指出，PLC 程序中未得到妥善处理的竞态条件或非原子操作，可能在特定序列的恶意输入触发下，导致输出信号产生异常振荡，进而引发执行单元（如阀门、电机）的误动作^[22]。

此类代码级威胁的核心在于，攻击者主要可通过两种路径实施破坏：其一是直接注入或篡改 PLC 的用户程序，例如在工程站与 PLC 之间的代码传输通道中植入恶意指令；其二是利用编程、测试阶段遗留的逻辑漏洞，例如因变量作用域管理不当而引发的数据污染、因隐藏跳转指令或未初始化变量导致的控制流劫持等。尤其值得注意的是，部分 PLC 编程环境语法校验机制的不完善，使得简单的符号错误或数据类型混淆在特定工况下也可能演变为系统级故障^[14]。

代码安全威胁主要与 PLC 程序的设计、实现及维护环节相关。此类威胁的隐蔽性极高，因其既可能源于开发人员的技术局限性，也可能因跨团队协作中的设计文档缺失、版本管理混乱而加剧，最终在工业物联网的开放生态中形成“漏洞孵化温床”，对从现场设备控制到生产调度管理的全链条安全构成持续威胁^[21]。

以下通过几个简化的结构化文本（ST）代码示例，说明常见的 PLC 代码安全问题：

```
1 | TON(Timer1, PT:=T#5s);
```

```

2 IF Timer1.Q THEN
3   Motor := TRUE;
4 END_IF

```

此代码片段使用了一个接通延时定时器（TON）。当定时器达到预设时间（5 秒）时，电机启动。然而，代码缺乏对定时器的复位逻辑，一旦 Timer1.Q 变为 TRUE，它将始终保持为 TRUE，导致电机持续运行，无法停止。

```

1 CASE State OF
2   1:
3     IF StartButton THEN
4       State := 2;
5     END_IF
6   2:
7     IF StopButton THEN
8       State := 1;
9     END_IF
10  END_CASE

```

此代码实现了一个简单的两状态机。然而，它未处理 StartButton 与 StopButton 同时为 TRUE（例如因信号抖动或故障）的边缘情况。此时，状态机的行为将是不确定的，可能导致流程失控。

```

1 IF Counter1 >= 10 THEN
2   Alarm := TRUE;
3 END_IF

```

此代码在计数器值达到 10 时触发报警。但代码未考虑计数器发生溢出（例如，从最大值回绕到 0）的情况。这种边界条件处理不当可能导致报警功能失效或产生未定义行为。

1.3.5 PLC 的安全威胁小结

在工业控制系统的安全研究历程中，学术界与工业界长期将研究焦点置于组件完整性验证、工业通信协议的安全加固，以及人机界面的访问权限控制等传统领域，而对可编程逻辑控制器代码本身的安全性问题缺乏系统性的深入探索。这种研究倾向在很大程度上源于早期工业网络普遍采用的物理隔离架构所形成的固有安全假设——即认为 PLC 程序因部署于相对封闭的操作技术环境中，难以被外部攻击者直接触及，故其代码逻辑的安全性未被置于优先考量位置。

然而，工业互联网的深度演进与 IT/OT 融合趋势彻底颠覆了这一传统安全范式。OT 网络与 IT 网络的深度融合使得原本孤立的控制程序暴露于更为开放的互联边界。攻击者可利用协议逆向分析、中间人劫持等多种技术手段，突破传统基于边界的防护屏障，直接访问、分析甚至篡改 PLC 程序。更严重的是，攻击者可能通过组合利用系统漏洞，将 PLC 代码中存在的逻辑缺陷（如竞态条件、边界校验缺失等）转化为对物理设备的实际操控能力。例如，未受保护的代码传输通道可

能被恶意注入后门指令；而程序中存在的逻辑错误若被精心构造的异常输入触发，则可能导致阀门启闭时序错乱、电机非预期启停等连锁反应，进而引发生产中断或设备故障。

这一根本性转变使得 PLC 代码安全从以往相对次要的防御维度，跃升为构建弹性工业控制系统安全体系的重要支柱。对其开展深入研究不仅有助于弥补传统安全防护策略存在的盲区，更是应对工业系统智能化、网络化转型过程中所涌现新型攻击面的必然要求，为最终构建具备“内生安全”特性的工业控制生态提供必要的理论支撑与实践路径。

1.4 PLC 中的竞态条件问题

在众多 PLC 代码安全隐患中，竞态条件（Race Condition）问题因其触发机制的隐蔽性、后果的严重性以及检测的困难性，成为了一个亟待解决的关键技术难题。PLC 的竞态条件问题主要指三种情况：

1. 多个指令或逻辑分支同时去控制同一个输出或资源，导致结果不确定；
2. 两个或多个指令试图在同一时间改变同一个输出的状态；
3. 即使输入信号没有变化，输出的状态却在连续的扫描周期里不停地跳动。

此类漏洞通常源于 PLC 程序中多个并发执行的逻辑分支对共享资源（如全局变量、输出线圈、数据块）进行的非原子性访问操作，最终表现为即便在输入信号保持稳定的条件下，也可能导致输出状态产生非预期的、不确定性的跳变^[23]。

下图以梯形图的形式展示了 PLC 竞态条件的一个典型例子。

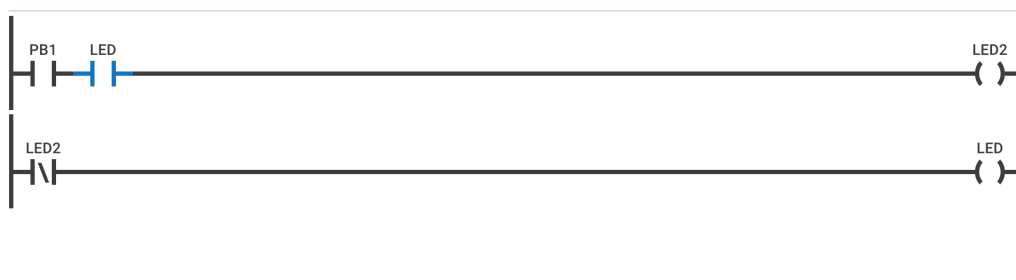


图 1.1 PLC 中的竞态条件示例

该梯形图逻辑包含一个输入点 PB1 和两个输出点 LED 与 LED2。假设输入 PB1 持续为 TRUE。在第一个扫描周期内：当扫描执行第一行逻辑时，由于 PB1 为真，故线圈 LED 得电（为 TRUE），随即其常开触点 LED 闭合，导致线圈 LED2 也得电（为 TRUE）；当扫描执行至第二行逻辑时，由于 LED2 为真，其常闭触点 LED2 断开，导致线圈 LED 失电（为 FALSE）。在第二个扫描周期内：扫描第一行时，因 LED 已于上一周期末变为 FALSE，其常开触点断开，故 LED2 失电（FALSE）；扫

描第二行时，因 LED2 变为 FALSE，其常闭触点闭合，又导致 LED 得电（TRUE）。此过程循环往复，尽管输入 PB1 始终无变化，输出 LED 和 LED2 的状态却在每个扫描周期内发生非预期的振荡。

在实际工业场景中，此类因竞态条件引发的输出信号振荡将直接导致系统的控制稳定性与安全性严重恶化。当输出变量因程序内部逻辑冲突而在扫描周期内持续震荡时，其所控制的执行器（如继电器、电磁阀、电机等）会频繁切换工作状态。这种非预期的快速动作不仅会显著加速机械部件的物理磨损，缩短设备寿命，还可能导致设备局部过热，甚至造成永久性损坏，从而引发高昂的维修成本与巨大的生产中断损失。在化工生产、能源输送等安全苛求场景中，此类问题的后果尤为致命。

例如，若物料混合流程中的阀门控制逻辑存在竞态问题，可能导致反应釜内压力失控、物料配比失衡，轻则造成产品批次报废，重则触发泄漏、燃烧甚至爆炸等灾难性安全事故，直接威胁人员生命安全并造成不可逆的环境污染。此外，输出信号的持续波动会破坏生产流程的平稳性，使温度、压力、液位等关键工艺参数偏离设定范围，可能引发下游设备的连锁故障，进一步加剧资源浪费与经济损失。

更为棘手的是，此类问题往往具有间歇性出现的特征，其显现依赖于特定的输入条件序列或内部状态组合，难以通过常规的测试用例完全覆盖，致使系统长期潜伏于难以预测的脆弱状态之中，最终形成“微小逻辑缺陷累积酿成重大运行风险”的恶性循环。因此，竞态条件问题不仅是 PLC 程序层面的逻辑缺陷，更是工业安全防护体系中的高风险漏洞，其潜在后果从微观的设备损耗到宏观的生产安全、环境生态危机层层递进，凸显了对该问题进行高效检测与彻底治理的极端重要性与紧迫性。

然而，对 PLC 程序中的竞态条件进行有效检测异常困难。其主要原因在于，传统的程序验证方法（如模型检测、定理证明等静态分析技术）在面对竞态条件问题时，常遭遇状态空间爆炸的瓶颈。随着现代工业自动化系统中传感器、执行器数量的快速增长，PLC 程序的规模与逻辑复杂度显著增加，导致其可能的状态空间呈指数级增长，使得完备性分析的计算复杂度难以承受。

与通用计算系统不同，工业 PLC 系统通常运行于资源受限的嵌入式环境，难以通过牺牲部分运行时性能来部署复杂的内存安全或控制流完整性等动态防御机制，这使其更易受到此类底层逻辑漏洞的影响。更为关键的是，若在系统运行过程中才发现竞态问题并采取停机检查措施，将导致巨大的生产损失，其经济与安全风险不可估量。因此，迫切需要在前期的开发与测试阶段，系统地挖掘并消除此类潜在安全隐患，以避免其被恶意攻击者利用。

因此，对 PLC 程序代码实施严格、高效的安全性分析至关重要。然而，现有的一些分析方法通常依赖于高度抽象的数学模型（例如扩展 Petri Net 等，相关知识将在第2章阐述），这类方法要求使用者具备深厚的形式化方法背景与数学建模能力，这在实际工业开发环境中极大地限制了其广泛应用与推广。因此，针对 PLC 程序中竞态条件问题的研究，不仅关乎程序验证理论与技术的突破，更直接关系到工业生产的实际安全与经济效益，具有显著的工程应用价值与社会意义。

1.5 本工作的主要贡献与创新点

本研究聚焦于 PLC 内部逻辑安全的核心挑战——竞态条件漏洞的检测，提出了一套融合静态分析与动态验证的混合检测框架。针对 PLC 程序中竞态条件检测的特殊性，其技术难点主要体现在以下两个方面：

静态分析的预判困境。竞态条件具有显著的时序依赖特征，其在代码的语法和静态结构层面通常缺乏明确表征，因此仅依靠传统的语法分析或控制流/数据流分析难以有效捕捉潜在的逻辑冲突。尽管部分静态分析技术能够在理论上识别部分可疑模式，但随着 PLC 程序复杂度的提升，其分析过程常面临状态空间爆炸问题，导致时间和计算资源成本高昂。而在生产运行阶段通过测试或监控来发现竞态条件，虽能通过触发异常输入暴露问题，却属于事后验证，往往滞后于实际损害的发生。

输入组合的爆炸性复杂度。在真实的工业控制场景中，PLC 程序通常需要处理多路输入信号且逻辑间存在强耦合关系。触发竞态条件的输入序列往往位于巨大的输入空间中的特定路径上。传统的安全分析方法（如符号执行、模型检测）由于难以有效模拟 PLC 与复杂硬件外设的动态交互行为，通常无法高效地遍历这些关键路径。

为解决上述挑战，本研究设计并实现了一个面向工业控制系统、结合静态分析与动态测试的动静态协同模糊测试框架。该框架通过：（1）将 PLC 代码重新编译；（2）对外设的虚拟化建模；（3）静态分析和动态分析方法相结合这三个维度的技术创新，构建了覆盖 PLC 代码的竞态条件问题安全分析的技术体系。

在方法论层面，本工作的创新性主要体现在以下几个维度：

首先，将 PLC 的编程语言在通用计算机平台上重新编译，并应用通用模糊测试框架，实现对 PLC 的源代码级别的分析。该方法摆脱了传统 PLC 的模糊测试对硬件的依赖，可以极大地提升检测效率。

同时，设计并实现了通用的外设交互模拟层。该组件精准地抽象了 PLC 与物理外设（传感器、执行器）的交互接口和行为，有效解决了模糊测试过程中对多样

化传感器信号模拟与执行器状态反馈的实时适配难题。此外，在外设交互模拟层中增加了 PLC 的竞态条件问题监测层，实现对 PLC 执行过程中触发的竞态条件问题的实时监控。

此外，创新性地实现了静态分析与动态模糊测试的深度协同机制。该机制构建了针对 PLC 的竞态条件问题的跨阶段检测链条：在静态分析阶段，识别敏感代码区域和外设接口，并根据分析结果定制模糊测试的种子变异策略；在动态测试阶段，则结合从静态分析获取的引导信息与程序执行覆盖率反馈，进行种子变异。

1.6 论文结构与安排

本章系统性地阐述了本论文的研究背景、核心问题及其理论价值与应用意义。可编程逻辑控制器作为工业控制系统的核心，在工业互联网中处于关键位置。随着工业互联网与智能制造深度融合，PLC 代码安全面临越来越严峻的挑战。其中，竞态条件问题可能引发设备过载、生产流程失控甚至安全事故；然而该问题具有一定的隐蔽性。这充分表明，对 PLC 代码安全，特别是竞态条件问题进行深入研究，具有重要的理论价值和现实紧迫性。

本文后续章节的组织结构如下：

第2章（相关工作）：全面梳理与评述国内外在 PLC 程序分析（特别是竞态条件检测）方面的现有研究方法与技术路线，同时借鉴通用软件安全分析领域（如模糊测试、符号执行）的相关成果，深入剖析现有技术的优势与内在局限性，为本研究工作的定位提供参照。

第3章（方法设计）：详细阐述所提出的动静态协同检测框架的整体架构与核心设计思想。重点介绍其如何通过静态分析与动态测试的协同，应对 PLC 竞态条件检测的挑战，并说明各核心模块的设计原理与创新路径。

第4章（系统实现）：具体介绍原型系统的实现方案与技术细节。包括静态分析模块的关键算法、硬件交互模拟层的构建机制、模糊测试引擎的适配与优化，以及整个协同框架的集成实现。

第5章（实验评估）：设计多组对比实验，基于真实的工业领域 PLC 程序案例集或基准测试程序，对本文所提方法的有效性、效率及可扩展性进行量化评估与深入分析，并与现有方法进行性能对比。

第6章（总结与展望）：系统总结本研究取得的主要成果与创新点，客观讨论当前方法的局限性，并对未来可能的研究方向与应用前景进行展望。

第2章 相关工作与预备知识

本研究聚焦于可编程逻辑控制器（Programmable Logic Controller, PLC）程序中的竞态条件（Race Condition）安全问题。该问题属于控制逻辑缺陷的核心研究领域。根据国际电工委员会（International Electrotechnical Commission, IEC）61131-3标准对 PLC 程序执行机制的规范，此类安全问题源于 PLC 程序的执行过程与周期性扫描机制之间的时序耦合。本文在延续第1章所述 PLC 安全体系架构的基础上，重点探讨代码安全层面的方法论研究。

本研究的基本思路是借鉴通用计算机系统中成熟的软件漏洞挖掘技术（如模糊测试^[24]等），并针对 PLC 代码的特殊性进行适应性改进。本章将首先讨论通用的安全分析方法，包括静态分析与动态分析两大类，以及这些方法在 PLC 代码安全这一特定领域的研究与应用现状。

2.1 静态分析方法

静态分析是一种在不执行程序的情况下，通过分析源代码或编译后代码来评估程序行为的方法。该方法通过解析代码结构、控制流和数据流等，识别代码中的特定模式或异常。这类安全分析技术有助于开发者深入理解代码行为，发现潜在的错误、安全漏洞、代码质量问题以及优化机会。目前，主流的静态分析方法包括形式化验证与符号执行等，这些方法在 PLC 安全分析领域也得到了相应的应用^[25]。

形式化验证是一种基于数学方法的验证技术，用于证明软件、硬件或协议是否满足其形式化规约^[26]。该方法通过严格的逻辑推理（如定理证明和模型检测），将系统行为转化为数学模型（如状态机和逻辑公式），并使用精确的数学语言描述系统需满足的属性。随后，利用自动化或半自动化工具验证模型是否符合规约，或发现违反规约的反例。形式化验证能够确保系统在所有可能情况下均符合预期行为，从而避免依赖传统人工构造测试用例进行测试所带来的抽样性局限^[27]。

具体而言，形式化验证主要包含两种技术路径：**定理证明**与**模型检测**。定理证明将系统行为及其规约转化为数学命题，通过人工引导或半自动化方式构造证明过程，适用于复杂算法验证、编译器正确性证明等场景，常用工具包括 Coq^[28]等。模型检测则通过穷举系统的所有可能状态来检验其是否违反给定规约，适用于死锁检测、硬件电路验证等场景，常用工具有 SPIN 等。

在安全领域，形式化验证技术被广泛应用于密码学算法的正确性验证、智能

合约漏洞检测、操作系统内核的安全保障以及硬件安全属性的验证。然而，形式化验证也存在一定的局限性。例如，定理证明通常需要深厚的专业知识，模型检测则面临状态空间爆炸的问题，且形式化规约本身可能存在错误或不完整性。此外，较高的实施成本也限制了其在大规模复杂系统中的广泛应用。

当前，形式化验证的发展趋势呈现出结合符号执行与模糊测试构建混合验证框架的特点，并引入人工智能技术以辅助证明过程。尽管面临诸多挑战，形式化验证仍是实现高保障安全（如零信任架构）的核心技术之一，在航空航天、区块链及自动驾驶等高可靠性要求领域发挥着不可替代的作用。

在 PLC 安全分析领域，形式化验证方法已得到一定程度的应用。具体而言，形式化验证可用于检验 PLC 梯形图程序的实际执行逻辑是否符合其设计规范^[29]。该方法适用于防御针对 PLC 代码的逻辑篡改攻击，并检测程序逻辑是否偏离预期行为。其中，Petri 网^[30]作为一种重要的数学工具，被广泛应用于离散事件系统的建模、分析与控制。该模型由德国科学家 Carl Adam Petri 于 1962 年提出，其核心构成要素包括：

- **库所**：表示系统的状态或资源（图形化表示为圆圈）。
- **变迁**：表示事件或动作（图形化表示为矩形或短横线）。
- **弧**：连接库所与变迁，表示二者之间的关联关系（图形化表示为有向箭头）。
- **令牌**：位于库所内部，表示资源的数量或系统的当前状态（图形化表示为黑点）。

Petri 网凭借其图形化建模能力，能够直观描述系统的动态行为，并有效表征多个并行发生的事件，因此在 PLC 代码的安全分析中具有重要的应用价值。基于 Petri 网的建模既可用于辅助编写 PLC 代码，也可将现有 PLC 代码转换为 Petri 网模型，进而验证其中是否存在潜在的安全缺陷。例如，Luo 等人^[23]提出了一种基于普通 Petri 网的形式化验证方法，用于建模和检测可编程逻辑控制器梯形图程序中的竞态条件问题。该方法通过系统化的转换算法，将整个梯形图程序自动转换为普通 Petri 网模型。相比扩展 Petri 网（如带抑制弧的 Petri 网），该模型更为直观且易于分析。同时，该方法利用 Petri 网的结构分析技术，将梯形图划分为若干子部分并分别进行验证，显著减少了需检查的状态数量，有效缓解了状态空间爆炸问题，并能够从整体 Petri 网中提取与竞态条件相关的子网，直观展示问题根源，便于工程师进行定位与修正。该工作通过一个化工厂剂量控制系统的实例验证了该方法的有效性。

此外，其他类型的扩展 Petri 网也被应用于 PLC 的安全分析中。其他 PLC 和 Petri 网相关的研究主要集中在如何基于 Petri 网对程序进行建模，然后基于建模结

果来直接构建 PLC 代码。例如, Wightkin 等人提出了一种将顺序功能图转换为时间 Petri 网的方法, 通过引入时间参数来描述 PLC 继电器的特性 (如延时动作)^[31]。Oliveira 等人^[32]则实现了梯形图程序向着色 Petri 网的自动转换。着色 Petri 网支持复杂数据类型, 在建模效率与模型可维护性之间取得了较好的平衡。逻辑变更仅需在代码层进行调整, 无需重构整个模型, 从而降低了维护成本, 拓展了其应用范围。

然而, 基于形式化验证的 PLC 安全分析方法存在若干固有局限性^[29]。首先, 由于缺乏代码上下文信息 (例如, 未充分考虑运行时环境), 可能导致较高的误报率。同时, 分析精度不足 (例如, 对复杂数据流的追踪能力有限) 也会引发漏报风险。其次, 程序中的循环结构与条件分支可能导致分析路径数量呈指数级增长, 分析大型代码库时需消耗大量时间和内存资源。此外, PLC 程序的执行高度依赖外设状态, 外设行为对程序逻辑至关重要, 任何完备的安全分析都必须考虑外设输入数据的影响。然而静态分析技术在精确建模 PLC 与外部系统 (如传感器、执行器等外设) 交互行为方面存在固有困难、若在分析模型中试图完整涵盖此类动态因素, 其复杂程度将远超现有技术工具的处理能力。对于 PLC 应用场景而言, 上述局限性不仅制约了分析结果的可靠性, 也严重限制了静态分析技术在实时性要求严苛的工业环境中的实际应用价值^[23]。

综上所述, 尽管静态分析技术在 PLC 程序安全分析中已得到广泛应用, 但其局限性在实际工业控制场景中尤为突出, 在应对复杂工业控制系统时仍面临诸多挑战。

为突破这些限制, 部分研究者开始探索结合动态分析技术的混合方法, 通过运行时监控和实时数据注入等手段捕捉系统的实际行为, 旨在在保障分析深度的同时提升效率。下一节将重点探讨动态分析技术在 PLC 安全验证中的研究进展与应用现状。

2.2 动态分析方法

软件安全的动态分析 (Dynamic Analysis) 方法是指在程序实际运行过程中, 对其行为和安全性进行评估的技术。该方法通过监控程序的执行流程、输入输出数据、内存状态变化以及系统调用等运行时信息, 识别潜在的安全漏洞或异常行为。在通用计算机软件安全领域, 符号执行 (Symbolic Execution) 与模糊测试 (Fuzzing) 是两种常用的动态分析技术。近年来, 这些方法在可编程逻辑控制器程序的安全分析中也得到了相应的探索与应用。

2.2.1 符号执行

符号执行是一种重要的程序分析技术，通常被视为兼具静态分析与动态分析特征的混合分析方法。该方法通过使用符号值（而非具体输入值）来模拟程序的执行过程。与基于具体输入数据的传统测试不同，符号执行依赖符号化变量和表达式，系统地探索程序的所有潜在执行路径。在分析过程中，每条路径都会生成一个对应的路径条件，即描述该路径被执行所需前提的约束条件集合。通过结合约束求解器（如 Z3、CVC4 等），符号执行能够自动生成满足特定路径条件的测试输入，从而触发程序的深层行为或验证特定属性^[33]。

符号执行技术涉及以下几个核心概念：

1. **符号值**：符号执行使用符号（例如变量 x 、 y ）替代具体的输入值。这些符号值在程序执行过程中持续传播，用于抽象表示程序的中间状态。
2. **路径条件**：每条可行的执行路径均对应一个路径条件，该条件由符号变量和程序中的条件语句（如 `if`、`while` 等）构成的逻辑表达式组成，用以表征执行该路径所必须满足的前提条件。
3. **约束求解**：利用约束求解器对路径条件进行求解，生成能够触发该路径的具体输入值。所生成的测试用例可用于程序的深入测试或正确性验证。

尽管符号执行面临路径爆炸和约束求解复杂性等挑战，但它在软件测试、漏洞检测和程序验证等领域具有重要的应用价值。符号执行在自动化测试和安全分析中的潜力，使其成为现代软件工程中不可或缺的技术之一。例如，在软件测试中，符号执行能够生成高覆盖率的测试用例；在安全分析中，它可以识别缓冲区溢出、空指针解引用等漏洞。然而，符号执行在处理大规模程序时，仍然面临路径爆炸和约束求解复杂性等问题。为了解决这些挑战，研究者们提出了多种优化技术，如路径剪枝、混合执行（结合符号执行与具体执行）以及并行化探索等。

目前，已有研究尝试将符号执行技术应用于可编程逻辑控制器（PLC）的安全性分析领域。例如，Guo 等人提出的 SymPLC 框架^[34]，首次实现了符号执行技术在 PLC 代码自动化测试中的系统化应用。与依赖人工编写测试用例的传统方法不同，SymPLC 能够自动生成高覆盖率的测试输入及任务调度序列。该框架通过整合符号执行与 PLC 程序自动优化技术，显著提升了多任务环境下的测试效率。然而，该方法仍面临状态空间爆炸问题带来的计算挑战。为应对这一挑战，部分研究者开始探索将模糊测试等其他动态分析技术应用于 PLC 安全性分析的可行性。

2.2.2 模糊测试

模糊测试（Fuzzing）是一种自动化的软件安全测试技术，其核心思想是通过向目标程序注入大量随机或半随机的输入数据，观察程序在异常或边界条件下的行为表现，从而发现潜在的崩溃、逻辑缺陷及安全漏洞^[24]。相比依赖预设测试用例的传统方法，模糊测试凭借其大规模自动化执行能力和反馈驱动机制，能够有效揭示常规测试难以触及的深层缺陷，尤其在识别内存破坏、输入验证绕过及异常处理机制失效等安全问题方面具有显著优势^[35]。

模糊测试的标准流程通常始于种子池的初始化。种子池中包含的初始测试用例多为符合程序预期输入格式的合法样本（如规范的文件结构或协议数据包）。在测试过程中，模糊器基于这些种子，通过位翻转、字节替换、数据块插入或删除等变异操作，生成大量新的测试输入。现代模糊测试技术的关键进展在于引入覆盖率导向的反馈机制。该机制通过代码插桩技术，实时收集程序执行过程中的覆盖率信息，包括基本块覆盖率和路径覆盖率。其中，基本块覆盖率衡量程序中线性代码序列的执行情况，而路径覆盖率则反映不同控制流分支组合的执行状况。两者共同刻画测试输入对程序状态空间的探索程度。当某个变异输入触发了新的代码路径或覆盖了此前未执行的基本块时，该输入将被判定为“有效”，并加入种子池，用于指导后续的变异方向，从而形成一个持续优化的闭环反馈过程^[36]。

在测试执行阶段，模糊测试器将生成的测试用例逐一输入目标程序，并密切监控其运行状态，例如是否发生内存访问违规、断言失败或进程异常终止等。对于引发异常的输入，模糊测试器会记录相关的上下文信息，包括调用堆栈、内存状态及覆盖路径等，并借助去重算法筛选出引发独特崩溃模式的样本，以降低后续人工分析的工作量。基于覆盖率反馈的模糊测试通过不断迭代“生成—执行—评估”这一循环，逐步扩大对程序可能行为空间的探索范围。其核心优势在于能够动态识别并保留那些能够触发新程序行为的测试输入，从而克服纯随机测试的盲目性，显著提升漏洞发现的效率与深度^[35]。

总体而言，模糊测试通过构建“生成—执行—反馈”的自适应闭环，形成了一个高效的自动化漏洞挖掘系统。该技术不仅被广泛应用于软件开发生命周期的测试阶段，也成为渗透测试与漏洞赏金计划中的重要工具。随着硬件并行化（如分布式模糊测试架构）和机器学习技术的引入，模糊测试在路径探索效率和输入生成智能化方面持续发展，进一步巩固了其在软件安全动态分析领域的关键地位^[35]。

随着工业控制系统安全研究深入发展，模糊测试技术也逐步被引入可编程逻辑控制器代码安全分析领域，以应对工业控制环境中特有的安全威胁。其中，Siz-
zler^[22]、ICSFuzz^[37] 及 ICS3Fuzzer^[15] 为代表的研究工作，针对 PLC 生态系统中

的多层级攻击面进行了系统性探索。研究者基于不同的威胁假设，分别在梯形图逻辑代码层、PLC 可执行二进制文件层以及监控与数据采集（Supervisory Control And Data Acquisition, SCADA）系统通信层构建了相应的测试框架。

具体而言，Sizzler 主要针对 PLC 的梯形图逻辑代码层进行模糊测试。ICSFuzz 则通过逆向分析 PLC 的运行时环境，对梯形图编译后生成的二进制文件实施输入变异，旨在模拟异常信号或非法指令注入等攻击场景。而 ICS3Fuzzer^[15] 将测试焦点置于 SCADA 系统的通信协议层，重点针对 Modbus/TCP、S7Comm 等典型工业控制协议生成异常通信报文，以探测协议解析过程中的潜在漏洞及协议状态机可能存在的逻辑缺陷。

这些模糊测试方法分别在不同的抽象层次上实施——从应用逻辑、二进制代码到网络通信——共同覆盖了 PLC 系统从代码开发、编译构建到部署运行的全生命周期关键风险点，为构建纵深防御体系提供了重要的技术支撑。

PLC 梯形图逻辑的安全性分析 Sizzler 专注于对可编程逻辑控制器梯形图编译器进行动态安全分析。该研究揭示了传统开发工具链在检测竞态条件（Race Condition）、未初始化输出、无限循环等八类关键安全漏洞方面存在的显著不足。其核心威胁模型假设攻击者可能通过软件供应链污染或工程项目工作站渗透等手段，直接向目标系统注入恶意的梯形图逻辑代码。由于 PLC 程序开发与部署环境通常缺乏严格的版本控制与安全审计机制，此类细微的逻辑篡改极易被伪装成常规的编程失误，进而可能导致严重的工业过程失控（例如，实证研究表明，单个未初始化的输出线圈即可引发平均长达 2.3 秒的系统异常状态）。为验证其分析能力，研究团队构建了一个包含 30 个预设漏洞的基准测试集。实证分析表明，此类漏洞可被攻击者利用实施控制系统劫持、植入持久化后门等高危攻击，其中竞态条件类漏洞尤其可能触发连锁性的安全事件。

在技术实现层面，Sizzler 框架基于开源工具链（如 LDmicro 编译器与 OpenPLC 运行时平台^[38]）实现了从梯形图到 ANSI C 代码的两阶段转换流程。该流程完整涵盖词法分析、语法解析、控制流图推导以及符号表构建等编译环节，最终生成可在目标环境中执行的程序。同时，Sizzler 创新性地构建了一个基于 QEMU 的全系统仿真环境。该环境利用精细的内存映射技术，将针对物理输入/输出（I/O）端口的操作重定向至受控的虚拟地址空间，并通过硬件抽象层（例如对通用输入/输出、内部集成电路等接口的精确仿真）有效突破了传统硬件在环测试中面临的外设行为仿真瓶颈。

为应对 PLC 代码变异空间的高维复杂性挑战，该研究提出了一种基于序列生成对抗网络（SeqGAN）^[39] 的智能模糊测试策略。该策略利用深度强化学习技术构

建指令序列的概率生成模型，能够在确保生成代码语法合法的前提下，对位翻转、字节替换等传统变异操作进行定向优化。实验结果表明，该策略生成的测试用例在路径覆盖率等关键指标上显著优于基于随机或启发式规则的传统模糊测试方法。

实验评估结果表明，该框架通过动态调整外设驱动模型，有效实现了在异构硬件环境下的兼容性测试，显著提升了跨平台适应能力。与 P2IM^[40] 和 μ Emu^[41] 等现有平台相比，该框架在工业总线协议仿真的成功率提升了 37.2%。在漏洞检测能力方面，该框架不仅成功复现了 CVE-2019-18243 等已知漏洞，还新发现了 OpenPLC 运行时环境中的 4 个零日漏洞及一类新型内存越界漏洞（已分配 CVE 编号 CVE-2021-32782）。在美国国家标准与技术研究院（National Institute of Standards and Technology, NIST）软件保障工具与技术评估（Software Assurance Tools and Techniques Evaluation, SATE）IV 基准测试中，该框架实现了 83.6% 的漏洞检出率。

然而，该框架也存在一定的局限性。首先，其依赖的开源工具链与主流商业 PLC 开发环境（如西门子 TIA Portal）的代码生成机制存在显著差异，限制了该框架在商业平台上的直接移植性。其次，全系统仿真方法引入了约 4.8 倍的性能开销，在针对资源受限的微控制器单元进行直接测试时，由于硬件计算能力有限，模糊测试的整体效率受到明显影响。

PLC 固件的安全性分析 针对可编程逻辑控制器可执行二进制文件的安全性分析，是 PLC 安全研究领域的一个关键方向。ICSFuzz 框架^[37]是该领域的代表性成果之一。该研究聚焦于基于 IEC 61131-3 标准编译生成的 PLC 二进制程序的动态安全分析，揭示了编译器优化策略与特定运行时环境交互所引发的深层安全风险。

研究者指出，不同 PLC 编程语言在编译过程中的差异，会显著影响最终二进制代码中漏洞的表现形式。例如，结构化文本因支持指针操作和循环结构，其编译产物中常包含用于内存对齐填充的无操作（No-Operation, NOP）指令。这些 NOP 指令可被攻击者利用，通过重写为程序计数器监控点，为实施基于代码覆盖率的反馈驱动模糊测试提供底层支持。

同时，研究发现，在 PLC 运行时库的核心函数中，如 SysMemCpy、SysMemMove 等，普遍缺乏健全的内存边界检查机制，导致缓冲区溢出和越界写入等内存安全漏洞长期潜伏于工业控制逻辑内部。通过对 CoDeSys 运行时环境进行逆向工程分析，研究者证实攻击者可利用此类漏洞覆盖关键栈帧中的函数返回地址，甚至实现完整的控制流劫持。

例如，在海水淡化等实际工业控制场景中，恶意构造的输入若触发此类内存破坏，可能直接导致生产线非计划停机，造成重大经济损失，这凸显了在二进制层

面进行深度安全分析的现实紧迫性。

在技术实现方面, ICSFuzz 创新性地提出了一种深度集成物理 PLC 设备的模糊测试方法。该框架通过逆向工程解析目标 PLC 固件的精确内存布局及其内部通信协议, 能够直接拦截 CoDeSys 运行时环境中的 KBUS 子系统通信, 将精心构造的测试输入注入控制进程的私有内存空间, 从而绕过传统模糊测试技术对标准化输入接口的依赖。借助动态二进制插桩技术, ICSFuzz 在 WAGO PFC-100 等真实物理 PLC 设备上构建了细粒度的执行轨迹追踪系统, 能够实时捕获堆栈指针异常偏移、非法内存读写等运行时异常行为。

实验验证结果表明, 该方法成功触发了堆溢出、格式化字符串漏洞等共计九类高危安全缺陷。特别是针对比例-积分-微分控制模块的越界写入攻击实例表明, 攻击可在 1.3 秒内突破系统的实时防护机制, 实证了二进制层攻击在工业环境中的可行性。值得注意的是, ICSFuzz 所构建的威胁模型假设攻击者已具备逆向分析或篡改 PLC 二进制文件的能力。鉴于工业现场普遍存在固件更新机制不健全、代码签名验证缺失的情况, 该假设具有高度现实意义。此外, 其基于物理设备验证的漏洞利用路径, 清晰展示了通过覆盖返回地址进而执行特权指令等攻击步骤, 进一步增强了整个攻击链的可操作性和现实威胁性。

然而, 该框架的核心局限在于其对物理硬件环境的高度依赖。尽管基于真实 PLC 设备的测试方案能够精确模拟工业控制系统固有的实时约束(例如扫描周期的严格同步机制和中断响应的确定性延迟), 但这种依赖也直接导致测试成本显著增加, 且整体可扩展性严重受限。硬件依赖不仅使大规模并行模糊测试难以高效部署, 还阻碍了漏洞的自动化复现流程, 以及对不同厂商、不同型号 PLC 平台的跨平台兼容性验证。

此外, 针对 CoDeSys 等专有运行时系统, 其闭源特性导致系统封闭, 限制了研究人员对内部行为进行深入分析的能力。例如, 其关键的实时任务调度算法和精细的内存管理机制尚未被完全逆向解析和拆解。这种“黑盒”状态制约了框架对 Shellcode 注入等涉及复杂控制流篡改的高级攻击进行有效检测的能力。

面向未来, 相关研究需探索将全系统仿真技术与动态污点跟踪分析相结合的技术路径, 或尝试在满足实时性约束的工业控制环境中引入经过优化的轻量级符号执行技术, 旨在最终构建一个不依赖特定物理硬件的通用 PLC 安全测试框架。与此同时, 如何在保持高漏洞检测精度的前提下, 满足工业控制系统对强实时性和确定性的严苛要求, 仍是该领域未来研究亟需突破的关键技术瓶颈。

PLC 监控层的安全分析 在针对 PLC 监控程序层(通常表现为监控与数据采集系统, 即 SCADA 系统)的安全研究中, ICS3Fuzzer^[15] 是一个具有代表性的分析框

架。该项目致力于通过对 SCADA 系统通信协议进行动态模糊测试，以发现潜在漏洞，旨在应对攻击者可能对 PLC 运行程序及其输入/输出数据流实施的实时篡改威胁。该框架采用模块化架构设计，支持对多种厂商的 SCADA 监控软件进行定制化的协议模糊测试。

其核心威胁模型假设攻击者已通过网络渗透或物理接触等方式获取相应权限，能够在 PLC 运行阶段注入恶意控制指令或篡改 I/O 信号流，进而利用 SCADA 协议解析器中的潜在缺陷（例如 Modbus/TCP 等协议的状态机逻辑错误）绕过工业过程控制系统中既有的安全防护机制。

研究人员通过逆向工程分析发现，主流商业 SCADA 软件普遍存在会话状态管理机制松散、对异常协议报文容忍度过高等安全隐患。这使得攻击者可能通过精心构造非预期的协议报文序列，破坏监控系统状态的完整性，甚至触发远程代码执行等严重后果。在技术实现层面，ICS3Fuzzer 提出并实现了一种基于执行轨迹跟踪的状态感知模糊测试策略。该策略通过动态切换协议会话状态（例如，从“设备查询”状态跳转至“参数配置”状态），旨在突破传统黑盒模糊测试在探索复杂协议状态机时的盲目性。

为实现对工业场景真实性的验证，该框架创新性地采用了混合部署模式：既支持通过 OPC UA 等标准化接口与真实 PLC 设备联动，捕获物理 I/O 信号在异常输入下的反馈；也能在纯软件环境中模拟 PLC 的响应报文，构建虚拟测试床，从而在测试覆盖范围与硬件资源消耗之间实现平衡。

然而，该框架的核心局限在于其本质上仍属于黑盒测试范式，缺乏代码覆盖率等细粒度反馈机制。由于 SCADA 系统通常采用闭源架构，且缺乏标准化的动态插桩接口，研究人员难以获取代码基本块覆盖率、执行路径等关键内部反馈信息。这导致测试用例的生成在很大程度上依赖随机或启发式变异策略，从而在探索复杂协议状态空间时面临效率瓶颈。此外，尽管混合部署模式兼顾了测试的真实性与部署的灵活性，但其在大规模工业网络环境中的可扩展性仍受到显著制约：一方面，物理 PLC 设备的异构性带来了高昂的通信协议适配成本；另一方面，纯软件模拟环境难以完全复现真实工业控制系统中存在的硬实时约束（例如微秒级中断响应的精确时序差异）。

2.3 本章小结

本章系统性地探讨了工业控制系统中可编程逻辑控制器安全分析的技术体系，围绕静态分析与动态分析两大核心方法，深入剖析了其理论基础与工程实践。

在静态分析领域，形式化验证技术通过数学建模与逻辑推理，为 PLC 代码的

安全性提供了严密的理论保障。以 Petri 网为代表的形式化工具被广泛应用于梯形图程序逻辑的建模与验证。例如, 普通 Petri 网可用于系统性地检测竞态条件 (Race Condition), 而时间 Petri 网与着色 Petri 网则分别用于刻画程序的时序特性和处理复杂数据流。此类方法能够从数学层面证明程序行为是否符合规约, 尤其在防御逻辑篡改攻击与验证工业流程完整性方面展现出独特价值。

然而, 静态分析在工业现场的实际应用中面临显著挑战。首先, 程序中的循环与条件分支结构容易引发路径爆炸问题, 导致计算复杂度呈指数级增长, 从而使大规模代码的分析效率低下。其次, 对外部设备 (如传感器、执行器) 运行状态的精确建模构成关键瓶颈——PLC 程序的执行逻辑高度依赖于与这些外设的实时交互。若在模型中试图完整涵盖此类动态因素, 其状态空间规模将远超现有分析工具的承载能力。此外, 由于保守性假设产生的误报, 以及复杂数据流追踪能力不足导致的漏报, 进一步削弱了分析结果的可靠性。这些局限性揭示了纯静态方法在应对工业系统强实时性与高环境耦合性方面的固有不足。

动态分析技术通过监控程序运行时行为并注入异常输入, 为 PLC 安全分析开辟了互补路径。在符号执行方面, SymPLC 框架^[34]开创性地将该技术引入 PLC 程序分析, 通过结合符号执行与特定优化策略, 提升了测试效率, 但仍面临状态空间爆炸的挑战。在模糊测试方面, 以 Sizzler^[22]、ICSFuzz^[37]和 ICS3Fuzzer^[15]为代表的框架, 分别针对梯形图源码、PLC 固件及 SCADA 监控程序构建了多层测试体系。Sizzler 通过集成序列生成对抗网络 (SeqGAN)^[39]实现了对高维输入空间的智能探索, 并在 OpenPLC 平台上发现了包括内存越界在内的多个零日漏洞; ICSFuzz 通过直接对 PLC 运行时内存进行注入, 揭示了工业总线协议解析中的深层隐患; 而 ICS3Fuzzer^[15]采用的混合部署模式, 则通过协调真实设备与模拟环境, 有效平衡了测试覆盖率与资源消耗。这些动态方法在漏洞检出率 (如在 NIST SATE IV 基准测试中达到 83.6%) 与实际攻击链复现 (如模拟海水淡化控制系统产线停机) 方面表现出显著优势。

然而, 动态分析同样存在技术瓶颈: 基于 QEMU 的全系统仿真工具 Sizzler 引入了约 4.8 倍的性能开销, 限制了其在严苛实时场景下的适用性; ICSFuzz 对物理硬件的高度依赖阻碍了大规模并行测试和跨平台验证; 针对 SCADA 系统的闭源特性导致覆盖率反馈缺失, 使得测试用例生成容易陷入盲目随机变异, 难以高效探索复杂协议的状态空间。

综合来看, 当前 PLC 安全分析领域已建立较为完善的技术体系, 静态分析和动态分析两类研究方法, 均有研究者探索如何将其迁移至 PLC 安全分析领域。但是, PLC 的安全分析仍面临诸多挑战与研究空白。长期以来, 防护重点主要集中

在固件完整性校验、通信协议加密及访问控制等外围层面，而针对 PLC 代码逻辑本身安全性的系统性研究相对滞后。这一现象源于早期工业网络物理隔离的安全假设，然而，随着工业互联网推动信息技术与运营技术的深度融合，PLC 程序暴露于网络边界，面临通过协议逆向或中间人攻击实施逻辑篡改的威胁，原有的假设已经不再适用，PLC 的代码安全也成为值得关注的领域。在 PLC 竞态条件漏洞的检测与防御方面，尽管已有研究利用形式化方法（如 Petri 网）对并发逻辑进行建模，但难以捕捉由外设响应延迟等动态因素引发的隐性缺陷。另一方面，动态分析技术虽在路径覆盖和实时监控方面具备优势，但其输入生成策略缺乏针对 PLC 指令执行时序特性的优化，导致深层竞态漏洞的检测效率较低。这一研究空白严重制约了高安全需求工业场景下的可靠性保障。

基于对现有范式的批判性分析，本研究提出了一种静态代码分析引导的混合模糊测试框架，旨在构建动静态方法协同的 PLC 代码安全分析体系。该框架通过将静态分析生成的路径约束转化为覆盖率引导的模糊测试定向变异策略，提升以竞态条件为代表的漏洞的触发概率。相比传统动态分析，本框架通过先对 PLC 的代码进行静态分析，再根据静态分析的结果指导模糊测试的种子变异过程，力求减少冗余测试用例，突破性能瓶颈。后续第3章与第4章将详细阐述该方法论的设计与实现。

第3章 PLC 的竞态条件问题安全分析框架设计

正如第2章所言，可编程逻辑控制器的安全分析技术面临两大关键挑战。静态分析方法在效率方面存在明显不足，且未能充分考虑输入输出变量对程序行为的影响，导致分析结果在实际工业场景中的适用性受限。动态分析方法虽然能够通过运行环境进行验证，但缺乏对源代码的直接测试能力，且未针对 PLC 的竞争条件等时序相关问题进行专门优化。此外，现有的针对 PLC 的动态分析主要依赖实际工业硬件，受限于硬件条件，因此难以实现大规模推广和更充分的测试。上述局限性共同导致现有方法难以实现对 PLC 源代码的高效、精准的安全分析，难以满足工业控制系统对可靠性与实时性的严格要求。

为应对现有 PLC 安全分析方法在效率与精度方面的不足，本研究旨在基于 AFL++ 模糊测试框架，构建一个集静态分析与动态验证于一体的混合分析框架，以实现 PLC 源代码中的潜在问题，尤其是竞争问题的高效且精准识别。

3.1 工作目标和威胁模型

本工作的目标主要包括以下两点。

首先，设计并实现一种针对 PLC 源代码的安全分析框架，重点聚焦于竞争条件相关漏洞的识别与精确定位。

其次，该分析框架应能够支持在通用计算机上对 PLC 源码进行深入测试，在保证分析准确性的基础上，摆脱对工业硬件环境的依赖，提升测试效率并降低测试成本，使其达到工业级应用标准，确保所提方法在实际部署环境中的可行性与有效性。

威胁模型假设攻击者能够对 PLC 的源代码发起攻击，或者 PLC 的代码设计者提供了不合理的代码设计方案。

3.2 PLC 的竞态条件问题安全分析框架总体设计

如第2章所述，工业控制系统安全分析领域的静态与动态方法均面临显著的技术瓶颈。静态分析能够通过形式化建模揭示代码中的逻辑缺陷，但对于包含复杂循环结构和多分支路径的可编程逻辑控制器程序，容易引发状态空间爆炸问题，导致工业级代码的分析效率呈指数级下降。此外，静态分析在对外部设备动态行为建模方面存在固有局限：若试图在模型中精确刻画传感器响应延迟、执行器动作

时序等现实因素，模型复杂度将远超现有计算资源的承载能力；而引入简化假设则会导致误报率升高，并可能遗漏关键漏洞。

与此同时，由于 PLC 采用专用编程语言，现有的动态分析工具不具备对 PLC 直接进行源码级别测试的能力。

此外特别值得注意的是，虽然通用的模糊测试的灰盒测试方案：依赖代码的覆盖率反馈机制进行模糊测试的方案，已经被相关研究者用于研究 PLC 的二进制运行时文件，因此采用这种方案进行测试时不受到 PLC 的源码的限制；但是即便如此，该方案仍然不具备检测 PLC 的竞态条件问题的能力。原因在于，在灰盒模糊测试方案中，模糊测试器通过检测程序的崩溃情况，来检测输入数据是否触发了程序漏洞。然而，PLC 的竞态条件问题并不表现为程序崩溃，也就是说模糊测试框架无法感知当前输入是否能够触发 PLC 的竞态条件问题。因此，直接应用原始的模糊测试框架无法检测到 PLC 的竞态条件问题。

为突破上述困境，本研究创新性地提出了一种融合静态分析与模糊测试的混合分析框架，专门用于检测 PLC 中的竞态条件。该框架具有以下三个特点。

首先，该框架首先对 PLC 代码进行跨环境重新编译，将 PLC 的源码重新编译为语义等价的 C++ 语言。一方面，使得编译后的语言可以应用模糊测试框架进行测试，另一方面，也可以利用通用计算机相比 PLC 硬件的高处理能力，提高模糊测试的运行效率。

其次，为了使该框架具备检测 PLC 的竞态条件问题的能力，增加了虚拟外设层，模拟 PLC 的输入输出端口数据。在虚拟外设层处增加竞态条件问题检测层，对 PLC 的竞态条件问题进行实时检测，在检测到竞态条件问题时主动触发漏洞。这样，竞态条件问题可以被模糊测试框架感知，使模糊测试框架具备了检测 PLC 的竞态条件问题的能力。

最后，为了进一步提高模糊测试的测试效率，本工作提出了将静态代码分析和模糊测试框架相结合的方案。首先对 PLC 的代码进行深度语义解析，识别并提取关键变量；随后利用解析所得信息指导模糊测试引擎实施定向变异。该方法将测试资源聚焦于高风险代码区域，从而显著提升时序敏感型漏洞的触发概率。

具体来说，使用该框架对 PLC 代码进行测试，其测试流程如下所示。

1. 第一阶段：语言转换。通过本研究构建的代码转换框架，将采用 PLC 专用编程语言编写的源代码转换为语义等效的 C++ 语言代码，从而使其能够在通用计算平台上运行。
2. 第二阶段：静态分析。分别对原始 PLC 源代码及其转换生成的 C 代码进行静态程序分析，生成用于指导后续动态测试的元数据文件。该文件为模糊测

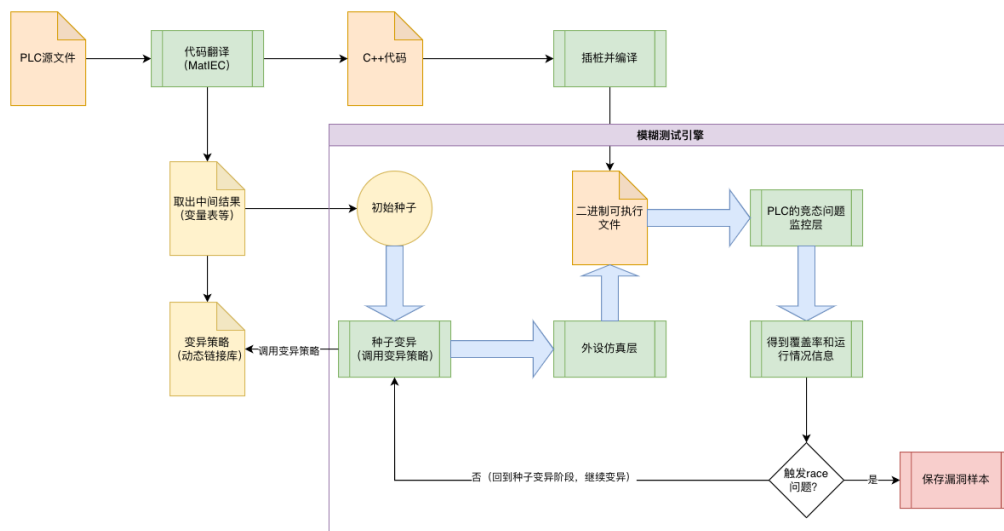


图 3.1 动静态协同的 PLC 安全分析框架

试的种子变异策略提供数据支持。

3. 第三阶段：模糊测试。基于静态分析阶段生成的指导信息，优化模糊测试中的种子选择与变异策略，实现对关键代码路径的定向测试。

本工作在通用计算机语言的 AFL++ 模糊测试框架^[42]的基础上，针对 PLC 的 ST 编程语言的竞态条件问题分析进行了一系列适配工作。接下来的几节中将对框架的设计方案要点进行详细讲解。

3.3 PLC 跨环境重新编译

在 PLC 代码安全测试领域，测试环境的构建策略直接影响检测效率与结果的可靠性。当前主流方案包括真实设备测试与模拟环境测试，二者在保真度与效率方面存在显著权衡。真实设备测试虽能最大程度地复现工业现场工况，但受限于专用硬件的封闭生态，既无法在通用计算机平台上部署成熟的动态分析工具（如内存检测框架 Valgrind^[43]、覆盖率追踪工具 LLVM Sanitizer、模糊测试框架 AFL++^[42]等），也难以满足模糊测试对海量测试用例的高吞吐需求（PLC 设备指令周期通常为毫秒级，而 x86 服务器可达纳秒级）。此外，对 PLC 进行模糊测试离不开外设的状态，而这在物理 PLC 设备上实现成本极高^[44]。为此，本研究创新性地采用 PLC 代码至 x86 架构的跨平台编译方案，通过开源工具链将结构化文本程序转换为 C++ 代码，进而编译为通用计算机上原生可执行的二进制文件，便于在商用服务器集群中搭建高保真模拟测试环境。

相比直接将模糊测试应用于 PLC 硬件设备，将 PLC 程序重新编译并运行于模拟环境中具有以下显著优势^[45]：

1. 可直接复用 x86 架构生态系统中丰富的动态分析工具，无需针对专用 PLC 芯片进行工具链适配，从而实现静态分析、模糊测试等先进技术的快速应用。
2. PLC 的扫描周期通常设定为 50ms。在模拟环境中（即基于通用计算机硬件平台），其运算速度显著提升，因此扫描周期可以大幅缩短，因此可以极大地加快仿真速度。此外，借助通用计算机的硬件资源，能够实现多进程并行测试，从而进一步提高测试效率。该机制可显著提升测试用例的执行速度^[44]。关于具体加速效果的详细分析，将在实验部分5中进一步阐述。
3. 在模拟环境中，可以方便地监测 PLC 程序的内存状态及输入输出情况，从而为竞态条件检测及其他类型漏洞的识别提供更丰富的观测手段，有助于提升 PLC 漏洞检测的可行性与有效性^[45]。

因此，为了提高测试方案的通用性并尽可能提升测试性能，本工作采用了将 PLC 重新编译到模拟环境中进行测试的方案。

基于第1章所述的工业应用趋势分析，PLC 采用的编程语言涵盖 IEC 61131-3 标准定义的五种语言（IL、ST、FBD、LD、SFC）。本研究选择结构化文本作为核心分析对象。主要基于以下技术考量：

1. **技术可行性优势：**结构化文本作为一种文本型编程语言，相较于图形化语言更易于构建形式化的语法规则。结构化文本与 C++ 语言之间的语句转换具有直接的语法对应关系，其理论基础在于 C++ 语言在表达能力上严格包含了 ST 的语言特征集合。具体而言，在基础语句映射方面，ST 语言中的条件分支（如 IF-THEN-ELSE）、循环控制（如 FOR、WHILE）及赋值操作可以直接转换为 C++ 语言中的 if、for、while 及赋值语句，且执行逻辑保持严格一致。在函数库适配方面，鉴于 ST 语言内置函数库（如数学运算和位操作函数）规模有限，仅包含 42 个标准函数，每个 ST 函数均可通过定制的 C++ 函数实现等效功能。因此，原始 PLC 程序中的所有 ST 语句均可通过等效的 C++ 语言结构实现语义上的等价转换，并确保逻辑一致性。
2. **工业应用率提升：**ST 语言的特性使其非常适合复杂算法的应用场景，尤其在复杂控制需求和数据密集型任务中表现出显著优势^[11]。尽管梯形图语言在当前工业环境中仍占主导地位，但在工业互联网应用场景下，ST 语言的重要性日益凸显。这一变化主要得益于工业 4.0 与工业物联网的兴起^[46]。
3. **安全分析的适配性：**ST 语言支持显式变量声明，在 PLC 程序向 C++ 语言转换的编译过程中，会生成一个结构化的变量映射表。该映射表记录了源程序中变量的类型、地址范围及访问权限等关键元数据。通过利用该变量表，可以为模糊测试的种子变异阶段提供语义层面的指导信息，从而提升测试用例

生成的针对性和覆盖效率。具体而言，变量表能够约束变异操作的有效取值范围，避免生成大量无效输入，同时确保变异过程更贴合目标 PLC 程序的运行时环境特性。

4. **可扩展性强：**ST 语言在 PLC 的五种编程语言中表现出较强的表达能力。此外，基于 PLCOpenXML 标准，PLC 的其他几种编程语言可以转换为 PLC 的 ST 语言。因此，解决了 PLC 中 ST 语言的安全性分析问题后，就可在此基础上进一步解决其他语言的安全性分析问题。

因此，本工作确立了针对 PLC 的 ST 语言进行分析的目标，并且采取将其重新编译到模拟环境中的方案来进行安全性分析。

3.4 模拟外设的响应

在 PLC 的安全分析中，外设建模是确保检测准确性和场景真实性的关键环节。由于 PLC 程序的实际运行高度依赖于传感器信号和执行器状态等外部输入，其逻辑执行路径和输出结果往往与外设行为的动态变化紧密耦合。典型的竞态条件问题通常表现为输入信号的时序波动引发输出的非预期振荡；然而，此类振荡并非在所有工况下都会触发。如果任意输入均可诱发振荡，竞态条件的检测将失去技术挑战性。

以第1章所述的简单竞态条件实例为例，该问题并非在所有情况下都会被触发。只有当输入信号 PB1 在特定扫描周期内持续保持高电平时，才会引发竞态条件，进而导致 LED 与 LED2 输出异常跳变。在复杂的工业系统中，触发竞态条件所需的输入模式通常具有非线性特征，因此其检测难度呈指数级增长。

现有基于 Petri 网等形式化方法的静态分析，通常未将外设模型纳入考量，而仅聚焦于 PLC 代码逻辑本身的分析。这种方法存在根本性矛盾：若在静态建模中引入外设的完整状态空间，模型复杂度将呈指数级增长，导致形式化验证的计算开销远远超出实际可承受的范围。

在动态分析的研究中，Sizzler 和 ICS3Fuzzer^[15,22] 主要采用基于真实硬件接口的测试方案，即将外设的输出信号转发至 PLC 程序。然而，该方法存在两个根本性缺陷：其一是经济限制——工业级 I/O 模块的单件成本通常高达数千元人民币，而复杂控制系统需配置多种类型的外设，导致测试平台建设成本呈指数级增长；其二是技术限制——专用硬件接口的电气特性差异导致测试结果难以复现，且在并行测试场景下易出现设备争用问题。

为此，本研究提出将外设建模深度集成到 PLC 安全性分析中，并将其纳入动静结合的混合分析框架。为有效缓解状态空间爆炸问题，本研究在引入外设时，将

其抽象为理想外设模型。针对外设输入构造，本研究提出一种方案，使模糊测试工具能够根据实际执行情况动态模拟外设响应，从而进一步提高代码覆盖率。本研究的重点在于使外设行为尽可能触发异常，因此无需精确模拟外设的工作机制，仅需探索可能引发 PLC 代码异常的外设行为。此外，该设计亦旨在提升测试效率。

具体而言，本研究通过构建**虚拟外设仿真层**，实时模拟传感器信号的生成及执行器状态的反馈；同时，结合覆盖率引导的模糊测试机制，动态调整输入信号的时序与端口组合。该方法在测试过程中持续监控 PLC 输出与外设响应之间的交互关系，并利用反馈数据自适应地优化测试用例的时序扰动，从而在可控的计算资源范围内，尽可能高效地触发竞态条件漏洞。

相比于静态模型的理想化假设，基于软件建模的外设虚拟化方案能够在仿真环境中模拟外设的输入，不仅优化了成本效益，还保证了测试的可扩展性和并行支持，从而显著提升了测试效率（详见第5章）。

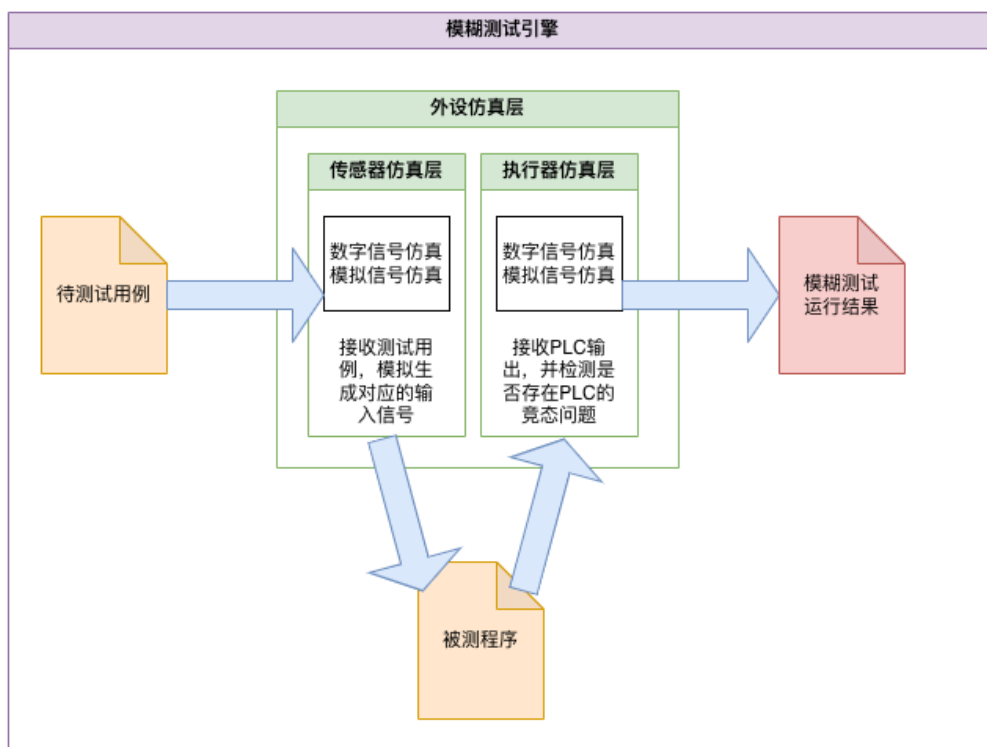


图 3.2 虚拟外设仿真层架构图

在已有外设建模的基础上，本研究将外设模型与模糊测试流程深度融合，通过自定义语法规则指导 AFL++ 框架的种子生成过程。PLC 程序的输入输出机制具有典型的工业控制系统特征：输入信号来源于传感器等外部检测设备，输出信号则作用于 LED 指示灯等执行单元。由于 PLC 模糊测试中的种子变异实质上是对外设行为模式的修改，为避免纯随机变异导致路径探索效率降低，本研究通过 AFL++

的规则定制接口显式传递外设建模的结构化语义约束。该方法可确保生成的测试用例（即外设模拟数据）始终符合预设的模型规范，既维持了变异的多样性，又有效提升了代码覆盖率和路径探索的针对性。

3.5 静态分析与动态分析的协同的安全分析框架

正如第2章所述，单纯采用静态分析方法对 PLC 代码进行建模分析时，存在时间复杂度呈指数级增长的问题。因此，本研究主要采用动态分析方法，旨在构建更高效的 PLC 代码模糊测试框架。在上一节中，已展示了对外设的模拟方案及其与 AFL++ 模糊测试框架结合的实现方案。

然而，即使在完成此建模的基础上，变异后的种子数据结构完全符合外设的建模规范，仅依赖动态分析的随机变异和基于代码覆盖率引导的变异方案，仍然存在输入数据种子变异空间过大、变异效率低下的局限性。

原因主要在于，在基于覆盖率引导的 AFL++ 模糊测试框架中，模糊测试器会在编译代码的过程中对程序插桩，在程序执行过程中统计程序的运行状态，根据基本块覆盖率或程序基本块路径覆盖率对种子进行变异。但是在 PLC 场景下，程序覆盖率能够反映的程序行为较为有限。代码覆盖率仍然能够在一定程度上反映程序的执行情况；但是 PLC 的竞态条件问题的特征不仅仅依靠覆盖率体现，而是体现在输出数据的波动。即使程序的执行了相同的代码逻辑，也可能在竞态条件问题检测方面得到不同的结论。而这一特性使得输入空间（例如传感器引脚状态的组合）达到 2^n 级别，叠加外设时序变量后，实际输入空间更加庞大且复杂。传统基于覆盖率引导的模糊测试方案因难以通过覆盖率反馈有效指导种子变异。

为此，本研究在数据外设建模的基础上，为了提高模糊测试的种子变异效率，进一步提出了动静结合的混合变异方法。通过构建精确的输入访问模型来指导种子变异过程，在保证分析质量的前提下，显著提升了测试效率。

具体实现方案是在模糊测试的预处理阶段，在从 PLC 的 ST 语言翻译为 C++ 语言的过程中生成变量映射表。通过该变量映射表，可以识别输入数据（即 PLC 输入端口）中对程序执行流程具有关键影响的数据项。生成的变量映射表将指导模糊测试种子变异器优先变异关键数据位置，并将关键结果进行存储。

在此变量映射表的基础上，本工作设计的自定义种子变异器能够基于先前的静态分析结果，动态调整输入数据的变异策略。变异过程中采用选择性编译机制，仅对与程序执行逻辑相关的变量进行变异，忽略无关变量；并根据不同类型数据结构的特性，分别采用针对性的变异方案。自定义种子变异器以动态链接库形式集成于 AFL++ 模糊测试框架，通过标准化的 API 接口与框架交互，在保持原有架

构完整性的基础上，有效扩展了变异能力该实现不仅完全兼容 AFL++ 现有的变异策略，还引入了针对可编程逻辑控制器的专用变异操作，作为补充的变异阶段。

3.6 动态运行状态监控与异常检测

在 PLC 程序的安全性分析中，输出结果的合规性验证是检测逻辑缺陷的关键环节。尤其对于竞态条件这类隐蔽性漏洞，其危害往往仅在程序运行时显现——即便代码本身未引发语法错误或系统崩溃，输出信号的异常振荡（如继电器频繁切换、阀门启停周期紊乱）仍可能导致生产流程失控。

然而，在 PLC 模糊测试过程中，竞态条件问题的检测面临独特挑战：与通用软件漏洞不同，此类问题触发时通常不会导致程序崩溃，而是表现为输出信号的持续振荡。这使得传统基于崩溃检测的模糊测试框架（如 AFL++）难以有效识别异常，因为它们主要依赖进程监控机制（如 fork/waitpid）和内存检测工具（如 AddressSanitizer）来识别程序崩溃或内存安全漏洞，无法检测 PLC 运行时出现的功能性异常。

为应对上述挑战，本研究设计的动态分析框架创新性地集成了程序运行时状态监控框架。通过在 PLC 虚拟执行环境中嵌入轻量级 PLC 的竞态条件问题检测模块，实时采集变量状态、I/O 端口信号及扫描周期时序等关键运行时数据，完整记录每个扫描周期的状态变化，并根据其结果分析是否触发了竞态条件问题。

在检测机制方面，框架采用双缓冲区架构，输入端口和输出端口分别配置容量为 100 个扫描周期的环形缓冲区，用于记录历史数据。在每个扫描周期结束时，检测算法会对缓冲区中的数据进行分析：当输入数据保持稳定而输出数据出现异常跳变（变化量超过预设阈值）时，系统将触发异常警报。通过向 PLC 模拟程序注入特定异常信号，使 AFL++ 模糊测试框架能够准确识别此类功能性异常状态。此外，竞态条件问题检测模块记录的时序日志也可以辅助重构漏洞触发前后的 PLC 程序执行状态，为竞态条件的进一步定位和根源分析提供高粒度的数据支撑。

竞态条件问题检测模块盒主动异常注入机制，有效解决了传统崩溃检测方法在 PLC 的竞态条件问题检测方面的不足，实现了对竞态条件引发输出振荡的精准捕获。

3.7 PLC 的竞态条件问题模糊测试框架

本研究介绍了一个面向 PLC 代码竞态条件问题的安全分析框架的设计，其中，在对 PLC 的安全分析以 AFL++ 模糊测试框架为基础实现。如第2章所述，通用计

算领域的模糊测试技术已较为成熟，但其检测机制主要针对程序崩溃等传统异常模式。针对 PLC 环境中特有的竞态条件安全问题，本研究基于 AFL++ 模糊测试框架，对这一模糊测试通用框架在 PLC 的竞态条件问题的检测方面进行了多维度的适配与增强。

从模糊测试的整体执行流程的角度来看，一个完整的模糊测试系统应包含三个核心组件：程序输入输出处理机制、测试用例生成与变异策略，以及异常检测与反馈系统。图3.3展示了本研究所提出的动静态协同分析框架的整体工作流程。

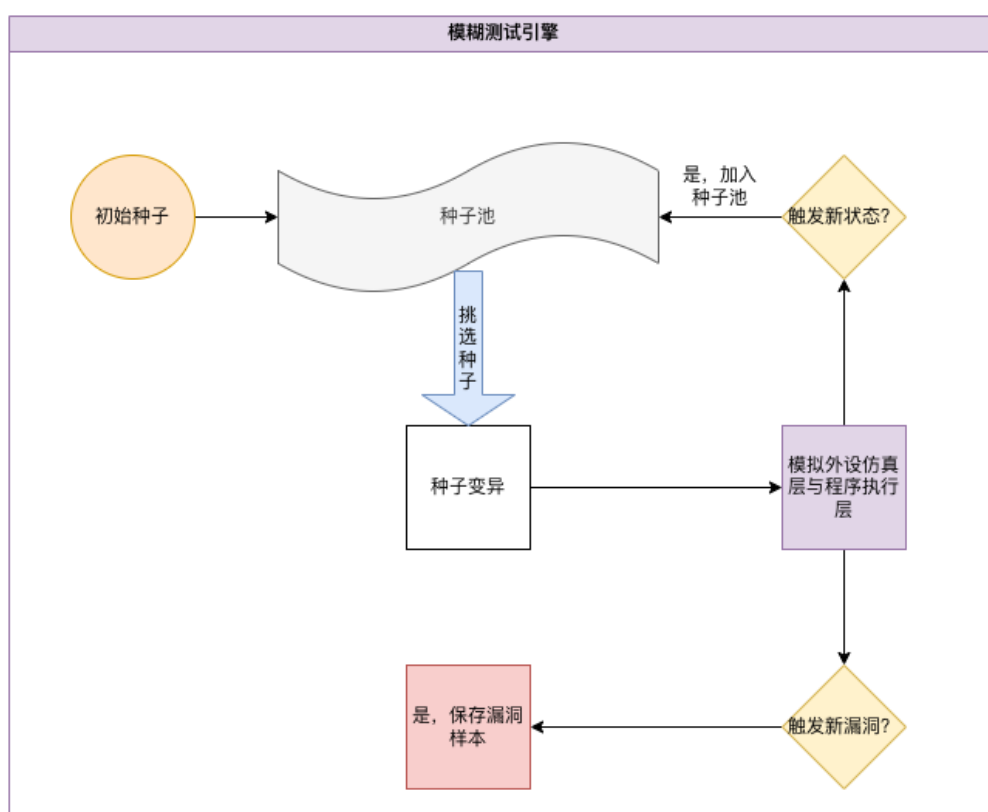


图 3.3 模糊测试执行流程

该框架是基于通用计算机领域的模糊测试框架进行的重要改进，具体改进点有：

1. **PLC 代码适配：**模糊测试框架仅支持测试 C++ 等通用计算机语言的代码。本工作将 PLC 代码先转换为 C++ 代码，随后进行重新插桩和编译，以便于后续的测试与分析。
2. **输入输出适配层：**针对 PLC 的 I/O 映像区特性，设计了虚拟外设仿真层，对数字量与模拟量实现统一的处理接口。
3. **变异策略优化：**结合对 PLC 的代码进行静态分析获得的变量映射表，实现基于程序语义的智能变异策略。

4. 异常检测机制：通过运行时监控模块捕捉输出振荡等 PLC 特有的异常模式。

本章系统性地设计了一个面向可编程逻辑控制器竞态条件漏洞检测的混合分析框架，结合静态分析技术与动态模糊测试技术，旨在应对传统方法在复杂工业场景下的适应性挑战。首先，将 PLC 程序重新编译并运行于模拟环境中，实现了结构化文本到 C++ 代码的语义保持转换，突破了 IEC 61131-3 语言与通用模糊测试工具之间的适配障碍，构建了可在通用计算环境中执行的仿真环境。然后，构建虚拟外设仿真层来模拟物理外设的行为，并根据模糊测试的反馈数据来模拟外设的响应。此外，结合静态分析和动态分析技术，将静态代码分析结果用于引导 PLC 的模糊测试种子变异过程，通过提取关键变量引导模糊测试引擎对高危代码区域实施定向变异，有效缩减了输入空间的指数级增长范围。最后，嵌入的 PLC 竞态条件问题监控模块实时监测竞态条件。上述技术路径系统性地解决了 PLC 的代码安全分析领域的竞态条件问题检测特有的技术挑战，为后续实验验证奠定了坚实基础。下一章将介绍本方案的具体实现。

第4章 具体实现

本研究为了解决现有 PLC 的竞态条件问题安全分析方法在效率与精度方面的不足,旨在构建一个集静态分析与动态验证于一体的混合分析框架,以实现 PLC 源代码中的竞态条件问题的高效且精准识别。

该框架的总体设计已经在第3章中完成介绍,本章将介绍具体实现。根据本工作的方案设计,对 PLC 代码进行安全性分析,可以分为以下几个阶段。

1. 第一阶段: PLC 跨环境重新编译。为了在模拟环境中执行 PLC 的竞态条件问题的安全性分析,本研究将采用 PLC 专用编程语言编写的源代码转换为语义等效的 C 语言代码,从而使其能够在通用计算平台上运行,提高测试效率,方便进行进一步的分析。
2. 第二阶段: 模拟外设响应。构建虚拟外设层来模拟外设对 PLC 的相应,并将其深度集成到 PLC 安全性分析中。
3. 第三阶段: 静态分析与模糊测试结合分析。对原始 PLC 源代码进行静态程序分析,生成用于指导后续动态测试的元数据文件,并基于此分析结果,优化模糊测试中的种子选择与变异策略,实现对关键代码路径的定向测试。
4. 第四阶段: 动态运行状态监控与异常检测。通过在每个扫描周期结束前执行输出稳定性检测,实现动态监控。

具体实现方面,本工作以通用计算机语言的 AFL++ 模糊测试框架作为安全分析框架的基础,并在此基础上针对 PLC 编程语言进行了系列适配工作。本章接下来的几节将详细讲解具体的实现方案。

4.1 PLC 跨环境重新编译

根据第3章所述,为克服真实设备测试的固有局限,本工作采用在模拟环境中执行 PLC 代码安全测试的方案。为此,需要将 PLC 的代码重新编译到模拟环境中。本工作基于 MatIEC 开源框架,本工作将 PLC 代码转换功能集成到了 PLC 模糊测试框架。

4.1.1 跨环境重新编译的相关工作介绍

在具体的重新编译方案中,本研究选取了 MatIEC 编译器作为原型,并将其集成至模糊测试系统。MatIEC 是一款开源的 IEC 61131-3 编译器框架,其核心目标是将标准 PLC 编程语言转换为可移植的 C 代码或机器指令。因此,本研究将这一

工业控制领域的开源编译器集成到研究框架中。

该框架采用典型的三阶段编译模型。各阶段的技术实现细节如下：

1. 词法分析与语法分析阶段

源代码首先通过 Flex 词法分析器进行词法单元的切分，生成标记流；随后，Bison 语法分析器根据 IEC 61131-3 标准定义的文法规则构建抽象语法树。该阶段确保源程序的结构符合语言规范，输出结果为具有完整语法结构的 AST 表示。

2. 语义分析阶段

编译器执行深度语义检查：

- 类型系统验证：检查表达式与变量的数据类型是否匹配。
- 函数调用校验：验证参数的数量和类型是否与函数声明一致。
- 作用域分析：确定变量和函数的可见范围及其访问规则。

此阶段输出的增强型抽象语法树包含完整的语义标注信息。

3. 目标代码生成阶段

将经过语义验证的抽象语法树转换为等价的 C++ 代码时，转换过程严格遵循以下原则：

- 语义等价性：保持原始 PLC 程序的所有功能语义不变。
- 结构可追溯性：保持控制流与数据流之间的对应关系。
- 可扩展性：生成的代码支持后续插入检测探针。

该编译流程通过分层验证机制（语法-语义-代码生成）确保转换的可靠性；同时，生成的 C++ 代码保留了 PLC 特有的时序特性标记，为后续混合测试提供了具备工业控制语义的基础代码结构。

4.1.2 兼容性问题改进

本研究的改进主要集中在前端编译模块。MatIEC 工具基于 2003 年发布的 IEC 61131-3 第二版语法规则实现，然而在过去二十年间，该标准经历了多次版本更新并引入了新的语法特性。同时，MatIEC 支持的语法与当前主流 PLC 编译平台存在一定差异。这些差异导致 MatIEC 在编译现代 PLC 项目时可能出现语法兼容性问题。

以变量类型定义为例。MatIEC 关于变量定义语法分析器中进行了如下定义：。MatIEC 允许定义的变量类型有三种，分别是 input, output, input_output。

```
1 io_var_declarations :  
2   input_declarations  
3   | output_declarations  
4   | input_output_declarations
```

```
6 ;
```

以其中 `input_output` 类型的变量为例，这种变量应当在变量定义的开头和结尾加入 `VAR_IN_OUT` 和 `END_VAR`。

```
1
2 input_output_declarations:
3   VAR_IN_OUT var_declaration_list END_VAR
4   { $$ = new input_output_declarations_c($2, locloc(@$)); }
5   /* ERROR_CHECK_BEGIN */
6   .....
7   /* ERROR_CHECK_END */
8 ;
```

可以声明多个变量，每个变量声明必须以分号结尾。

```
1 /* helper symbol for input_output_declarations */
2 var_declaration_list:
3   var_declaration ','
4   { $$ = new var_declaration_list_c(locloc(@$)); $$->add_element($1); }
5 | var_declaration_list var_declaration ','
6   { $$ = $1; $$->add_element($2); }
7   /* ERROR_CHECK_BEGIN */
8   .....
9   /* ERROR_CHECK_END */
10 ;
```

变量类型可以有很多种。

```
1 var_declaration:
2   temp_var_decl
3 | fb_name_decl
4 ;
5
6 temp_var_decl:
7   var1_declaration
8 | array_var_declaration
9 | structured_var_declaration
10 | string_var_declaration
11 ;
12
13 var1_declaration:
14   var1_list ':' simple_specification
15   { $$ = new var1_init_decl_c($1, $3, locloc(@$)); }
16 | var1_list ':' subrange_specification
17   { $$ = new var1_init_decl_c($1, $3, locloc(@$)); }
18 | var1_list ':' enumerated_specification
19   { $$ = new var1_init_decl_c($1, $3, locloc(@$)); }
20 | var1_list ':' ref_spec /* defined in IEC 61131-3 v3 (REF_TO ...) */
21   { $$ = new var1_init_decl_c($1, $3, locloc(@$)); }
22 /* ERROR_CHECK_BEGIN */
23 .....
24 /* ERROR_CHECK_END */
25 ;
```

MatIEC 中的这些规定符合 IEC 61131-3 标准中对编程语言语义的定义，但仅是该标准允许的多种实现之一，因为该标准并未严格规范具体语法。这种灵活性导致该编译器与不同工业 PLC 编译器在结构化文本的实现上存在语法差异。

以另一个 PLC 编译平台 Codesys 为例，同样是关于函数变量声明，在 Codesys 中使用的和在 MatIEC 中并不相同。此外，西门子、罗克韦尔自动化、三菱等使用的语法也都各有自己的特点。这是由于历史原因导致的：各大厂商为了抢占市场份额，各自独立开发了自己的硬件体系、操作系统和配套的编程软件，是技术竞争和商业策略的体现。但是，这就使得为特定平台编写的结构化文本代码。往往无法直接在 MatIEC 中编译，成为跨平台移植的主要障碍。此外，第5章所提到的 G4LTL-ST 工具给出的 ST 语言也与 Matiec 不兼容。

本工作开发了 PLC 的 ST 语言适配的转换脚本。主要的改进点有：

1. 变量的定义语法等问题
2. 部分关键字有区别
3. G4LTL-ST 工具生成的 ST 语言没有指定引脚

集成上述方案后，系统能够处理来自多种开发环境，甚至不同编程语言的 PLC 代码，显著提升了本研究的通用性。经转换生成的符合 ANSI C 标准的代码，为后续在仿真环境中进行高精度测试奠定了坚实基础。

此外，MatIEC 原生仅支持结构化文本语言，无法处理梯形图等图形化语言。不过，尽管梯形图语言与结构化文本语言在语义上并非完全等价，尤其是在底层硬件互连逻辑方面存在差异，但其核心控制流程通常可以转换为结构化文本语句来表达。

而 PLCOpen 组织为了解决不同厂商的语言标准各异的问题，推动了 PLCOpenXML 标准的建立。PLCOpenXML 是一种基于 XML 的国际标准文件格式，它旨在让不同厂商的 PLC 编程软件之间能够交换和共享程序逻辑。因为该标准是付费文件，所以本工作暂时不具备将基于这一标准的代码组件集成至模糊测试框架的条件。但是，可以利用现有的厂商提供的代码转化工具来解决兼容性问题。现有的主流的 PLC 开发软件都提供将 PLC 的源码导出至 PLCOpenXML 的功能，而 OpenPLC 提供了将 PLCOpenXML 支持的编程语言编译至 Matiec 可识别的 ST 语言的工具。因此，只要能够用 PLCOpenxml 表示的 PLC 程序，都可以通过本框架进行模糊测试。不过由于技术专利限制代码格式转换过程难以实现自动化，因此此方案暂时不具备大规模测试的条件。

4.1.3 C++ 源码到模糊测试插桩

经过上一节所述的工作，已经获得了一份 C++ 代码。在未引入 AFL++ 模糊测试框架的情况下，该代码能够成功编译，并在 x86 等通用计算机平台上原生运行。然而，由于 PLC 与通用计算机的架构存在差异，在模糊测试环境中，代码未能达到预期效果。

为什么会出现这种情况呢？这与重新编译过程中目标代码生成阶段密切相关。为了确保可编程逻辑控制器进程在每个扫描周期内保持稳定的执行性能，本研究实施了多项实时性优化措施。首先，通过 CPU 隔离技术将目标进程绑定到特定计算核心，有效避免了通用计算机平台上其他进程对资源的干扰；随后，采用 Linux 系统的 `SCHED_FIFO` 实时调度策略，并赋予该进程最高优先级，确保进程一旦就绪即可立即抢占当前核心上的其他任务；此外，基于该调度策略无时间片限制的特性，进程将持续运行，直到主动释放 CPU 或被更高优先级的进程抢占。由于主进程被设置为最高优先级，其执行流程几乎不会被中断，从而为扫描周期的稳定性提供了系统级保障。经过上述优化配置后，PLC 模拟进程在通用计算机的原生环境下能够维持微秒级的扫描周期精度。

然而，当引入模糊测试活动时，系统的实时性将面临严峻挑战。其原因在于，AFL++ 模糊测试框架采用多进程并发模型，会与上述设置的运行时进程在 CPU 时间片、缓存及内存带宽等资源上产生激烈竞争，进而引发不可预测的调度延迟。同时，频繁的进程创建与销毁操作、系统调用及中断处理通过系统总线影响所有 CPU 核心，这种跨核心的干扰导致 OpenPLC 扫描周期出现百微秒级的时序抖动，严重影响控制系统的实时性要求。该优化措施与实际负载之间的矛盾，揭示了软硬件协同设计中资源竞争问题的复杂性，也为后续优化方案的设计提供了重要依据。

为解决上述实时性干扰问题，本研究提出了一种基于资源隔离的优化方案。该方案以“空间分离”和“干扰最小化”为核心设计原则，通过硬件资源与软件调度的协同配置，在物理层面和逻辑层面实现模糊测试活动与 PLC 运行环境的有效隔离。其核心创新在于对 AFL++ 模糊测试框架及其子进程实施精细化的资源分配与控制，从而在保障测试效率的同时，最大限度地维护 PLC 运行的稳定性。

这种空间隔离策略带来了双重效益：一方面，通过将测试负载分散到多个计算核心，有效避免了所有测试实例集中于单一核心所引发的资源竞争问题，充分发挥了多核处理器的并行计算潜力；另一方面，由于 PLC 测试实例与模糊测试主控进程实现了物理隔离，显著降低了调度延迟和内存带宽竞争对 PLC 扫描周期的干扰。实验表明，该方案在维持模糊测试吞吐量的同时，能够将 PLC 扫描周期的时序抖动控制在微秒级范围内，为工业控制系统的模糊测试提供了可靠的实时性保障。

4.2 模拟外设的响应

为了更好地对 PLC 的竞态条件问题进行安全性分析，本研究提出将外设建模深度集成到 PLC 安全性分析中，并将其纳入动静结合的混合分析框架。在既有研究

中, Pretender^[47]、AutoMap^[48]等工具通过探测真实外设对 I/O 请求的响应, 基于启发式规则学习外设模型, 进而在仿真环境中替代真实硬件。而 P2IM^[40]、 μ Emu^[41]、Fuzzware^[49]等方案仅需分析固件, 并在仿真时构造内存映射 I/O (MMIO) 响应, 使固件能够正常运行, 无需依赖真实外设的硬件行为。此类方案旨在最大化提升被测固件的代码覆盖率。

本研究重点关注输入稳定时输出是否产生异常行为, 故无需精确模拟外设工作机制, 仅需探索可能引发 PLC 代码异常的外设行为即可。因此, 本工作的外设模拟方案更加接近后者的思路。在引入虚拟外设层时, 将其抽象为理想外设模型。模糊测试工具能够根据实际执行情况动态模拟外设响应, 使模糊测试能够深入探测代码路径, 从而进一步提高代码覆盖率。尽管该方法可能生成不符合实际外设行为的输入, 但能够有效发现潜在异常。在完成外设建模后, 为避免纯随机变异导致路径探索效率降低, 本研究通过 AFL++ 的规则定制接口显式传递外设建模的结构化语义约束, 使得构造的外设相应符合建模的接口。

4.2.1 模糊测试的外设建模方案

PLC 程序的 I/O 映像区包含数字量和模拟量等多种类型的数据区域。具体而言, 输入映像区可细分为位 (bit)、字节 (byte)、字 (word) 和双字 (double word) 等不同的数据类型。

不同的物理外设会映射到不同的输入映射区, 具体接口情况如下表所示。

因此, 本工作针对不同数据类型的外设分别建模; 本质上是在模拟测试的环境下, 直接模拟输入影响区的数据。下面展示的是对外设建模的代码实现。

```

1 #pragma once
2
3 #include <iostream>
4
5 #include "ladder.h"
6
7 class SuperBasicInputBlock {};
8
9 template <typename T, int Dim = 1>
10 class BasicInputBlock : public SuperBasicInputBlock {
11     public:
12         int cycles;
13         T input[BUFFER_SIZE];
14         BasicInputBlock() {
15             cycles = 0;
16             for (int i = 0; i < BUFFER_SIZE; i++) {
17                 input[i] = 0;
18             }
19         }
20         friend std::istream& operator>>(std::istream& is, BasicInputBlock<T, Dim>&
           obj) {
21             is >> obj.cycles;
22

```

表 4.1 PLC 数据映像区接口类型分类

接口类型	数据长度 (位)	数据数量	数值范围	存储结构描述
布尔块	8	64	0-255 (每 8 位为一个单元)	由 1 个头部标识符 (范围 1-10000) 和 64 个 8 位无符号整数组成, 每个整数表示一个布尔状态。
字节块	8	8	0-255	头部标识符后接 8 个 8 位无符号整数, 每个整数对应一个字节数据。
整型块	16	8	0-65535	头部标识符后接 8 个 16 位无符号整数, 用于存储短整型数据。
双整型块	32	8	$0-2^{32}-1$	头部标识符后接 8 个 32 位无符号整数, 用于存储长整型数据。
长整型块	64	8	$0-2^{64}-1$	头部标识符后接 8 个 64 位无符号整数, 支持极大整数范围。
整型内存块	16	8	0-65535	结构同整型块, 但专用于内存映射区域, 可能涉及地址连续存储。
双整型内存块	32	8	$0-2^{32}-1$	结构同双整型块, 但针对内存映射优化, 支持高效数据访问。

```

23         for (int i = 0; i < BUFFER_SIZE; i++) {
24             uint64_t tmp;
25             is >> tmp;
26             obj.input[i] = (T)tmp;
27         }
28         return is;
29     }
30     virtual void print() {
31         std::cout << "BasicInputBlock<" << typeid(T).name() << ", " << Dim <<
32         ">" << std::endl;
33         std::cout << "cycles: " << cycles << std::endl;
34         for (int i = 0; i < BUFFER_SIZE; i++) {
35             std::cout << input[i] << " ";
36         }
37         std::cout << std::endl;
38     };
39     virtual std::string serialize_data() const {
40         std::string buffer;
41         buffer += std::to_string(this->cycles);
42         for (int i = 0; i < BUFFER_SIZE; i++) {
43             buffer += std::to_string(this->input[i]);
44         }
45         return buffer;
46     }

```

```

47     size_t size() const {
48         return sizeof(cycles) +          // int类型的大小
49             sizeof(T) * BUFFER_SIZE;    // 一维数组的大小
50     }
51 };
52
53 template <typename T>
54 class BasicInputBlock<T, 2> {
55     public:
56         int cycles;
57         T input[BUFFER_SIZE][8];
58         BasicInputBlock() {
59             cycles = 0;
60             for (int i = 0; i < BUFFER_SIZE; i++) {
61                 for (int j = 0; j < 8; j++) {
62                     input[i][j] = 0;
63                 }
64             }
65         }
66         friend std::istream& operator>>(std::istream& is, BasicInputBlock<T, 2>&
        obj) {
67             is >> obj.cycles;
68             // std::cout << "cycles=" << obj.cycles << std::endl;
69             for (int i = 0; i < BUFFER_SIZE; i++) {
70                 for (int j = 0; j < 8; j++) {
71                     uint64_t tmp;
72                     is >> tmp;
73                     // std::cout << "i=" << i << " j=" << j << " tmp=" << tmp <<
74                     std::endl;
75                     obj.input[i][j] = (T)tmp;
76                     // std::cout << "i=" << i << " j=" << j << " input=" << obj.
77                     input[i][j] << std::endl;
78                 }
79             }
80             return is;
81         }
82         virtual void print() {
83             std::cout << "BasicInputBlock<" << typeid(T).name() << ">" << std::
84             endl;
85             std::cout << "cycles: " << cycles << std::endl;
86             for (int i = 0; i < BUFFER_SIZE; i++) {
87                 for (int j = 0; j < 8; j++) {
88                     std::cout << (uint64_t)input[i][j] << " ";
89                 }
90                 std::cout << std::endl;
91             }
92             std::cout << std::endl;
93         }
94         virtual std::string serialize_data() const {
95             std::string buffer;
96             buffer += std::to_string(this->cycles);
97             for (int i = 0; i < BUFFER_SIZE; i++) {
98                 for (int j = 0; j < 8; j++) {
99                     buffer += std::to_string(this->input[i][j]);
100                 }
101             }
102             return buffer;
103         }
104     size_t size() const {
105         return sizeof(cycles) +          // int类型的大小
106             sizeof(T) * BUFFER_SIZE * 8; // 一维数组的大小

```

```

105     }
106 };
107
108 class BoolBlock : public BasicInputBlock<unsigned char, 2> {};
109
110 class ByteBlock : public BasicInputBlock<IEC_BYTE, 1> {};
111
112 class IntBlock : public BasicInputBlock<IEC_UINT, 1> {};
113
114 class DIntBlock : public BasicInputBlock<IEC_UDINT, 1> {};
115
116 class LIntBlock : public BasicInputBlock<IEC_ULINT, 1> {};
117
118 class IntMemoryBlock : public BasicInputBlock<IEC_UINT, 1> {};
119
120 class DIntMemoryBlock : public BasicInputBlock<IEC_UDINT, 1> {};

```

4.2.2 融入模糊测试框架

在已有外设建模的基础上，本研究将外设模型与模糊测试流程深度融合，通过自定义语法规则指导 AFL++ 框架的种子生成过程。PLC 程序的输入输出机制具有典型的工业控制系统特征：输入信号来源于传感器等外部检测设备，输出信号则作用于 LED 指示灯等执行单元。而在构建了外设仿真层后，PLC 模糊测试中的种子变异，实质上就成为了对外设行为模式的修改。

为避免纯随机变异导致路径探索效率降低，本研究通过 AFL++ 的规则定制接口显式传递外设建模的结构化语义约束。该方法可确保生成的测试用例（即外设模拟数据）始终符合预设的模型规范，避免因种子不符合模型规范而导致代码路径探索深度不足的问题。既维持了变异的多样性，又有效提升了代码覆盖率和路径探索的针对性。

```

1  start          = bigblocks
2  bigblocks      = bigblock | bigblocks bigblock
3  bigblock       = "[BIGBLOCK]" blocks "[ENDBLOCK]"
4  blocks         = bool_block byte_block int_block dint_block lint_block
5                  int_mem_block dint_mem_block
6  bool_block     = "[bool_block]" header 64_w8numbers
7  byte_block     = "[byte_block]" header 8_w8numbers
8  int_block      = "[int_block]" header 8_w16numbers
9  dint_block     = "[dint_block]" header 8_w32numbers
10 lint_block     = "[lint_block]" header 8_w64numbers
11 int_mem_block  = "[int_mem_block]" header 8_w16numbers
12 dint_mem_block = "[dint_mem_block]" header 8_w32numbers
13 header         = number_1_10000
14 64_w8numbers    = {64} number_0_w8
15 8_w8numbers     = {8} number_0_w8
16 8_w16numbers    = {8} number_0_w16
17 8_w32numbers    = {8} number_0_w32
18 8_w64numbers    = {8} number_0_w64
19 number_1_10000 = "1" | "2" | ... | "10000"
20 number_0_w8    = "0" | "1" | ... | "255"
21 number_0_w32   = "0" | "1" | ... | "4294967295"

```

```

22 number_0_w16 = "0" | "1" | ... | "65535"
23 number_0_w64 = "0" | "1" | ... | "18446744073709551615"

```

该虚拟外设模拟层与 PLC 程序保持扫描周期同步，通过模拟外设的时序行为，为模糊测试提供可控的输入信号源。该方案采用轻量化建模方法，在保证模拟精度的同时，显著提升了测试效率，避免了不符合约束的种子变异带来的无效外设响应探索。

4.3 静态分析与动态分析的协同机制

如第2章所述，单纯采用静态分析方法对 PLC 代码进行建模和分析时，存在时间复杂度呈指数级增长的问题。因此，本研究主要采用动态分析方法，旨在构建更高效的 PLC 代码模糊测试框架。在上一节中，已展示了对外设的模拟方案及其与 AFL++ 模糊测试框架结合的实现方案。

然而，即使在完成此建模的基础上，变异后的种子数据结构完全符合外设的建模规范，仅依赖动态分析的随机变异和基于代码覆盖率引导的变异方案，仍然存在输入数据种子变异空间过大、变异效率低下的局限性。正如3章所述，PLC 的竞态条件问题的特征不仅仅依靠覆盖率体现，而是体现在输出数据的波动。即使程序的执行了相同的代码逻辑，也可能在竞态条件问题检测方面得到不同的结论。为此，本研究进一步提出了动静结合的混合变异方法，通过构建精确的输入访问模型来指导种子变异过程，在保证分析质量的前提下，显著提升了测试效率。

具体实现方案是在模糊测试的预处理阶段，于 PLC 语言翻译为 C++ 语言的过程中生成变量映射表。通过该变量映射表，可以识别输入数据（即 PLC 输入端口）中对程序执行流程具有关键影响的数据项。生成的变量映射表将指导模糊测试种子变异器优先变异关键数据位置，并将关键结果进行存储。全局变量映射表作为静态分析的信息传递接口，其数据结构定义如下：

```

1 // 变量映射结构体
2 struct VariableMapping {
3     std::string var_type;
4     int array_index;
5     int bit_index; // 仅布尔类型使用
6     std::string var_name;
7 };
8
9 // 全局变量映射表
10 std::vector<VariableMapping> plc_variable_mappings;

```

在此变量映射表的基础上，可以自定义 PLC 模糊测试框架的种子变异算法。本研究的模糊测试程序针对不同类型的外设，设计了差异化的时序变异策略。具体而言，输入种子的变异过程分为两个阶段。首先，对输入的变动周期进行独立

变异；随后，为每个外设组件生成变异方案。该策略既保证了变异操作符合外设的时序特性，又能充分探索潜在的异常状态。

因此，本工作设计的自定义种子变异器能够基于先前的静态分析结果，动态调整输入数据的变异策略。变异过程中采用选择性编译机制，仅对与程序执行逻辑相关的变量进行变异，忽略无关变量；并根据不同类型数据结构的特性，分别采用针对性的变异方案。

首先，是对外设的变异周期进行调整。为此，需要重点考虑的问题是：假设当前的外设输入能够触发 PLC 的竞态条件问题，那么需要经过多少个周期才能持续到该输入触发竞态条件问题？针对这一问题，本工作参考并分析了相关静态分析论文的研究成果。

根据静态分析相关论文 PetriNet^[23]中的定理，可以得知 PLC 代码中存在竞态条件问题，这等价于 PLC 的可达图中存在竞争路径。在已知 PLC 可达图的情况下，搜索算法可以采用深度优先搜索或广度优先搜索。也就是说，在 PetriNet 中的搜索深度最多等于 PLC 可达图的节点数。然而，PLC 可达图的节点数与输入输出线圈数量呈指数级增长，导致最终的复杂度为指数级别。不过，在 PLC 动态分析的情况下，当输入输出保持不变时，只需要模拟输入输出多项式次的 PLC 周期。因此，本工作在变异种子的选择上，选取有效输入和输出线圈的数量的 2 倍作为基础周期。因此，在变异策略中，选择对 cycle 值进行少量增减。

```

1 // 公共变异策略 - cycles
2 void mutate_cycles(PLCInputBlock &block, MutatorState *state) {
3     int cycle_delta = (random() % 5) - 2;
4     // 随机增减cycles值
5     if (random() % 100 < state->mutation_rate()) {
6         // 布尔块
7         block.input_bool_block.cycles += cycle_delta;
8         if (block.input_bool_block.cycles < 0)
9             block.input_bool_block.cycles = 0;
10
11        // 字节块
12        cycle_delta = (random() % 5) - 2;
13        block.input_byte_block.cycles += cycle_delta;
14        if (block.input_byte_block.cycles < 0)
15            block.input_byte_block.cycles = 0;
16
17        // 整型块
18        cycle_delta = (random() % 5) - 2;
19        block.input_int_block.cycles += cycle_delta;
20        if (block.input_int_block.cycles < 0)
21            block.input_int_block.cycles = 0;
22
23        // 双整型块
24        cycle_delta = (random() % 5) - 2;
25        block.input_dint_block.cycles += cycle_delta;
26        if (block.input_dint_block.cycles < 0)
27            block.input_dint_block.cycles = 0;
28
29        // 长整型块

```

```

30     cycle_delta = (random() % 5) - 2;
31     block.input_lint_block.cycles += cycle_delta;
32     if (block.input_lint_block.cycles < 0)
33         block.input_lint_block.cycles = 0;
34
35     // 整型内存块
36     cycle_delta = (random() % 5) - 2;
37     block.input_int_mem_block.cycles += cycle_delta;
38     if (block.input_int_mem_block.cycles < 0)
39         block.input_int_mem_block.cycles = 0;
40
41     // 双整型内存块
42     cycle_delta = (random() % 5) - 2;
43     block.input_dint_mem_block.cycles += cycle_delta;
44     if (block.input_dint_mem_block.cycles < 0)
45         block.input_dint_mem_block.cycles = 0;
46 }
47 }

```

在此之后，是对 PLC 的不同输入组件分别进行变异。

针对 PLC 系统中的布尔型输入块（BoolBlock）设计的变异策略如下。通过遍历预设的变量映射表（plc_variable_mappings）实现定向变异。当映射项的变量类型为字节输入（"byte_inputs"）时，算法以用户定义的变异概率（mutation_rate）为条件，随机选择三种位级操作之一：位翻转（通过异或运算反转特定位置）、强制置1（将目标位设为高电平）或强制置0（将目标位设为低电平）。此类操作直接作用于输入块的指定数组索引（array_index）与位索引（bit_index），通过改变布尔变量的逻辑状态来模拟硬件输入信号的异常波动，从而测试 PLC 程序对输入扰动的鲁棒性。该策略通过细粒度的位操作增强了模糊测试对 PLC 设备输入通道的覆盖能力，有助于发现潜在的逻辑漏洞或边界条件错误。

```

1 void mutate_bool_block(BasicInputBlock<unsigned char, 2> &block, MutatorState
  *state) {
2     for (const auto &mapping : plc_variable_mappings) {
3         if (mapping.var_type == "byte_inputs") {
4             if (random() % 100 < state->mutation_rate()) {
5                 switch (random() % 3) {
6                     case 0:
7                         block.input[mapping.array_index][mapping.bit_index] ^=
8                             1; // 位翻转
9                         break;
10                    case 1:
11                        block.input[mapping.array_index][mapping.bit_index] =
12                            1; // 强制置1
13                        break;
14                    case 2:
15                        block.input[mapping.array_index][mapping.bit_index] =
16                            0; // 强制置0
17                        break;
18                }
19            }
20        }
21    }
22 }

```

在字节块变异函数的设计中，本文实现了针对 PLC 输入字节数据的多策略变异机制。该函数通过遍历预定义的变量映射表，当检测到变量类型为字节输入时，依据当前变异状态中的概率参数决定是否执行变异操作。变异策略包括三种随机选择的方式：位翻转操作通过异或运算实现特定位的取反，增减扰动在 $[-2, 2]$ 范围内对原值进行微小调整，随机替换则将字节内容重置为 0 至 255 范围内的任意值。这种变异机制有效覆盖了数值扰动、边界跳变和随机化测试等关键测试维度，能够系统性地验证 PLC 程序对异常字节数据的容错能力与安全性表现。

```

1 void mutate_byte_block(BasicInputBlock<IEC_BYTE, 1> &block, MutatorState *
2 state) {
3     for (const auto &mapping : plc_variable_mappings) {
4         if (mapping.var_type == "byte_inputs") {
5             if (random() % 100 < state->mutation_rate()) {
6 // 字节级变异：位翻转、增减小量、随机值
7                 switch (random() % 3) {
8                     case 0:
9                         block.input[mapping.array_index] ^= (1 << (random() %
10 8));
11                     break; // 位翻转
12                     case 1:
13                         block.input[mapping.array_index] += (random() % 5) -
14 2;
15                     break; // 小量增减
16                     case 2:
17                         block.input[mapping.array_index] = random() & 0xFF;
18                         break; // 随机值
19                 }
20             }
21 }

```

针对整型数据块的专用变异策略，本研究采用如下的变异方案。如代码所示，该函数通过轮询机制在 PLC 变量映射表中筛选字节输入型变量，并基于 16 位整型数据的特征空间实施差异化变异：位翻转操作通过异或运算扰动特定位模式；中等范围增减在 $[-50, 50]$ 区间内模拟数值漂移；全范围随机赋值可探索 0 至 65535 的边界条件；而符号取反操作则专门针对有符号整数的负值边界进行测试。这种结合位级操作、数值扰动与符号变换的复合变异策略，能够有效激发 PLC 程序在处理整型数据异常时的潜在安全漏洞，显著提升对数值越界、符号错误等典型安全风险的检测能力。

```

1 void mutate_int_block(BasicInputBlock<IEC_INT, 1> &block, MutatorState *state)
2 {
3     for (const auto &mapping : plc_variable_mappings) {
4         if (mapping.var_type == "byte_inputs") {
5             switch (random() % 4) {
6                 case 0:
7                     block.input[mapping.array_index] ^= (1 << (random() % 16))
8 ;
9                 break; // 位翻转

```



```

8         case 1:
9             block.input[mapping.array_index] += (random() % 100) - 50;
10            break; // 中等范围增减
11        case 2:
12            block.input[mapping.array_index] = random() & 0xFFFF;
13            break; // 随机值
14        case 3:
15            block.input[mapping.array_index] = -block.input[mapping.
array_index];
16            break; // 取负
17        }
18    }
19 }
20 }

```

针对双整型数据块的特殊性，本研究设计了更为精细的变异策略，以应对 32 位数据空间的测试挑战。如代码所示，该函数在满足变异概率阈值的前提下，对标识为双整型输入的变量实施定向变异：位翻转操作扩展至 32 位空间，覆盖全部比特位扰动；大幅度增减在 $[-500, 500]$ 范围内，模拟工业场景中的数值跃变；全范围随机赋值遍历 32 位整数的极值边界；符号取反操作专门测试有符号双整型的负值处理逻辑；新增的字节序翻转功能则针对大端序/小端序混合环境下的数据解析漏洞。这种融合数值边界测试、位操作异常与字节序混淆的多维变异方法，能够有效揭示 PLC 程序在处理双整型数据时可能存在的缓冲区溢出、类型混淆及字节序解析等深层安全缺陷。

```

1 void mutate_dint_block(BasicInputBlock <IEC_DINT, 1> &block, MutatorState *
state) {
2     for (const auto &mapping : plc_variable_mappings) {
3         if (mapping.var_type == "dint_inputs") {
4             if (random() % 100 < state->mutation_rate()) {
5                 // 针对双整型的更复杂变异
6                 switch (random() % 5) {
7                     case 0:
8                         block.input[mapping.array_index] ^= (1 << (random() %
32));
9                         break;
10                    case 1:
11                        block.input[mapping.array_index] += (random() % 1000)
- 500;
12                        break;
13                    case 2:
14                        block.input[mapping.array_index] = random();
15                        break;
16                    case 3:
17                        block.input[mapping.array_index] = -block.input[
mapping.array_index];
18                        break;
19                    case 4:
20                        reverse_bytes(block.input[mapping.array_index]);
21                        break; // 字节序翻转
22                }
23            }
24        }
25    }
26 }

```

针对长整型数据块的变异策略如下。如代码所示，该函数通过扩展的变异操作集对长整型输入进行全面测试：位翻转操作覆盖 64 位全比特空间；大幅值扰动范围扩展至 $[-5000, 5000]$ ，以模拟工业场景中的极端数值波动；全范围随机赋值可探测 2^{64} 量级的边界条件；符号取反操作测试有符号长整型的负值处理机制；字节序翻转针对跨平台数据交换场景；而新增的按位取反操作则通过全比特翻转生成极端异常值。这种结合数值极值测试、位级全面扰动与结构异常注入的复合变异策略，能够有效揭示 PLC 程序在处理 64 位长整型数据时可能存在的数值溢出、符号扩展错误及内存对齐等深层安全漏洞，显著增强对现代工业控制系统中大数据处理模块的鲁棒性验证。

```

1 void mutate_lint_block(BasicInputBlock<IEC_LINT, 1> &block, MutatorState *
2 state) {
3     for (const auto &mapping : plc_variable_mappings) {
4         if (mapping.var_type == "lint_inputs") {
5             // for (int i = 0; i < BUFFER_SIZE; ++i) {
6             //     if (state->should_mutate("lint_inputs", i) && random() %
7             100 < state->mutation_rate()) {
8                 if (random() % 100 < state->mutation_rate()) {
9                     switch (random() % 6) {
10                        case 0:
11                            block.input[mapping.array_index] ^= (1LL << (random()
12                                % 64));
13                            break;
14                        case 1:
15                            block.input[mapping.array_index] += (random() % 10000)
16                                - 5000;
17                            break;
18                        case 2:
19                            block.input[mapping.array_index] = random();
20                            break;
21                        case 3:
22                            block.input[mapping.array_index] = -block.input[
23                                mapping.array_index];
24                            break;
25                        case 4:
26                            reverse_bytes(block.input[mapping.array_index]);
27                            break;
28                        case 5:
29                            block.input[mapping.array_index] = ~block.input[
30                                mapping.array_index];
31                            break; // 按位取反
32                    }
33                }
34            }
35        }
36    }
37 }

```

针对内存块数据的专用变异策略如下。本研究设计了面向无符号整型的边界值测试方法。如代码所示，mutate_int_mem_block 函数针对 16 位无符号整型内存变量实施的编译策略为：清零操作用于测试零值边界条件，最大值赋值用

于验证上溢处理机制，递增和递减操作则通过 ± 1 的微小扰动探索边界过渡行为。而 `mutate_dint_mem_block` 函数扩展至 32 位无符号整型空间，在保留基础边界测试的基础上新增字节序翻转操作。这种专注于极值边界与单步增减的定向变异方法，能够有效模拟内存操作中的典型异常场景，如缓冲区边界检查失效、数值回绕漏洞及字节序解析错误等，特别适用于检测 PLC 程序在内存映射 I/O 处理过程中可能存在的安全缺陷。

```

1
2
3 void mutate_int_mem_block(BasicInputBlock<IEC_UINT, 1> &block, MutatorState *
  state) {
4     for (const auto &mapping : plc_variable_mappings) {
5         if (mapping.var_type == "int_mem_inputs") {
6             if (random() % 100 < state->mutation_rate()) {
7                 // if (state->should_mutate("int_mem_inputs", i) && random() %
100 < state->mutation_rate()) {
8                     switch (random() % 4) {
9                         case 0:
10                        block.input[mapping.array_index] = 0;
11                        break;
12                        case 1:
13                        block.input[mapping.array_index] = 0xFFFF;
14                        break;
15                        case 2:
16                        block.input[mapping.array_index] = block.input[mapping
17                        .array_index] + 1;
18                        break;
19                        case 3:
20                        block.input[mapping.array_index] = block.input[mapping
21                        .array_index] - 1;
22                        break;
23                    }
24                }
25            }
26        }
27
28 void mutate_dint_mem_block(BasicInputBlock<IEC_UDINT, 1> &block, MutatorState
  *state) {
29     // 类似int_mem_block但针对32位
30     for (const auto &mapping : plc_variable_mappings) {
31         if (mapping.var_type == "dint_mem_inputs") {
32             // for (int i = 0; i < BUFFER_SIZE; ++i) {
33             //     if (state->should_mutate("dint_mem_inputs", i) && random()
34             % 100 < state->mutation_rate()) {
35                 if (random() % 100 < state->mutation_rate()) {
36                     switch (random() % 5) {
37                         case 0:
38                        block.input[mapping.array_index] = 0;
39                        break;
40                        case 1:
41                        block.input[mapping.array_index] = 0xFFFFFFFF;
42                        break;
43                        case 2:
44                        block.input[mapping.array_index] = block.input[mapping
45                        .array_index] + 1;
46                        break;
47                        case 3:
48                        block.input[mapping.array_index] = block.input[mapping
49                        .array_index] - 1;
50                        break;
51                    }
52                }
53            }
54        }
55    }
56 }

```

```

45         block.input[mapping.array_index] = block.input[mapping
    .array_index] - 1;
46         break;
47     case 4:
48         reverse_bytes(block.input[mapping.array_index]);
49         break;
50     }
51 }
52 }
53 }
54 }

```

本研究提出的自定义种子变异算法以动态链接库形式集成于 AFL++ 模糊测试框架，通过标准化的 API 接口与框架交互。在兼容 AFL++ 模糊测试框架现有的变异策略的基础上，有效扩展了变异能力。

4.4 动态运行状态监控与异常检测

正如第3章所言，在 PLC 程序的安全性分析中，尤其竞态条件这一类漏洞，其危害往往仅在程序运行时显现。代码本身不会导致程序崩溃，但是会导致输出信号的异常振荡。即便代码本身未引发语法错误或系统崩溃，输出信号的异常振荡（如继电器频繁切换、阀门启停周期紊乱）仍可能导致生产流程失控。此类问题的特殊性在于，静态分析难以通过代码审查或形式化模型直接识别其存在，而动态运行时输入信号与程序扫描周期之间复杂的时序耦合效应才是触发异常的关键。

然而，这个特点使得竞态条件问题在应用模糊测试框架是面临独特挑战：原始的基于崩溃检测的模糊测试框架 AFL++ 难以有效识别异常，因为该模糊测试框架主要依赖进程监控机制（如 fork/waitpid）和内存检测工具（如 AddressSanitizer）来识别程序崩溃或内存安全漏洞，而无法检测 PLC 运行时出现的功能性异常。

为应对上述挑战，本研究设计的动态分析框架创新性地集成了程序运行时状态监控技术。通过在 PLC 虚拟执行环境中嵌入轻量级监控模块，实时采集变量状态、I/O 端口信号及扫描周期时序等关键运行时数据并进行分析，该监控层与 AFL++ 框架的异常检测机制深度集成，确保模糊测试过程中能够及时捕获潜在的功能异常。

具体实现方面，本研究针对 PLC 输出映像区设计了专门的竞态检测抽象层，通过在每个扫描周期结束前执行输出稳定性检测，实现动态监控。

```

1 // 周期结束时的竞态检测函数
2 template <typename T, int Dim = 1>
3 class BasicBufferHistory : public SuperBasicBufferHistory {
4 public:
5     bool check_change() {
6         // std::cout << "BasicBufferHistory<T, Dim>::check_change() called."
        << std::endl;
7         int change_count = 0;

```

```

8      for (size_t i = 1; i < MAX_RESULTS; i++) {
9          bool is_crash = false;
10         for (size_t k = 0; k < BUFFER_SIZE; k++) {
11             if (buffer_output[k][i] != buffer_output[k][i - 1]) {
12                 is_crash = true;
13                 break;
14             }
15         }
16         if (is_crash) {
17             break;
18         }
19         if (is_crash) {
20             change_count++;
21         }
22     }
23     if (change_count > 0) {
24         return true;
25     }
26     return false;
27 }
28 };
29 template <typename T>
30 class BasicBufferHistory<T, 2> : public SuperBasicBufferHistory {
31 public:
32     bool check_change() {
33         this->print_history();
34         std::cout << "bool check_change() called" << std::endl;
35         int change_count = 0;
36         for (size_t i = 1; i < MAX_RESULTS; i++) {
37             bool is_crash = false;
38             for (size_t j = 0; j < 8; j++) {
39                 for (size_t k = 0; k < BUFFER_SIZE; k++) {
40                     if (buffer_output[k][j][i] != buffer_output[k][j][i - 1])
41                     {
42                         is_crash = true;
43                         break;
44                     }
45                     if (is_crash) {
46                         break;
47                     }
48                 }
49                 if (is_crash) {
50                     change_count++;
51                 }
52             }
53             if (change_count > 0) {
54                 return true;
55             }
56             return false;
57         }
58     };

```

在传统 PLC 运行环境中，输出不稳定通常不会引发程序报错，导致竞态条件问题难以被常规检测机制发现。本框架的创新之处在于，当竞态检测层识别到异常振荡行为时，会主动触发报警机制，通过向 PLC 模拟程序注入特定异常信号，使 AFL++ 模糊测试框架能够准确识别此类功能性异常状态。该主动异常注入机制有效解决了传统崩溃检测方法在 PLC 运行时功能性漏洞检测能力不足的问题，实现

了对竞态条件引发输出振荡的精准捕获。同时，本框架提供了可扩展的接口设计，支持研究人员根据化工、能源等不同工业场景的安全需求，自定义变量阈值报警参数或特定时序约束条件。在有了本设计之后，PLC 的模糊测试框架才具有对 PLC 的竞态条件问题的检测能力。

4.5 本章总结

基于 AFL++ 模糊测试框架，本研究实现了一个面向 PLC 代码的覆盖率引导模糊测试系统。如第2章所述，通用计算领域的模糊测试技术已较为成熟，但其检测机制主要针对程序崩溃等传统异常模式。针对 PLC 环境中特有的竞态条件安全问题，本研究对通用框架进行了多维度的适配与增强。前文已详细阐述关键技术创新点，本节将从系统整体执行流程的角度进行综合介绍。图4.1展示了本研究所提出的动静态协同分析框架的整体工作流程。

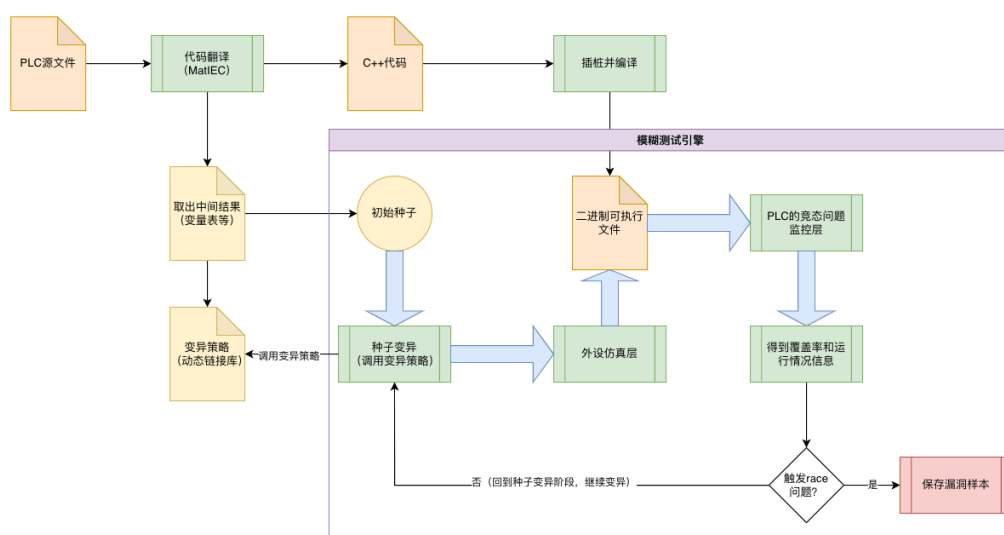


图 4.1 动静态协同的 PLC 安全分析框架的总体流程

步骤一：要对一个 PLC 程序进行模糊测试，首先要将其在模拟环境下重新编译为 C++ 代码。在编译转换层面，先构建 PLC 到 C++ 代码的转换器，突破了 IEC 61131-3 语言与通用模糊测试工具之间的适配障碍。基于 MatIEC 编译框架进行了深度改进，实现了结构化文本向 C++ 的语义保持转换，使 PLC 程序能够在通用计算环境中运行，为应用成熟的模糊测试工具链奠定了坚实的技术基础。然后，使用 AFL++ 模糊测试框架的插桩编译工具，对转换后的 C++ 代码进行编译，得到等待被测试的二进制可执行文件。

步骤二：接下来要对 PLC 的代码进行静态分析，然后得到 PLC 的定制化种子变异策略。为应对静态分析中状态空间爆炸及外设行为建模的瓶颈，本方案创

新性地将静态代码分析技术应用于 PLC 代码解析，通过提取关键变量，引导模糊测试引擎对高危代码区域实施定向变异，有效缩减了输入空间的指数级增长范围。通过静态分析指导测试用例的变异，显著降低了输入空间的冗余度。经过本步骤，得到一个变异策略动态链接库，等待被模糊测试引擎在进行测试时调用。

步骤三：在外设模拟层面，构建虚拟外设仿真层来模拟物理外设的行为，并根据模糊测试的反馈数据来模拟外设的响应。与传统的硬件依赖方案相比，该虚拟化方案不仅保障了扫描周期的同步性，还支持并行测试场景下的设备虚拟化副本管理、基于代码覆盖导向的动态响应生成以及历史测试场景的版本化回溯。虚拟外设层的外设行为相应遵循外设约束，可以有效减少无效的变异，增加模糊测试的探测深度。

步骤四：嵌入式运行时监控模块实现了对微秒级指令周期的追踪及 I/O 状态变化的细粒度分析，为漏洞根因定位提供了全方位的数据支持。动态监控模块采用环形缓冲区记录微秒级 I/O 历史数据，实现竞态条件检测。

在经过以上步骤后，可以运行模糊测试，等待模糊测试引擎给出能够触发 PLC 的竞态条件问题的漏洞样本。

下一章将通过多组对比实验，对所提系统的有效性进行综合评估。

第 5 章 实验评估与结果分析

本研究基于 AFL++ 模糊测试框架^[42]，结合 PLC 重编译、构造虚拟外设响应、静态分析技术、PLC 的竞态条件问题检测，构建了一套完整的 PLC 安全分析系统。系统实现流程如下：首先，利用改进过的 MatIEC 编译器将 PLC 任务中的结构化文本代码，转换为 C++ 语言的中间代码；随后，依据第4章所述设计方案，构建虚拟外设层，并通过静态分析技术构建自定义的种子变异算法库。在此基础上，本研究对 AFL++ 测试框架进行了深度扩展，开发了专门支持 PLC 结构化文本语言特性的测试框架，实现了包含轻量化监控模块和扩展接口的完整模糊测试反馈机制。

为了系统评估本研究的有效性，实验设计围绕以下几个核心研究问题展开。

首先，评估该方法在 PLC 程序模糊测试中的效率表现，特别是在竞态条件检测和程序正确性验证方面的有效性。

其次，通过对比实验量化静态分析技术引入后对测试效率的提升效果，明确其相较于纯动态分析方法的优势所在。

第三，从系统可扩展性的角度，评估该框架在挖掘各类 PLC 代码通用漏洞方面的能力。

第四，通过横向对比研究，分析本方法与现有相关工作在测试效率和检测能力方面的性能差异。

这些研究问题全面涵盖了方法的有效性、性能优势、实用性和创新性等关键评估维度，为本章的实验验证奠定了明确的评估框架。

5.1 实验环境和测试用例说明

实验环境搭建在配备 Intel Core i9-12900K 处理器的硬件平台上。该处理器采用 8 个性能核与 8 个能效核的混合架构设计，基础频率为 3.2 GHz，配备 64GB DDR5-4800 内存。软件环境采用 Ubuntu 22.04 LTS 操作系统，内核版本为 5.15.0。

测试数据集由两部分组成：第一部分选自工业开源社区（包括 PLCopen、GitHub 等平台）的含有已知漏洞的单任务 PLC 程序，这些程序涵盖梯形图、功能块图和结构化文本等多种 IEC 61131-3 标准编程语言；第二部分通过 G4LTL-ST 工具^[50]自动生成的具有代表性的测试用例。该工具能够基于线性时序逻辑规约生成符合工业控制语义的 PLC 程序，且已在相关研究领域的多项研究中得到应用。

G4LTL-ST^[50] 是一个能够根据约束自动生成结构化文本语言代码的工具。该工具采用 Java 语言编写，已在多个科研项目中（例如 SymPLC^[34]等）得到应用。

线性时序逻辑 (Linear Temporal Logic, 简称 LTL)^[51] 是一种形式化逻辑语言, 专门用于描述系统随时间演变的行为属性。LTL 由 Amir Pnueli 于 1977 年提出, 广泛应用于硬件验证、软件模型检测及控制系统规范等领域。G4LTL-ST 基于 LTL 作为基础规范语言进行了扩展, 以满足工业自动化的需求。该工具使用专用语言构建约束, 并在此基础上生成相应的代码。

根据静态分析相关论文 PetriNet^[23] 的描述, 可以得知 PLC 代码中存在竞态条件问题, 这等价于 PLC 的可达图中存在竞争路径。而在可达图中存在竞争路径, 可以用 LTL 语言描述为:

```
1 ALWAYS ((pb1 && led) -> X(led2))
2 ALWAYS (!(pb1 && led) -> X(!led2))
3 ALWAYS (led2 -> X(!led))
4 ALWAYS (!led2 -> X(led))
```

因此, 本工作构建测试用例的方法是先将形如上图的约束加入到测试用例构建中, 然后将使用该工具生成的包含漏洞的代码注入到测试用例中。

5.2 本工作在性能与漏洞挖掘能力方面的表现

通过将 PLC 程序跨平台编译至 x86 架构, 本研究显著提升了模糊测试的可迁移性与经济性。该技术路径使测试环境能够从专用工业控制器迁移至性能高出数个数量级的商用硬件平台, 在保持测试准确性的同时, 大幅降低硬件成本。若充分发挥服务器集群的并行计算能力, 测试效率将实现数量级的提升。需要特别说明的是, 虽然源代码编译与插桩过程会引入一定的性能开销, 但由于通用计算平台与工业 PLC 之间存在巨大的硬件性能差距, 基于 x86 架构的仿真测试仍能展现出显著的速度优势。

值得注意的是, 正如在第2章所说过的, PLC 的执行流程是每个扫描周期从输入扫描, 到程序执行, 到结果输出, 不停地循环往复的。也就是说, PLC 如非外界干预, 程序执行流程不会自主结束。然而通用计算机的模糊测试通常以用户态可执行程序单次运行时间作为评估标准, 也就是说传统的模糊测试的效率评估方案在 PLC 的场景中无法直接挪用。

因此本工作使用的评估指标为: 执行周期数。其含义是 PLC 在模糊测试时, 在一定时间内能够执行多少个循环周期 (而不是程序整体的执行次数)。本工作的方案在 PLC 的模糊测试根据程序复杂程度不同, 每秒内可以进行的测试周期数为 1000-10000 不等。相比之下, ICSFuzz^[37] 直接在物理 PLC 硬件平台上进行测试, 每秒内大约可以进行 70 次扫描周期。如此高的时间成本将严重制约模糊测试的效率, 显著延缓漏洞的发现进程。这说明本方案在测试效率与成本控制方面的双重

表 5.1 测试用例信息

文件名	ST 代码行数	空白代码行数	经过编译后的代码行数
industrial1.st	32	8	267
industrial2.st	30	6	143
industrial3.st	182	4	381
industrial4.st	52	7	195
industrial5.st	84	6	281
industrial6.st	84	6	281
industrial7.st	84	6	281
industrial8.st	230	6	639
industrial11.st	80	6	267
industrial12.st	1561	7	3821
industrial13.st	229	6	626
industrial15.st	3402	6	8790
auto1.st	68	6	324
auto2.st	55	6	219
auto3.st	66	4	282
auto4.st	67	5	282
auto5.st	67	5	282
auto6.st	66	4	289
auto7.st	66	5	288
auto8.st	65	3	311
auto9.st	65	4	312
auto10.st	66	4	288
auto11.st	55	5	324
auto12.st	67	5	314
总计	6823	130	19487

优势。

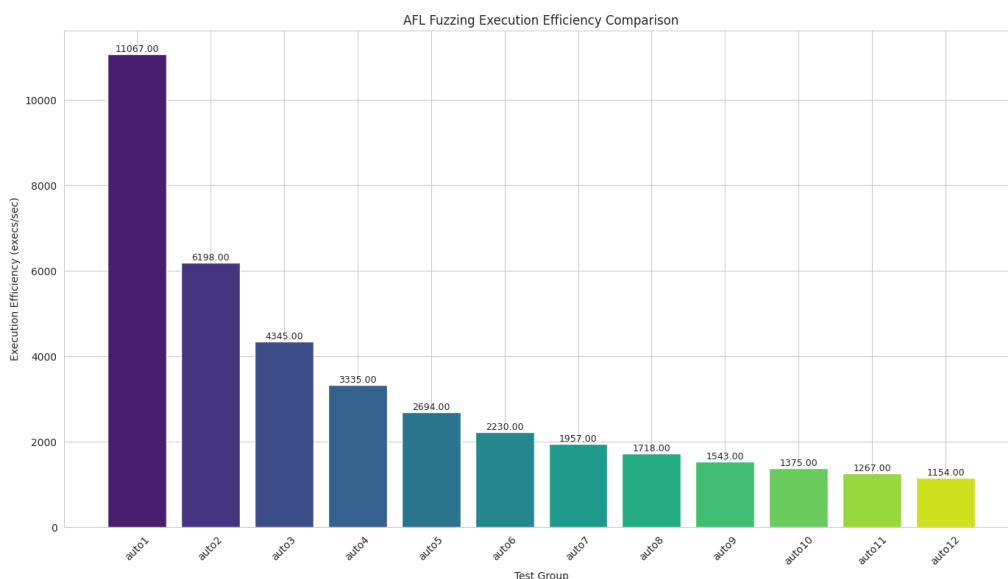


图 5.1 本工作的模糊测试的执行效率

此外，本工作在 PLC 竞态问题数据集上进行了测试，实验表明，在存在 PLC 的竞态问题的数据集中，都能在较短时间内检测出竞态问题。

5.3 与静态分析相关工作的对比研究

在 PLC 代码竞态问题的研究领域，静态分析方法一直是主流方向。Luo 等人^[23]提出的基于 Petri 网的静态分析方法（详见第2章）是该领域的代表性工作，本研究选择其作为主要对比基准。由于原论文未公开实现代码，本研究首先根据论文描述对该方法进行了完整复现。

5.3.1 静态分析工作的复现

复现工作严格遵循原文技术路线，主要包含四个关键步骤：首先将梯形图抽象为图结构 LD 图（LD Graph）；然后通过形式化转换算法将其转换为普通 PetriNet；接着对 PetriNet 的结构进行简化处理，以消除冗余过渡；最后基于可达图分析方法进行 PLC 的竞态条件检测。在实现层面，本研究选择 Python 作为开发语言，采用嵌套列表结构表示梯形图的层次化语义。具体数据结构设计如下：最外层列表表示梯形图的梯级序列，每个梯级用子列表表示，列表中的元组表示串联逻辑关系，花括号 {} 表示并联逻辑关系。例如，一个典型的三梯级梯形图可表示为：

```

1 # 定义LD程序
2 plc_ladder_diagram_logic = [
3     # Rung 1: 简单串联 - I0.0 控制 Q0.0
4     [
5         ('I0.0', 'NO'),

```

Control Application	Execution Speed (inputs/sec)	First Crash (time mm:ss)	First crash (inputs)	Crashes (1hr)
bf_mcpy_1	70.88	3:54	15270	32
bf_mcpy_6	64.2	3:08	12172	21
bf_mcpy_8	66.06	4:39	18216	17
bf_mcpy_12	62.11	7:06	26645	9
bf_mset_1	64.56	3:28	13441	21
bf_mset_3	62.68	2:54	10906	24
bf_mset_5	68.8	4:14	17554	16
bf_mset_9	69.76	10:23	43530	7
bf_mmove_1	64.63	2:56	11245	28
bf_mmove_4	63.1	2:39	10070	24
bf_mmove_7	66.31	3:49	15317	15
bf_mmove_12	64.53	13:03	50643	6
oob_1_arr_1	71.86	0:55	3880	39
oob_1_arr_6	77.03	1:43	8085	28
oob_1_arr_9	69.78	1:45	7326	27
oob_1_arr_13	75.2	3:27	27241	19
oob_2_arr_1	73.53	1:57	8558	35
oob_2_arr_5	71.1	2:45	22759	27
oob_2_arr_8	69.8	3:08	13366	22
oob_2_arr_13	70.95	3:12	13401	19
divby0_1	73.68	N/A	N/A	0

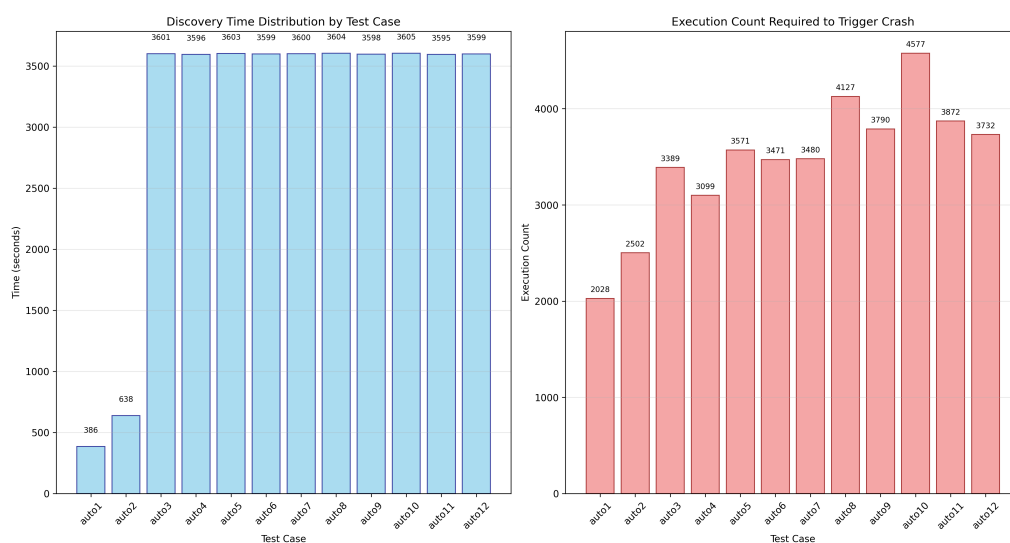
图 5.2 ICSFuzz 的模糊测试效率^[37]

图 5.3 竞态问题检测时间和执行次数

```

6      ('Q0.0', 'COIL')
7  ],
8
9  # Rung 2: 复杂逻辑 - I0.1 串联 (I0.2 并联 M1) 串联 Q0.1
10 [
11     ('I0.1', 'NO'),
12     { # 并联逻辑
13         (('I0.2', 'NC'),), # 路径1
14         (('M1', 'NO'),),   # 路径2
15     },
16     ('Q0.1', 'COIL')
17 ],
18
19 # Rung 3: 自保持电路 - I0.3 并联 Q0.2 串联 Q0.2
20 [
21     ('I0.3', 'NO'),
22     { # 并联逻辑 (自保持)
23         (('I0.3', 'NO'),),
24         (('Q0.2', 'NO'),),
25     },
26     ('Q0.2', 'COIL')
27 ]
28 ]

```

梯形图到图结构的语义转换 需要特别区分两个关键概念：梯形图（Ladder Diagram）是 IEC 61131-3 标准定义的 PLC 编程语言，而 LD 图（LD Graph）则是在静态分析工作中提出的一种形式化图结构。后者是研究者为分析梯形图语义行为专门设计的中间表示，其核心价值在于将图形化编程语言的直观表达转换为适合形式化分析的图模型。

转换过程遵循语义保持原则，将梯形图中的每个梯级映射为 LD 图中的一个子图结构，其中：

- 将常开/常闭触点转换为带权顶点，权重表示触点的状态。
- 将线圈输出转换为具有前向依赖的特殊顶点。
- 将并联分支转换为并联路径，保持电气回路的拓扑结构特性。
- 将串联元件转换为顺序边，以反映电流的流向。

这种转换的关键在于保持梯形图的电气特性与逻辑语义的等价性。例如，梯形图中的“电流流动”概念被抽象为 LD 图中的有向边，而“通电状态”则量化为顶点属性。通过这种形式化转换，原本基于继电器逻辑的图形化程序被转化为可进行数学分析的图结构，为后续 PetriNet 的转换奠定了基础。值得注意的是，转换过程中需要对梯形图特有的扫描周期语义进行特殊处理，这通过为 LD 图添加时序标记顶点来实现。

```

1 class LDGraphConverter:
2     def __init__(self):
3         self.vertices = set()
4         self.edges = set()

```

```

5         self.vertex_counter = 0
6         self.coil_counter = 0
7         self.rung_count = 0
8
9     def convert_to_ld_graph(self, ladder_diagram):
10        """
11        将梯形图转换为LD Graph
12        """
13        self.rung_count = len(ladder_diagram)
14        self._initialize_power_rail_vertices()
15
16        for rung_index, rung in enumerate(ladder_diagram, 1):
17            self._process_rung(rung, rung_index)
18
19        return {
20            'vertices': list(self.vertices),
21            'edges': list(self.edges),
22            'vertex_set_VL': [f'v_l{i}' for i in range(1, self.rung_count + 1)
23        ],
24            'vertex_set_VR': [f'v_r{i}' for i in range(1, self.rung_count + 1)
25        ],
26            'rung_count': self.rung_count
27        }
28
29    def _initialize_power_rail_vertices(self):
30        """初始化电源轨节点"""
31        for i in range(1, self.rung_count + 1):
32            self.vertices.add(f'v_l{i}') # 左电源轨节点
33            self.vertices.add(f'v_r{i}') # 右电源轨节点
34
35    def _process_rung(self, rung, rung_index):
36        """处理单个梯级"""
37        previous_vertex = f'v_l{rung_index}' # 起始于左电源轨
38        coil_vertex = None
39
40        for element in rung:
41            if isinstance(element, tuple):
42                # 处理单个触点或线圈
43                vertex = self._create_vertex(element, rung_index)
44                self._add_edge(previous_vertex, vertex)
45                previous_vertex = vertex
46
47                if element[1] in ['COIL', 'SET', 'RESET']:
48                    coil_vertex = vertex
49
50            elif isinstance(element, dict):
51                # 处理并联逻辑
52                parallel_vertices = []
53                for parallel_path in element.values():
54                    path_end_vertex = self._process_parallel_path(
55                        parallel_path,
56                        previous_vertex,
57                        rung_index)
58                    parallel_vertices.append(path_end_vertex)
59
60                # 创建并联合并点
61                merge_vertex = f'merge_{rung_index}_{self.vertex_counter}'
62                self.vertex_counter += 1
63                self.vertices.add(merge_vertex)
64
65                for p_vertex in parallel_vertices:

```

```

63         self._add_edge(p_vertex, merge_vertex)
64
65         previous_vertex = merge_vertex
66
67         # 连接到右电源轨
68         if coil_vertex:
69             self._add_edge(coil_vertex, f'v_r{rung_index}')
70         else:
71             self._add_edge(previous_vertex, f'v_r{rung_index}')
```

```

72
73 def _process_parallel_path(self, path_elements, start_vertex, rung_index):
74     """处理并联路径"""
75     current_vertex = start_vertex
76
77     for element in path_elements:
78         if isinstance(element, tuple):
79             vertex = self._create_vertex(element, rung_index)
80             self._add_edge(current_vertex, vertex)
81             current_vertex = vertex
82         elif isinstance(element, list):
83             # 嵌套路径处理
84             for sub_element in element:
85                 vertex = self._create_vertex(sub_element, rung_index)
86                 self._add_edge(current_vertex, vertex)
87                 current_vertex = vertex
88
89     return current_vertex
90
91 def _create_vertex(self, element, rung_index):
92     """创建变量节点"""
93     var_name, element_type = element
94
95     # 确定类型标识符s2
96     type_map = {
97         'NO': 0,      # 常开触点
98         'NC': 1,      # 常闭触点
99         'COIL': 2,    # 普通线圈
100        'SET': 4,      # 置位线圈
101        'RESET': 5     # 复位线圈
102    }
103
104     s2 = type_map.get(element_type, 0)
105
106     # 确定位置标识符s3 (如果是线圈)
107     if element_type in ['COIL', 'SET', 'RESET']:
108         self.coil_counter += 1
109         s3 = self.coil_counter
110     else:
111         s3 = 0 # 触点使用0
112
113     vertex = f'v_{var_name}_{s2}_{s3}'
114     self.vertices.add(vertex)
115
116     return vertex
117
118 def _add_edge(self, from_vertex, to_vertex):
119     """添加边"""
120     edge = (from_vertex, to_vertex)
121     self.edges.add(edge)
```

将LD图翻译为PetriNet 基于前阶段生成的LD图结构,本步骤实现其向PetriNet的形式化转换。

```

1
2 class LDGraphToPetriNetConverter:
3     """
4     完整的LD Graph到Petri网转换器
5     包含DFS/BFS图遍历算法实现
6     """
7
8     def __init__(self):
9         self.places = set()
10        self.transitions = set()
11        self.arcs = set()
12        self.marking = {}
13        self.variable_places = {}
14        self.ld_graph = None
15
16    def convert(self, ld_graph: Dict) -> Dict:
17        """主转换函数"""
18        self.ld_graph = ld_graph
19        vertices = ld_graph['vertices']
20        edges = ld_graph['edges']
21
22        # 构建邻接表用于图遍历
23        self.adjacency_list = self._build_adjacency_list(edges)
24        self.reverse_adjacency_list = self._build_reverse_adjacency_list(edges)
25    )
26
27    # 执行转换步骤
28    self._create_places_for_variables(vertices)
29    self._process_instruction_paths(vertices)
30    self._process_cut_node_sets(vertices)
31    self._process_input_variables(vertices)
32    self._set_initial_marking()
33
34    return self._get_petri_net_model()
35
36    def _build_adjacency_list(self, edges: Set[Tuple[str, str]]) -> Dict[str,
37    List[str]]:
38        """构建邻接表"""
39        adjacency_list = {}
40        for from_node, to_node in edges:
41            if from_node not in adjacency_list:
42                adjacency_list[from_node] = []
43            adjacency_list[from_node].append(to_node)
44        return adjacency_list
45
46    def _build_reverse_adjacency_list(self, edges: Set[Tuple[str, str]]) ->
47    Dict[str, List[str]]:
48        """构建反向邻接表"""
49        reverse_adjacency_list = {}
50        for from_node, to_node in edges:
51            if to_node not in reverse_adjacency_list:
52                reverse_adjacency_list[to_node] = []
53            reverse_adjacency_list[to_node].append(from_node)
54        return reverse_adjacency_list
55
56    def _create_places_for_variables(self, vertices: Set[str]):
57        """为每个变量创建库所对"""
58        variable_names = set()

```



```

56
57     for vertex in vertices:
58         if vertex.startswith('v_') and not (vertex.startswith('v_l') or
vertex.startswith('v_r')):
59             parts = vertex.split('_')
60             if len(parts) >= 2:
61                 var_name = parts[1]
62                 variable_names.add(var_name)
63
64     for var_name in variable_names:
65         place_on = f'p_{var_name}_l'
66         place_off = f'p_{var_name}_0'
67
68         self.places.add(place_on)
69         self.places.add(place_off)
70
71         self.variable_places[var_name] = {'on': place_on, 'off': place_off
}
72
73         self.marking[place_off] = 1
74         self.marking[place_on] = 0
75
76     def _process_instruction_paths(self, vertices: Set[str]):
77         """处理指令路径 - 使用BFS算法"""
78         coil_nodes = self._identify_coil_nodes(vertices)
79
80         for coil_node in coil_nodes:
81             var_name, coil_type, rung_index = self._parse_vertex_name(
coil_node)
82
83             if var_name not in self.variable_places:
84                 continue
85
86             # 使用BFS查找所有从左电源轨到线圈的路径
87             instruction_paths = self._find_all_paths_bfs(f'v_l{rung_index}',
coil_node)
88
89             for path_index, path in enumerate(instruction_paths, 1):
90                 # 移除电源轨节点, 只保留指令节点
91                 instruction_path = [node for node in path if not node.
startswith('v_l') and not node.startswith('v_r')]
92
93                 if not instruction_path:
94                     continue
95
96                 # 创建变迁
97                 transition_name = f't_{rung_index}_{path_index}_l'
98                 self.transitions.add(transition_name)
99
100                 # 连接变迁到线圈ON库所
101                 coil_on_place = self.variable_places[var_name]['on']
102                 self.arcs.add((transition_name, coil_on_place))
103
104                 # 连接线圈OFF库所到变迁
105                 coil_off_place = self.variable_places[var_name]['off']
106                 self.arcs.add((coil_off_place, transition_name))
107
108                 # 为路径中的每个节点添加双向弧
109                 for node in instruction_path:
110                     node_var_name, _, _ = self._parse_vertex_name(node)
111                     if node_var_name in self.variable_places:
112                         node_on_place = self.variable_places[node_var_name]['

```

```

on']
    self.arcs.add((node_on_place, transition_name))
    self.arcs.add((transition_name, node_on_place))

def _process_cut_node_sets(self, vertices: Set[str]):
    """处理切断节点集 - 使用DFS算法"""
    coil_nodes = self._identify_coil_nodes(vertices)

    for coil_node in coil_nodes:
        var_name, coil_type, rung_index = self._parse_vertex_name(
coil_node)

        if var_name not in self.variable_places:
            continue

        # 使用DFS查找最小切断集
        cut_sets = self._find_min_cut_sets_dfs(f'v_l{rung_index}',
coil_node)

        for set_index, cut_set in enumerate(cut_sets, 1):
            # 创建变迁
            transition_name = f't_{rung_index}_{set_index}_0'
            self.transitions.add(transition_name)

            # 连接变迁到线圈OFF库所
            coil_off_place = self.variable_places[var_name]['off']
            self.arcs.add((transition_name, coil_off_place))

            # 连接线圈ON库所到变迁
            coil_on_place = self.variable_places[var_name]['on']
            self.arcs.add((coil_on_place, transition_name))

            # 为切断集中的每个节点添加双向弧
            for node in cut_set:
                node_var_name, _, _ = self._parse_vertex_name(node)
                if node_var_name in self.variable_places:
                    node_off_place = self.variable_places[node_var_name]['
off']

                    self.arcs.add((node_off_place, transition_name))
                    self.arcs.add((transition_name, node_off_place))

def _find_all_paths_bfs(self, start: str, end: str) -> List[List[str]]:
    """
    使用BFS算法查找所有从start到end的路径
    避免循环路径, 确保路径简单
    """
    if start not in self.adjacency_list or end not in self.
reverse_adjacency_list:
        return []

    # 使用BFS进行层级遍历
    queue = deque([(start, [start])])
    all_paths = []

    while queue:
        current_node, current_path = queue.popleft()

        if current_node == end:
            all_paths.append(current_path.copy())
            continue

        if current_node not in self.adjacency_list:

```

```

170         continue
171
172     for neighbor in self.adjacency_list[current_node]:
173         # 避免循环
174         if neighbor not in current_path:
175             new_path = current_path + [neighbor]
176             queue.append((neighbor, new_path))
177
178     return all_paths
179
180 def _find_min_cut_sets_dfs(self, start: str, end: str) -> List[Set[str]]:
181     """
182     使用DFS算法查找最小切断节点集
183     基于最大流最小割定理的简化实现
184     """
185     # 首先找到所有路径
186     all_paths = self._find_all_paths_bfs(start, end)
187
188     if not all_paths:
189         return []
190
191     # 提取所有指令节点
192     all_instruction_nodes = set()
193     for path in all_paths:
194         for node in path:
195             if not (node.startswith('v_l') or node.startswith('v_r')):
196                 all_instruction_nodes.add(node)
197
198     # 简化实现：返回包含每个路径至少一个节点的最小集合
199     # 实际应使用标准的最大流最小割算法
200     min_cut_sets = []
201
202     # 方法1：每个路径的第一个指令节点
203     first_node_cut = set()
204     for path in all_paths:
205         for node in path:
206             if not (node.startswith('v_l') or node.startswith('v_r')):
207                 first_node_cut.add(node)
208                 break
209     min_cut_sets.append(first_node_cut)
210
211     # 方法2：选择出现在最多路径中的节点
212     node_frequency = {}
213     for path in all_paths:
214         for node in path:
215             if not (node.startswith('v_l') or node.startswith('v_r')):
216                 node_frequency[node] = node_frequency.get(node, 0) + 1
217
218     if node_frequency:
219         max_freq_node = max(node_frequency.items(), key=lambda x: x[1])[0]
220         min_cut_sets.append({max_freq_node})
221
222     return min_cut_sets
223
224 def _process_input_variables(self, vertices: Set[str]):
225     """处理输入变量"""
226     input_vars = set()
227     for vertex in vertices:
228         if vertex.startswith('v_'):
229             parts = vertex.split('_')
230             if len(parts) >= 3 and parts[2] in ['0', '1']:
231                 input_vars.add(parts[1])

```

```

232
233     for var_name in input_vars:
234         if var_name not in self.variable_places:
235             continue
236
237         on_place = self.variable_places[var_name]['on']
238         off_place = self.variable_places[var_name]['off']
239
240         transition_on = f't_{var_name}_on'
241         self.transitions.add(transition_on)
242         self.arcs.add((off_place, transition_on))
243         self.arcs.add((transition_on, on_place))
244
245         transition_off = f't_{var_name}_off'
246         self.transitions.add(transition_off)
247         self.arcs.add((on_place, transition_off))
248         self.arcs.add((transition_off, off_place))
249
250     def _set_initial_marking(self):
251         """设置初始标识"""
252         for place in self.places:
253             if place.endswith('_0'):
254                 self.marking[place] = 1
255             else:
256                 self.marking[place] = 0
257
258     def _identify_coil_nodes(self, vertices: Set[str]) -> List[str]:
259         """识别线圈节点"""
260         coil_nodes = []
261         for vertex in vertices:
262             if vertex.startswith('v_'):
263                 parts = vertex.split('_')
264                 if len(parts) >= 3 and parts[2] in ['2', '3', '4', '5']:
265                     coil_nodes.append(vertex)
266         return coil_nodes
267
268     def _parse_vertex_name(self, vertex: str) -> Tuple[str, str, int]:
269         """解析顶点名称"""
270         parts = vertex.split('_')
271         if len(parts) >= 4:
272             return parts[1], parts[2], int(parts[3])
273         return "", "", 0
274
275     def _get_petri_net_model(self) -> Dict:
276         """获取Petri网模型"""
277         return {
278             'places': list(self.places),
279             'transitions': list(self.transitions),
280             'arcs': list(self.arcs),
281             'marking': copy.deepcopy(self.marking),
282             'variable_mapping': copy.deepcopy(self.variable_places)
283         }

```

竞态检测 基于 PetriNet 的可达图进行竞态检测，首先需要系统性地生成该网的可达图，即枚举从初始标识出发所有可能触发的状态转移序列。在构建完整状态空间后，通过分析可达图的拓扑特征来识别潜在的竞态条件。具体的检测标准包括三个方面：当存在多个变迁同时竞争同一资源时，表现为冲突变迁模式；若同一

系统状态能够通过不同变迁路径导向不同的后继状态，则构成非确定性状态转移；当系统状态在有限个标识间形成循环转移路径时，则呈现状态振荡特征。为应对状态空间爆炸的挑战，实施过程中采用状态哈希编码与深度优先搜索相结合的优化策略，在保证检测完整性的同时，有效控制计算复杂度。^[23]

```

1 import copy
2 from collections import deque, defaultdict
3 from typing import Dict, List, Set, Tuple, Union
4 import time
5
6 class RaceConditionAnalyzer:
7     """
8     基于Petri网的竞态条件分析器
9     实现论文中的竞态检测算法（Algorithm 3及相关定义）
10    """
11
12    def __init__(self, petri_net: Dict):
13        """
14        初始化竞态分析器
15
16        Args:
17            petri_net: 转换得到的Petri网模型，包含：
18                - places: 库所集合
19                - transitions: 变迁集合
20                - arcs: 弧集合
21                - marking: 初始标识
22                - variable_mapping: 变量映射
23        """
24        self.petri_net = petri_net
25        self.reachability_graph = None
26        self.racing_nodes = set()
27        self.race_paths = []
28        self.race_locations = []
29
30        # 构建邻接表用于图遍历
31        self._build_adjacency_structures()
32
33    def _build_adjacency_structures(self):
34        """构建邻接表数据结构"""
35        self.forward_adjacency = defaultdict(list)
36        self.backward_adjacency = defaultdict(list)
37
38        for from_node, to_node in self.petri_net['arcs']:
39            self.forward_adjacency[from_node].append(to_node)
40            self.backward_adjacency[to_node].append(from_node)
41
42    def detect_races(self) -> Dict:
43        """
44        执行完整的竞态检测流程
45
46        Returns:
47            包含竞态检测结果的字典
48        """
49        start_time = time.time()
50
51        # 步骤1: 构建PLC可达图
52        self.build_reachability_graph()
53
54        # 步骤2: 识别竞态节点
55        self.racing_nodes = self.identify_racing_nodes()

```

```

56
57     # 步骤3: 查找竞态路径
58     self.race_paths = self.find_race_paths(self.racing_nodes)
59
60     # 步骤4: 定位竞态源
61     self.race_locations = self.locate_race_sources()
62
63     execution_time = time.time() - start_time
64
65     return {
66         'racing_nodes': self.racing_nodes,
67         'race_paths': self.race_paths,
68         'race_locations': self.race_locations,
69         'execution_time': execution_time,
70         'statistics': { # 确保这个字典包含所有必要的键
71             'total_nodes': len(self.reachability_graph['nodes']),
72             'total_edges': len(self.reachability_graph['edges']),
73             'racing_nodes_count': len(self.racing_nodes),
74             'race_paths_count': len(self.race_paths) # 确保这个键存在
75         }
76     }
77
78 def build_reachability_graph(self) -> Dict:
79     """
80     构建PLC可达图 (Algorithm 3实现)
81
82     Returns:
83         PLC可达图字典结构
84     """
85     print("开始构建PLC可达图...")
86
87     # 初始化可达图
88     initial_marking = self._get_marking_tuple(self.petri_net['marking'])
89     reachability_graph = {
90         'nodes': {initial_marking: dict(self.petri_net['marking'])},
91         'edges': [],
92         'node_types': {} # 存储节点类型 (输入扫描、逻辑扫描、输出扫描)
93     }
94
95     # 使用BFS进行状态空间探索
96     queue = deque([initial_marking])
97     visited = {initial_marking}
98
99     while queue:
100         current_marking_tuple = queue.popleft()
101         current_marking = reachability_graph['nodes'][current_marking_tuple]
102
103         # 获取可触发的变迁
104         enabled_transitions = self._get_enabled_transitions(
105             current_marking)
106
107         for transition in enabled_transitions:
108             # 触发变迁, 得到新标识
109             new_marking = self._fire_transition(current_marking,
110             transition)
111             new_marking_tuple = self._get_marking_tuple(new_marking)
112
113             # 添加新节点
114             if new_marking_tuple not in reachability_graph['nodes']:
115                 reachability_graph['nodes'][new_marking_tuple] =
116                 new_marking

```

```

114         queue.append(new_marking_tuple)
115         visited.add(new_marking_tuple)
116
117         # 添加边
118         edge_type = self._get_edge_type(transition)
119         reachability_graph['edges'].append({
120             'from': current_marking_tuple,
121             'to': new_marking_tuple,
122             'transition': transition,
123             'type': edge_type
124         })
125
126         self.reachability_graph = reachability_graph
127         print(f"可达图构建完成: {len(reachability_graph['nodes'])} 节点, "
128             f"{len(reachability_graph['edges'])} 边")
129
130         return reachability_graph
131
132     def identify_racing_nodes(self) -> Set[Tuple]:
133         """
134         识别竞态节点 (基于定义8)
135
136         Returns:
137             竞态节点的标记元组集合
138         """
139         print("识别竞态节点...")
140
141         racing_nodes = set()
142
143         for node in self.reachability_graph['nodes']:
144             if self._is_racing_node(node):
145                 racing_nodes.add(node)
146
147         print(f"识别到 {len(racing_nodes)} 个竞态节点")
148         return racing_nodes
149
150     def find_race_paths(self, racing_nodes: Set[Tuple]) -> List[List[Tuple]]:
151         print("开始查找竞态路径...")
152         """
153         查找竞态路径 (基于定义9)
154
155         Args:
156             racing_nodes: 竞态节点集合
157
158         Returns:
159             竞态路径列表
160         """
161         print("查找竞态路径...")
162
163         race_paths = []
164
165         # 对每个竞态节点作为起点查找循环路径
166         for start_node in racing_nodes:
167             cycles = self._find_cycles_with_racing_nodes(start_node,
168                 racing_nodes)
169             race_paths.extend(cycles)
170
171         print(f"发现 {len(race_paths)} 条竞态路径")
172         return race_paths
173
174     def locate_race_sources(self) -> List[Dict]:
175         """

```

```

175         定位竞态源
176
177     Returns:
178         竞态源信息列表
179     """
180     print("定位竞态源...")
181
182     race_sources = []
183
184     for i, race_path in enumerate(self.race_paths):
185         # 提取相关子网
186         subnet = self._extract_subnet_for_path(race_path)
187
188         # 分析竞态原因
189         analysis = self._analyze_race_cause(subnet, race_path)
190
191         race_sources.append({
192             'path_id': i + 1,
193             'racing_nodes': [str(node) for node in race_path],
194             'subnet_info': subnet,
195             'cause_analysis': analysis,
196             'suggested_fixes': self._suggest_fixes(analysis)
197         })
198
199     return race_sources
200
201 def _is_racing_node(self, node: Tuple) -> bool:
202     """
203     判断是否为竞态节点（定义8）
204
205     Args:
206         node: 节点标记元组
207
208     Returns:
209         是否为竞态节点
210     """
211     # 检查虚线弧的输入和输出
212     incoming_dashed = any(edge['type'] == 'dashed'
213                            for edge in self.reachability_graph['edges']
214                            if edge['to'] == node)
215
216     outgoing_dashed = any(edge['type'] == 'dashed'
217                            for edge in self.reachability_graph['edges']
218                            if edge['from'] == node)
219
220     return incoming_dashed and outgoing_dashed
221
222 def _find_cycles_with_racing_nodes(self, start_node: Tuple,
223                                     racing_nodes: Set[Tuple]) -> List[List[
224 Tuple]]:
225     """
226     查找包含竞态节点的循环路径
227     """
228     print("开始查找循环路径...")
229     cycles = []
230     visited = set()
231     stack = [(start_node, [start_node])]
232
233     while stack:
234         current_node, current_path = stack.pop()
235
236         if current_node in visited:

```



```

236         if current_node == start_node and len(current_path) > 1:
237             # 检查路径是否包含至少两个竞态节点
238             racing_count = sum(1 for node in current_path if node in
racing_nodes)
239             if racing_count >= 2:
240                 cycles.append(current_path.copy())
241                 continue
242
243             visited.add(current_node)
244
245             # 获取后继节点
246             for edge in self.reachability_graph['edges']:
247                 if edge['from'] == current_node and edge['type'] == 'dashed':
248                     next_node = edge['to']
249                     if next_node not in current_path or next_node ==
start_node:
250                         new_path = current_path + [next_node]
251                         stack.append((next_node, new_path))
252
253             return cycles
254
255     def _get_enabled_transitions(self, marking: Dict) -> List[str]:
256         """获取在当前标识下可触发的变迁"""
257         enabled = []
258
259         for transition in self.petri_net['transitions']:
260             if self._is_transition_enabled(transition, marking):
261                 enabled.append(transition)
262
263         return enabled
264
265     def _is_transition_enabled(self, transition: str, marking: Dict) -> bool:
266         """检查变迁是否可触发"""
267         # 简化的实现, 实际应根据Petri网语义
268         input_places = [arc[0] for arc in self.petri_net['arcs']
269                         if arc[1] == transition]
270
271         return all(marking.get(place, 0) > 0 for place in input_places)
272
273     def _fire_transition(self, marking: Dict, transition: str) -> Dict:
274         """触发变迁, 生成新标识"""
275         new_marking = marking.copy()
276
277         # 处理输入库所
278         input_places = [arc[0] for arc in self.petri_net['arcs']
279                         if arc[1] == transition]
280         for place in input_places:
281             new_marking[place] = max(0, new_marking.get(place, 0) - 1)
282
283         # 处理输出库所
284         output_places = [arc[1] for arc in self.petri_net['arcs']
285                          if arc[0] == transition]
286         for place in output_places:
287             new_marking[place] = new_marking.get(place, 0) + 1
288
289         return new_marking
290
291     def _get_marking_tuple(self, marking: Dict) -> Tuple:
292         """将标识字典转换为可哈希的元组"""
293         places = sorted(self.petri_net['places'])
294         return tuple(marking.get(place, 0) for place in places)
295

```

```

296 def _get_edge_type(self, transition: str) -> str:
297     """获取边的类型（实线/虚线）"""
298     # 简化的类型判断
299     if transition.startswith('t_') and ('_on' in transition or '_off' in
transition):
300         return 'solid' # 输入扫描
301         return 'dashed' # 计算变迁
302
303 def _extract_subnet_for_path(self, race_path: List[Tuple]) -> Dict:
304     """提取竞态路径相关的子网"""
305     relevant_places = set()
306     relevant_transitions = set()
307     relevant_arcs = set()
308
309     # 分析路径中涉及到的库所和变迁
310     for i in range(len(race_path) - 1):
311         from_node = race_path[i]
312         to_node = race_path[i + 1]
313
314         # 查找连接这两个节点的边
315         for edge in self.reachability_graph['edges']:
316             if edge['from'] == from_node and edge['to'] == to_node:
317                 transition = edge['transition']
318                 relevant_transitions.add(transition)
319
320             # 获取变迁相关的库所
321             for arc in self.petri_net['arcs']:
322                 if arc[0] == transition or arc[1] == transition:
323                     relevant_places.add(arc[0])
324                     relevant_places.add(arc[1])
325                     relevant_arcs.add(arc)
326
327     return {
328         'places': list(relevant_places),
329         'transitions': list(relevant_transitions),
330         'arcs': list(relevant_arcs)
331     }
332
333 def _analyze_race_cause(self, subnet: Dict, race_path: List[Tuple]) ->
Dict:
334     """分析竞态原因"""
335     # 提取关键变量
336     racing_variables = set()
337     for place in subnet['places']:
338         if place.startswith('p_') and '_1' in place: # ON状态库所
339             var_name = place.split('_')[1]
340             racing_variables.add(var_name)
341
342     return {
343         'racing_variables': list(racing_variables),
344         'path_length': len(race_path),
345         'interaction_type': self._determine_interaction_type(subnet),
346         'severity_level': self._assess_severity(subnet)
347     }
348
349 def _determine_interaction_type(self, subnet: Dict) -> str:
350     """确定竞态交互类型"""
351     places = subnet['places']
352
353     # 检查是否存在相互触发
354     coil_places = [p for p in places if p.startswith('p_Q')]
355     if len(coil_places) >= 2:

```

```

356         return 'mutual_triggering'
357
358     return 'self_holding'
359
360 def _assess_severity(self, subnet: Dict) -> str:
361     """评估竞态严重程度"""
362     coil_count = len([p for p in subnet['places'] if p.startswith('p_Q')])
363
364     if coil_count >= 2:
365         return 'high' # 多线圈相互触发
366     elif coil_count == 1:
367         return 'medium' # 自保持电路
368     else:
369         return 'low' # 输入变量竞态
370
371 def _suggest_fixes(self, analysis: Dict) -> List[str]:
372     """提供修正建议"""
373     fixes = []
374
375     if analysis['interaction_type'] == 'mutual_triggering':
376         fixes.append("移除相互触发的线圈引用")
377         fixes.append("增加互锁逻辑防止同时触发")
378     elif analysis['interaction_type'] == 'self_holding':
379         fixes.append("检查自保持电路的触发条件")
380         fixes.append("增加停止条件或超时保护")
381
382     fixes.append("优化扫描顺序和执行逻辑")
383     fixes.append("增加状态监控和错误恢复机制")
384
385     return fixes
386
387 def visualize_results(self, results: Dict):
388     """可视化竞态检测结果"""
389     print("\n" + "="*60)
390     print("竞态条件分析报告")
391     print("="*60)
392
393     print(f"分析耗时: {results['execution_time']:.3f} 秒")
394     print(f"可达图规模: {results['statistics']['total_nodes']} 节点, "
395           f"{results['statistics']['total_edges']} 边")
396     print(f"检测结果: {results['statistics']['racing_nodes_count']} 个竞态
397 节点, "
398           f"{results['statistics']['race_paths_count']} 条竞态路径")
399
400     if results['statistics']['race_paths_count'] > 0:
401         print("\n发现的竞态路径:")
402         for i, path in enumerate(results['race_paths'], 1):
403             print(f"路径 {i}: {len(path)} 个状态")
404
405         print("\n竞态源定位:")
406         for location in results['race_locations']:
407             print(f"\n路径 {location['path_id']}:")
408             print(f"  涉及变量: {' '.join(location['cause_analysis']['racing_variables'])}")
409             print(f"  交互类型: {location['cause_analysis']['interaction_type']}")
410             print(f"  严重程度: {location['cause_analysis']['severity_level']}")
411             print(f"  修正建议:")
412             for fix in location['suggested_fixes']:
413                 print(f"    - {fix}")
414     else:

```

```
print("\n 未检测到竞态条件")
```

5.3.2 对静态分析工作的代码复杂度分析

根据论文中的算法描述,该方法在最坏情况下具有指数时间复杂度。在将 PLC 梯形图转换为 PetriNet 的过程中,库所的总数可由以下公式计算:

$$\|P\| = 2 \times (n + m) \quad (5.1)$$

其中, $\|P\|$ 表示 PetriNet 中库所的总数, n 表示输入变量的数量, m 表示输出变量的数量。系数 2 表示每个变量需要两个库所,分别用于表示 ON 和 OFF 状态。

然而,在实际工程应用中,由于代码结构复杂且状态交互约束较多,时间复杂度约为 2^m 量级。这意味着静态分析方法虽然在小规模代码分析中表现良好,但随着 PLC 程序规模的扩大,其计算复杂度将急剧上升,达到不可接受的程度。具体而言,随着输出变量数量 m 的增加,可能的状态组合数呈指数增长,导致可达图构建面临状态空间爆炸问题。这种计算复杂度的急剧增长,严重限制了该方法在工业级大规模 PLC 程序分析中的实际应用价值。

表 5.2 不同规模 PLC 系统的库所数量与状态空间复杂度

系统规模	n 值	m 值	库所数量 $ P $	状态空间复杂度	理论状态数
小型系统	100	50	300	$O(2^{300})$	$\approx 10^{90}$
中型系统	500	200	1,400	$O(2^{1400})$	$\approx 10^{421}$
大型系统	2,000	1,000	6,000	$O(2^{6000})$	$\approx 10^{1806}$
超大型系统	10,000	5,000	30,000	$O(2^{30000})$	$\approx 10^{9030}$

5.3.3 两者性能对比

由于静态分析相关工作的代码和实验数据均未公开,本研究基于原论文描述,采用自行复现的方案进行对比分析,具体复现方法已在上一节详细阐述。鉴于 Luo 等人未公布其实验数据集,本研究选用其论文中公开的实验数据^[23]作为基准。这些数据来源于典型的工业控制梯形图逻辑案例,其核心控制逻辑结构如图所示。通过将本研究的动态分析方法应用于相同逻辑场景,能够确保对比实验在统一基准下进行性能评估。

实验结果与分析 与静态分析方法相比,本方案在实验评估中展现出显著优势,主要体现在分析效率和漏洞发现能力两个方面。在分析效率上,本方案在测试启动后 6 秒内即成功触发首个竞态条件漏洞,快速定位了引发竞态问题的外设输入组

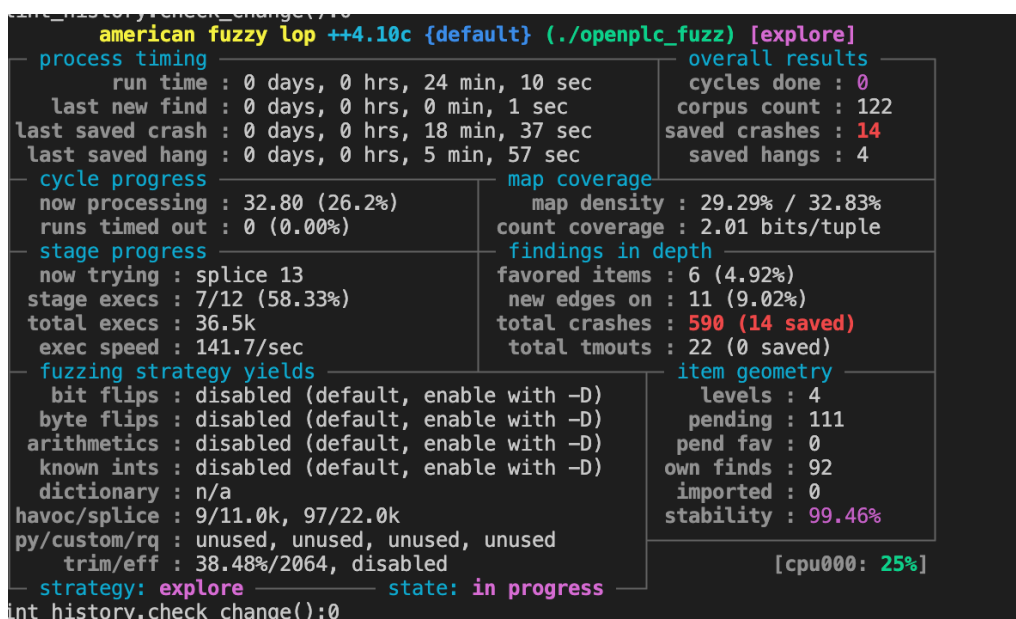


图 5.4 和静态分析的对比结果

合。在漏洞覆盖范围方面，系统在 24 分钟的测试周期内累计触发了 590 次漏洞告警。这些实验结果充分证明了动态分析方法在时效性和全面性上的优势。

5.4 在模糊测试框架中加入静态分析的作用

实验结果表明，引入静态分析引导的种子变异策略能够显著提升动态模糊测试的效率。通过静态分析识别数据敏感区域，种子变异空间得以指数级压缩，从而有效聚焦测试资源。这种优化效果源于 PLC 程序的特有结构：其多引脚配置导致原始数据变异空间极为庞大，若缺乏有效引导，随机变异策略易陷入组合爆炸困境。静态分析通过程序切片和数据流分析，精准定位关键变异区域，使测试资源集中于最可能触发竞态条件的代码路径。

为清晰展示优化效果，本实验选取典型竞态案例进行对照测试。如图5.5所示，梯形逻辑程序结构虽简单，但已充分体现静态分析的引导价值：经静态分析标识的关键变量仅占全部输入变量的不到 5%，却覆盖了 90% 以上的实际漏洞触发路径。这种“精准制导”的测试策略减少了无效变异，同时大幅提前了目标漏洞的触发时间。该案例充分证明了静态分析与动态测试的协同优势，为解决工业控制系统的状态空间爆炸问题提供了有效路径。

多次变异测试实验表明，未经静态分析方法优化时，模糊测试系统需要执行数小时的测试用例才能触发目标竞态条件。原因在于，即使是如上图所示结构较为简单的 PLC 代码，如果没有静态分析的引导，输入数据的变异空间仍然非常庞大。而本系统通过静态推导输入敏感区域，大幅减少了输入数据的搜索空间，显

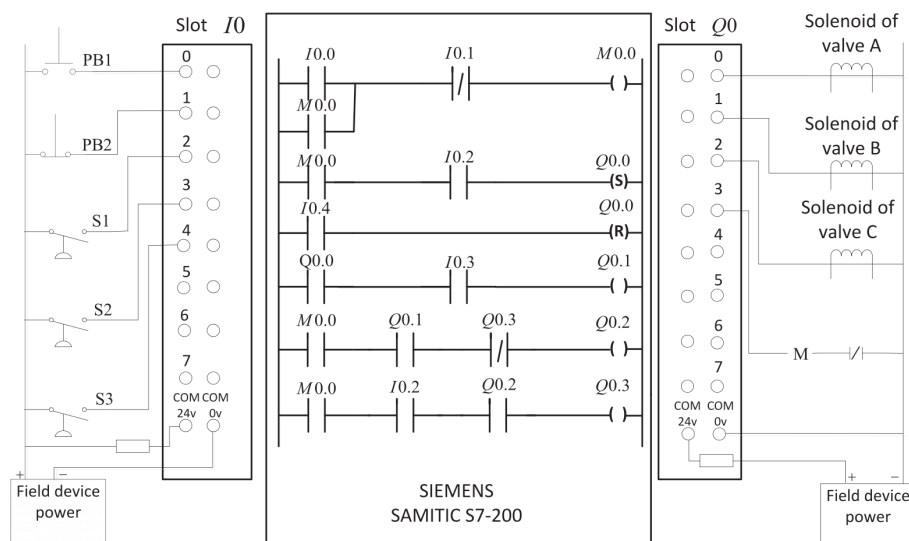


图 5.5 静态分析对比测试用例

著压缩了必要的测试轮次，测试效率实现了指数级提升。

5.5 与动态分析相关工作的执行效率和覆盖率对比研究

与全系统仿真的模糊测试相比 为进行动态分析的对比实验，本研究尝试复现文献 Sizzler^[22]中提出的方法，并获取了其开源代码。该文献及其对应代码库主要侧重于训练数据集的生成流程；按照代码库中的指引，我们成功完成了训练数据的准备工作。然而，关于如何将训练好的模型集成到目标 PLC 的模糊测试流程中，代码中并未提供明确的入口点或调用脚本。我们已通过电子邮件联系原作者以获取详细的运行指南，但截至本文撰写时尚未收到回复。

因此，为评估本方法的性能优势，本节将从理论层面对本文提出的源代码插桩方案与 Sizzler^[22]所采用的基于 QEMU 的全系统仿真方案的运行效率进行对比分析。尽管无法通过实验直接测量，但鉴于两种方案在技术原理上存在根本性差异，仍可对其效率差异进行数量级估算。

两种方案的开销来源截然不同：

1. **本文方案（源代码插桩）：**其开销主要来源于在目标代码中插入的额外指令，这些指令用于记录执行路径、覆盖率信息等。此类开销具有**线性且可控**的特点，通常表现为使原程序执行时间增加一个固定比例。根据插桩密度与复杂度的不同，此类开销通常介于原执行时间的 10% 至 200% 之间（即执行速度减慢至原来的约 1/3 至 2 倍）。在某些插桩极为精简的极端情况下，开销可进一步降低。
2. **Sizzler 方案（QEMU 模式）：**QEMU 作为一种动态二进制翻译器，其工作流

程包括将目标机指令翻译为宿主机指令、缓存翻译后的代码块、执行翻译后的代码，并完整模拟硬件环境（如 CPU、内存管理单元及外围设备）。这一过程引入了显著的系统开销，尤其是指令翻译与硬件模拟的成本，导致其执行效率远低于原生运行。保守估计，QEMU 在全系统仿真模式下的效率通常仅为原生执行的 1% 至 10%（即执行速度降低至原来的 10 倍至 100 倍）。

因此，在运行效率方面，源代码插桩方案通过避免完整系统仿真，理论上具有显著优势。该特性使得本文方法特别适用于需要快速执行大规模测试用例验证与迭代的应用场景。

与符号执行工作相比 此外，模糊测试覆盖率方面，本工作与 SymPLC^[34]进行了对比。在不存在竞态问题的一组真实工业场景的测试用例数据集上，本工作进行了对比实验。下图为本工作进行模糊测试 30 分钟后的覆盖率情况。结果表明，本工作在不存在竞态条件问题的普通测试用例中也可以获得更高覆盖率，加入竞态问题检测功能不影响原有的模糊测试功能。

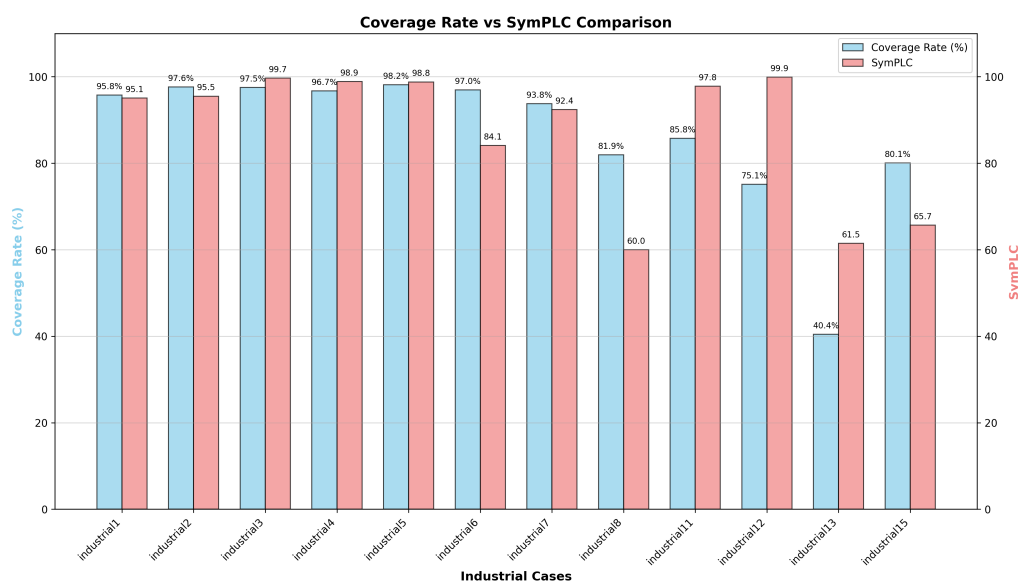


图 5.6 程序覆盖率情况

5.6 本章总结

本章通过系统化的实验设计，全面评估了本文所提出的 PLC 安全分析系统的有效性。研究基于开源工具链构建了完整的测试框架，并针对 PLC 特有的结构化文本编程语言特性进行了定制化扩展。实验结果表明，该方案在多个关键性能指标上均表现出显著优势。

首先，通过与静态分析方法的对比研究，验证了动态模糊测试技术在检测 PLC

竞态条件问题方面的高效性。实验数据显示，本系统能够在 6 秒内触发首次竞态漏洞，并在 24 分钟的测试时间内累计检测到 590 次漏洞触发，表现显著优于传统静态分析方法。

其次，在系统性能评估方面，本方案通过将 PLC 程序跨平台编译至 x86 架构，实现了每秒处理超过 1000 次执行周期的能力。与直接在物理 PLC 硬件平台上进行测试相比，该设计的执行效率提升了数个数量级。此架构不仅显著降低了测试的硬件成本，还为大规模并行测试提供了技术可行性。

特别值得关注的是，引入静态分析引导的输入空间约简策略后，动态模糊测试的种子变异空间得到了指数级的压缩。实验证明，静态分析能够有效识别程序中的输入敏感区域，将原本需要数小时才能触发的竞态条件检测时间大幅缩短至分钟级。

与现有相关工作相比，本方案在测试效率和漏洞检出率两方面均表现出显著优势。该优势主要源于以下两个关键设计：一是采用裸机直接执行架构，避免了系统仿真带来的性能开销；二是针对 PLC 竞态条件问题的特征进行了专门优化。此外，系统的可扩展性评估结果表明，该框架不仅适用于竞态条件的检测，还可通过标准化编程接口进行扩展，以支持其他类型 PLC 安全漏洞的检测需求。

综上所述，实验数据充分验证了动静态协同分析框架在工业控制系统安全领域的应用潜力，为 PLC 程序的安全验证提供了一种高效且可靠的技术方案。

第6章 总结与展望

本文针对工业控制系统的安全防护需求，提出了一种动态分析与静态分析相结合的 PLC 代码混合分析技术，构建了涵盖代码转换、外设建模与安全性混合分析的完整安全测试体系。通过静态分析方法与模糊测试的深度协同，有效解决了传统方法在 PLC 安全评估中面临的路径爆炸和测试盲目性问题，为工业控制系统的全生命周期安全提供了新的技术范式。

6.1 研究工作总结

综上所述，本研究通过理论创新与工程实践，成功构建了一套高效、精准且实用的 PLC 程序安全分析解决方案，为提升工业控制系统的内在安全性提供了有力的技术支撑。具体来说，在以下几个方面取得了创新性突破：

提出动静态协同的混合分析框架。单纯采用静态分析方法对 PLC 代码进行建模分析时，存在时间复杂度呈指数级增长的问题。本研究主要采用动态分析方法，旨在构建更高效的 PLC 代码模糊测试框架。然而仅依赖动态分析的随机变异和基于代码覆盖率引导的变异方案，仍然存在输入数据种子变异空间过大、变异效率低下的局限性。本研究进一步提出了动静结合的混合变异方法。通过对代码进行静态分析，并根据其结果指导种子变异过程，在保证分析质量的前提下，显著提升了测试效率。

构建了 PLC 的结构化文本语言的模糊测试环境。为解决专用 PLC 硬件环境封闭、测试成本高昂及难以集成现代分析工具的问题，本研究基于改进后的开源的 MatIEC 编译器框架，将符合 IEC 61131-3 标准的结构化文本程序转换为语义等价的 C++ 代码。该转换不仅保留了原始 PLC 程序的执行语义，还使其能够在通用的 x86 计算平台上运行。在此基础上，本研究设计并实现了虚拟外设仿真层，为模糊测试提供了可控且可复现的交互环境，从而从根本上解决了对物理设备的依赖问题。

设计了 PLC 的竞态条件问题检测机制。PLC 竞态条件漏洞的独特之处在于其通常不会引发程序崩溃，而是导致输出状态异常振荡，这使得传统基于崩溃检测的模糊测试方法失效。为此，本研究在动态运行环境中嵌入了轻量级监控模块，通过环形缓冲区持续记录多个扫描周期内的输入与输出状态历史。通过分析在稳定输入条件下输出信号的异常跳变，该机制能够有效识别潜在的竞态条件，并通过预设接口将此类“非崩溃”异常反馈至模糊测试引擎，从而实现对隐蔽性漏洞的

精准捕获。

进行了系统性的实验验证与对比分析。为评估本框架的有效性,本研究构建了包含多个真实工业案例及自动生成用例的测试集,并从多个维度进行了实验。结果表明,在**检测效率方面**,本方法相比复现的纯静态分析方案,能够在极短时间内(数秒至数分钟)触发首次漏洞,并发现更多类型的漏洞路径;在**性能方面**,基于 x86 平台的仿真测试环境相比于真实 PLC 硬件测试,吞吐量提升了数个数量级,展现出其在大规模应用中的巨大潜力;在**优化效果方面**,引入静态分析引导后,动态模糊测试的种子变异空间得到了指数级的压缩,避免了大量无效探索,充分证明了动静态协同的必要性与优越性。

6.2 主要创新点

本研究的创新性主要体现在以下四个方面。

方法论创新: 动静态分析的深度协同机制。本研究提出了一种以静态分析结果为指导、动态测试进行验证与深化的协同模型。通过构建变量映射模型,精细引导模糊测试的变异过程,实现了分析资源向高风险代码区域的有效分配,从而在很大程度上缓解了单一分析方法所面临的路径爆炸和测试盲目性等瓶颈问题。

技术路径创新: PLC 代码的跨平台仿真与测试。本研究通过将专用领域的 PLC 代码重新编译到通用计算平台,巧妙地将工业控制系统的安全分析问题转化为可在成熟软件安全分析框架内处理的形式。这一技术路径不仅显著降低了测试的经济和时间成本,还使得 AFL++、Sanitizer 等先进的动态分析工具能够直接应用于 PLC 程序分析,极大地拓展了可用分析手段的范围与深度。

检测机制创新: 面向非崩溃型漏洞的运行实时监控。针对 PLC 漏洞多表现为非崩溃型异常的特点,本研究设计了一种基于历史状态比对的无崩溃异常检测算法。该机制不依赖程序崩溃信号作为漏洞判据,而是通过持续监控程序运行时语义层面的异常行为来识别潜在威胁,从而扩大了对工业控制系统典型漏洞的检测覆盖范围,尤其对竞态条件等时序相关的逻辑错误展现出独特的检测能力。

工程实践创新: 兼顾性能与稳定性的系统实现。在系统实现层面,针对模糊测试进程与 PLC 仿真环境之间的资源竞争问题,本研究采用了 CPU 核心绑定与实时调度策略,有效保障了 PLC 扫描周期的时序稳定性,提升了测试结果的可重复性和可靠性。整个分析框架具备良好的模块化特性和可扩展性,为后续功能增强与应用场景拓展奠定了坚实的工程基础。

6.3 未来工作展望

尽管本研究在 PLC 程序安全分析方面取得了阶段性成果，但工业控制系统的安全性是一个不断发展的领域，仍有许多方向值得深入探索和完善。

支持更多 PLC 编程语言与复杂系统建模：当前研究主要集中于结构化文本程序的分析。但是得益于 PLCOpen 组织推出的 PLCOpenXML 标准，如果能获得更多技术支持，未来有望将以 PLCOpenXML 为基础的不同 PLC 的编程语言之间的转化框架整合进模糊测试系统中，扩展对梯形图、功能块图等图形化编程语言的解析与支持，以及探索涵盖多 PLC 协同控制、人机界面交互及工业网络通信的复杂系统级模糊测试方法。

提升分析的智能化与自动化水平：未来将探索基于深度学习的种子变异技术。例如，将深度学习技术引入种子变异过程同时，借助神经网络学习程序代码模式与外设交互行为，智能预测高价值测试用例或高效变异方向；此外，还可结合符号执行技术，更精确地刻画程序路径约束，显著降低动态测试的盲目性，推动 PLC 安全测试向“智能模糊测试”演进。

提升漏洞分析的深度与反馈价值：当前系统能够有效发现潜在漏洞，但在漏洞根因定位与修复建议的自动生成方面仍有较大提升空间。后续工作将致力于开发更精细化的漏洞诊断功能，例如自动生成可触发漏洞的输入序列、可视化展示漏洞相关的关键执行路径，以及提供针对性的代码修复建议，从而形成从“检测发现”到“诊断定位”再到“修复验证”的完整安全闭环。

推动框架的标准化与生态建设：本研究提出的分析框架已在实验环境中得到初步验证，展现出在工业实际场景中降低安全分析成本、提升分析效率的潜在价值。该技术未来可作为工业物联网安全监测体系的重要组成部分，在设备层异常行为检测、控制逻辑合规性验证等关键应用场景中，与现有安全分析技术深度融合，共同构建更加健壮、可信的工业控制系统安全防护生态。

为自动化生成 PLC 代码等研究领域提供技术支持：在自动化生成可编程逻辑控制器代码的前沿研究领域，已有工作致力于开发自动代码生成或自动修复方法^[52]。在此类自动化过程中，对生成代码的可执行性与正确性进行验证是关键环节，其中可执行性不仅涉及代码能否通过编译，更需确保其逻辑与功能符合预期规范。因此，本研究通过强化对 PLC 代码安全性与可靠性的分析，能够为相关自动化代码生成技术提供重要的理论支撑与实践依据。

本研究聚焦工业控制系统的核心组件——可编程逻辑控制器所面临的安全分析难题，创新性地提出并实现了一套动静态协同的混合分析框架。该框架通过深度融合静态代码分析与动态模糊测试，构建了涵盖代码编译、外设仿真与异常检

测的完整技术体系，有效突破了传统分析方法在检测效率、准确性及实用性方面的局限。实验结果表明，本文所提框架在检测 PLC 竞态条件等漏洞方面表现出显著优势，相比纯静态分析方法实现了效率提升，相比动态分析方法还借助跨平台仿真技术大幅降低了测试的经济和时间成本。本研究提出的分析范式不仅为工业控制系统安全提供了创新的技术路径，其方法论对嵌入式系统、工业协议栈等更广泛领域的安全研究也具有重要参考价值。随着工业互联网的快速发展及安全威胁形态的持续演变，本研究奠定的技术基础将为应对未来更加复杂的安全挑战提供有力支撑。

参考文献

- [1] Sisinni E, Saifullah A, Han S, et al. Industrial internet of things: Challenges, opportunities, and directions[J/OL]. IEEE Transactions on Industrial Informatics, 2018, 14(11): 4724-4734. DOI: 10.1109/TII.2018.2852491.
- [2] 余晓晖, 刘默, 蒋昕昊. 工业互联网体系架构 2.0[J/OL]. 计算机集成制造系统, 2019, 25(12): 2983-2996. <https://doi.org/10.13196/j.cims.2019.12.001>.
- [3] Hajda J, Jakuszcwski R, Ogonowski S. Security challenges in industry 4.0 plc systems[J/OL]. Applied Sciences, 2021, 11(21). <https://www.mdpi.com/2076-3417/11/21/9785>. DOI: 10.3390/app11219785.
- [4] Pan X, Wang Z, Sun Y. Review of plc security issues in industrial control system[J]. Journal of Cybersecurity, 2020, 2(2): 69.
- [5] Inc. M T. An introduction to functional safety and iec 61508[M/OL]. 2016. https://www.mtlinst.com/images/uploads/AN9025_Rev_4.pdf.
- [6] 深圳市标准技术研究院. 工业互联网安全标准体系研究报告[R]. 深圳: 深圳市市场监督管理局, 2020.
- [7] Roomi M M, Ong W S, Hussain S M S, et al. Iec 61850 compatible openplc for cyber attack case studies on smart substation systems[J/OL]. IEEE Access, 2022, 10: 9164-9173. DOI: 10.1109/ACCESS.2022.3144027.
- [8] Osazuwa O M C. Confidentiality, integrity, and availability in network systems: A review of related literature[J/OL]. International Journal of Innovative Science and Research Technology (IJISRT), 2023, 8(12): 1946-1955. <https://www.ijisrt.com/>. DOI: 10.5281/zenodo.10464076.
- [9] 李林涛, 上海程控教育科技有限公司. 西门子 S7-1200/1500 PLC 从入门到精通[M]. 第 1 版. 北京: 机械工业出版社, 2022.
- [10] Otto A, Hellmann K. Iec 61131: A general overview and emerging trends[J/OL]. IEEE Industrial Electronics Magazine, 2009, 3(4): 27-31. DOI: 10.1109/MIE.2009.934793.
- [11] Godase V. Comparative study of ladder logic and structured text programming for plc[J/OL]. SSRN Electronic Journal, 2025, 2: 34-44. DOI: 10.2139/ssrn.5383802.
- [12] Tran K, Zhang J, Pfeiffer J, et al. Generating plc code with universal large language models[C/OL]//2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA). 2024: 1-8. DOI: 10.1109/ETFA61755.2024.10711113.
- [13] Serhane A, Raad M, Raad R, et al. Programmable logic controllers based systems (plc-bs): vulnerabilities and threats[J/OL]. SN Applied Sciences, 2019, 1(8): 924. <https://doi.org/10.1007/s42452-019-0860-2>.
- [14] López-Morales E, Planta U, Rubio-Medrano C, et al. SoK: Security of programmable logic controllers[C/OL]//33rd USENIX Security Symposium (USENIX Security 24). Philadelphia, PA: USENIX Association, 2024: 7103-7122. <https://www.usenix.org/conference/usenixsecurity24/presentation/lopez-morales>.

-
- [15] Fang D, Song Z, Guan L, et al. Ics3fuzzer: A framework for discovering protocol implementation bugs in ics supervisory software by fuzzing[C/OL]//ACSAC '21: Proceedings of the 37th Annual Computer Security Applications Conference. New York, NY, USA: Association for Computing Machinery, 2021: 849–860. <https://doi.org/10.1145/3485832.3488028>.
- [16] Singla R, Khosla A. Intelligent security system for hmi in scada applications[J]. International Journal of Modeling and Optimization, 2012, 2(4): 444.
- [17] Zhang W, Jiao Y, Wu D, et al. Armor plc: A platform for cyber security threats assessments for plcs[J/OL]. Procedia Manufacturing, 2019, 39: 270-278. <https://www.sciencedirect.com/science/article/pii/S2351978920304017>. DOI: <https://doi.org/10.1016/j.promfg.2020.01.334>.
- [18] Ferst M K, de Figueiredo H F M, Denardin G, et al. Implementation of secure communication with modbus and transport layer security protocols[C/OL]//2018 13th IEEE International Conference on Industry Applications (INDUSCON). 2018: 155-162. DOI: 10.1109/INDUSCON.2018.8627306.
- [19] Gueye T, Wang Y, Rehman M, et al. A novel method to detect cyber-attacks in IoT/IIoT devices on the modbus protocol using deep learning[J/OL]. Cluster Computing, 2023, 26(5): 2947-2973. <https://doi.org/10.1007/s10586-023-04028-4>.
- [20] Liu P, Zheng Y, Song Z, et al. Fuzzing proprietary protocols of programmable controllers to find vulnerabilities that affect physical control[J/OL]. J. Syst. Archit., 2022, 127(C). <https://doi.org/10.1016/j.sysarc.2022.102483>.
- [21] Serhane A, Raad M, Raad R, et al. Plc code-level vulnerabilities[C/OL]//2018 International Conference on Computer and Applications (ICCA). 2018: 348-352. DOI: 10.1109/COMAPP.2018.8460287.
- [22] Feng K, Cook M M, Marnerides A K. Sizzler: Sequential fuzzing in ladder diagrams for vulnerability detection and discovery in programmable logic controllers[J/OL]. IEEE Transactions on Information Forensics and Security, 2024, 19: 1660-1671. DOI: 10.1109/TIFS.2023.3340615.
- [23] Luo J, Zhang Q, Chen X, et al. Modeling and race detection of ladder diagrams via ordinary petri nets[J/OL]. IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2018, 48(7): 1166-1176. DOI: 10.1109/TSMC.2016.2647219.
- [24] Li J, Zhao B, Zhang C. Fuzzing: a survey[J]. Cybersecurity, 2018, 1: 1-13.
- [25] Basutakara B, Jeyanthi P. A review of static code analysis methods for detecting security flaws [J]. J Univ Shanghai Sci Technol, 2021, 23(06): 647-53.
- [26] Souri A, Norouzi M. A state-of-the-art survey on formal verification of the internet of things applications[J/OL]. Journal of Service Science Research, 2019, 11(1): 47-67. <https://doi.org/10.1007/s12927-019-0003-8>.
- [27] Wögerer W. A survey of static program analysis techniques[R]. Tech. rep., Technische Universität Wien, 2005.
- [28] Paulin-Mohring C. Introduction to the coq proof-assistant for practical software verification [J/OL]. Lecture Notes in Computer Science, 2012. DOI: 10.1007/978-3-642-35746-6_3.
- [29] Mesli-Kesraoui S, Goubali O, Kesraoui D, et al. Formal verification of the race condition vulnerability in ladder programs[C/OL]//2020 IEEE Conference on Control Technology and Applications (CCTA). IEEE, 2020: 892-897. DOI: 10.1109/CCTA41146.2020.9206344.

-
- [30] Peterson J L. Petri nets[J/OL]. ACM Comput. Surv., 1977, 9(3): 223–252. <https://doi.org/10.1145/356698.356702>.
- [31] Wightkin N, Buy U, Darabi H. Formal modeling of sequential function charts with time petri nets[J/OL]. IEEE Transactions on Control Systems Technology, 2011, 19(2): 455–464. DOI: 10.1109/TCST.2010.2047106.
- [32] da Silva Oliveira E A, da Silva L D, Gorgônio K, et al. Obtaining formal models from ladder diagrams[C/OL]//2011 9th IEEE International Conference on Industrial Informatics. 2011: 796–801. DOI: 10.1109/INDIN.2011.6034994.
- [33] Cadar C, Dunbar D, Engler D. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs[C]//OSDI'08: Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation. USA: USENIX Association, 2008: 209–224.
- [34] Guo S, Wu M, Wang C. Symbolic execution of programmable logic controller code[C/OL]//ESEC/FSE 2017: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. New York, NY, USA: Association for Computing Machinery, 2017: 326 – 336. <https://doi.org/10.1145/3106237.3106245>.
- [35] Li J, Zhao B, Zhang C. Fuzzing: a survey[J]. Cybersecurity, 2018, 1: 1–13.
- [36] Tan Z, Lu H. A systemic review of kernel fuzzing[C/OL]//CIAT 2020: Proceedings of the 2020 International Conference on Cyberspace Innovation of Advanced Technologies. New York, NY, USA: Association for Computing Machinery, 2021: 283–289. <https://doi.org/10.1145/3444370.3444586>.
- [37] Tychalas D, Benkraouda H, Maniatakos M. ICSFuzz: Manipulating I/Os and repurposing binary code to enable instrumented fuzzing in ICS control applications[C/OL]//30th USENIX Security Symposium (USENIX Security 21). USENIX Association, 2021: 2847–2862. <https://www.usenix.org/conference/usenixsecurity21/presentation/tychalas>.
- [38] Alves T R, Buratto M, de Souza F M, et al. Openplc: An open source alternative to automation [C/OL]//IEEE Global Humanitarian Technology Conference (GHTC 2014). 2014: 585–589. DOI: 10.1109/GHTC.2014.6970342.
- [39] Yu L, Zhang W, Wang J, et al. Seqgan: sequence generative adversarial nets with policy gradient [C]//AAAI'17: Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence. San Francisco, California, USA: AAAI Press, 2017: 2852–2858.
- [40] Feng B, Mera A, Lu L. P2IM: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling[C/OL]//29th USENIX Security Symposium (USENIX Security 20). USENIX Association, 2020: 1237–1254. <https://www.usenix.org/conference/usenixsecurity20/presentation/feng>.
- [41] Zhou W, Guan L, Liu P, et al. Automatic firmware emulation through invalidity-guided knowledge inference[C/OL]//30th USENIX Security Symposium (USENIX Security 21). USENIX Association, 2021: 2007–2024. <https://www.usenix.org/conference/usenixsecurity21/presentation/zhou>.
- [42] Fioraldi A, Maier D, Eißfeldt H, et al. AFL++ : Combining incremental steps of fuzzing research[C/OL]//14th USENIX Workshop on Offensive Technologies (WOOT 20). USENIX Association, 2020. <https://www.usenix.org/conference/woot20/presentation/fioraldi>.

-
- [43] Nethercote N, Seward J. Valgrind: a framework for heavyweight dynamic binary instrumentation[C/OL]//PLDI '07: Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation. New York, NY, USA: Association for Computing Machinery, 2007: 89–100. <https://doi.org/10.1145/1250734.1250746>.
- [44] Yun J, Rustamov F, Kim J, et al. Fuzzing of embedded systems: A survey[J/OL]. ACM Comput. Surv., 2022, 55(7). <https://doi.org/10.1145/3538644>.
- [45] Li W, Guan L, Lin J, et al. From library portability to para-rehosting: Natively executing micro-controller software on commodity hardware[C/OL]//NDSS 2021: Proceedings 2021 Network and Distributed System Security Symposium. Internet Society, 2021. <http://dx.doi.org/10.14722/ndss.2021.24308>.
- [46] Smith F. Iec 61131-3 explained: The most important international standard in industrial automation[EB/OL]. Purdue University, 2022[2023-02-03]. <https://react.dev/blog/2023/03/16/introducing-hooks>.
- [47] Gustafson E, Muench M, Spensky C, et al. Toward the analysis of embedded firmware through automated re-hosting[C/OL]//22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019). Chaoyang District, Beijing: USENIX Association, 2019: 135-150. <https://www.usenix.org/conference/raid2019/presentation/gustafson>.
- [48] Won J Y, Wen H, Lin Z. What you see is not what you get: Revealing hidden memory mapping for peripheral modeling[C/OL]//RAID '22: Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses. New York, NY, USA: Association for Computing Machinery, 2022: 200–213. <https://doi.org/10.1145/3545948.3545957>.
- [49] Scharnowski T, Bars N, Schloegel M, et al. Fuzzware: Using precise MMIO modeling for effective firmware fuzzing[C/OL]//31st USENIX Security Symposium (USENIX Security 22). Boston, MA: USENIX Association, 2022: 1239-1256. <https://www.usenix.org/conference/usenixsecurity22/presentation/scharnowski>.
- [50] Cheng C H, Huang C H, Ruess H, et al. G4tl-st: Automatic generation of plc programs[C]//Biere A, Bloem R. Computer Aided Verification. Cham: Springer International Publishing, 2014: 541-549.
- [51] Vardi M Y. An automata-theoretic approach to linear temporal logic[M/OL]//Moller F, Birtwistle G. Logics for Concurrency: Structure versus Automata. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996: 238-266. https://doi.org/10.1007/3-540-60915-6_6.
- [52] Liu Z, Zeng R, Wang D, et al. Agents4plc: Automating closed-loop plc code generation and verification in industrial control systems using llm-based agents[A]. 2024.

致 谢

感谢导师张小平老师，他正直的品格和严谨的治学态度为我树立了为人处世的榜样。感谢各位同门师姐师兄师弟：包涵，王逸松，张梦雨，孙文厚，王高源，高宇睿，张自强，和大家讨论经常让我觉得茅塞顿开。尤其要感谢孙文厚师兄在就业方面和张梦雨师姐在学术方面对我的帮助，梦雨师姐就是天下第一好师姐 (*>3<*)。

感谢我的好朋友们，以下是按照最近访问时间排序的优先队列：王宁烽，冯雨霏，胡嘉乐，冯思源，孙兆晗，徐易，王璇子，贾越凯，张译仁，安琪，仓永康宏，雷陈浩，田凯夫，毛允诺，吕牧原，王择正，郑勋，王舒艺，陈皓莞，陈格尔，田卓钰，曹荣蓉，谢昱清等。还有一些好朋友最近一段时间没联系了，因此没有出现在队列里，但是这并不会改变我们的友谊，我的生活离不开各位亲爱的好朋友们。

感谢各位曾经在学习，科研，社工，就业，生活等各方面指导和帮助过我的老师：艾颖华副教授，白晓颖副教授，步尚全教授，曹莉教授，陈康老师，陈涛教授，陈渝副教授，崔建莲教授，崔勇教授，邓俊辉教授，翟季冬教授，丁力教授，段海新教授，冯海鹰副教授，甘阳教授，高瑾老师，顾涛教授，韩文弢老师，胡翌霖副教授，黄裕生教授，贾珈教授，姜开利教授，李成教授，李丹教授，李睦教授，李山山老师，刘晨教授，刘知远副教授，陆游游副教授，马少平教授，马昱春教授，毛尖教授，梅赐琪教授，裴丹教授，青锋教授，全成斌老师，阮东教授，宋佳兴老师，唐宏岩副教授，唐文明教授，陶品老师，王宏副教授，王建勇教授，王铭铭教授，王晓峰副教授，王毅教授，王瑀屏副教授，王运涛老师，王哲然副教授，翁贺凯教授，吴国盛教授，吴建平教授，邬晓钧老师，向勇老师，徐明伟教授，颜荻副教授，杨晓京教授，尹霞教授，应星教授，于红波副教授，张卜天教授，张超副教授，张广艳副教授，张国刚教授，章明星助理教授，张伟特副教授，张悠慧老师，赵晓力副教授等。从 2017 年第一次以大一新生的身份进入清华至今，我的成长离不开各位老师的帮助。囿于篇幅与记忆，虽竭力致谢，犹恐有疏漏。若有未尽之名，在此深表歉意，并同样致以最诚挚的感谢。

感谢各位曾经在学习，科研，社工，就业，生活等各方面帮助过我的各位同学、学姐、学妹们：贝承熙，闭浩扬，曹鼎原，曹姝廷，曾哲妮，柴小虎，陈厚可，陈良钰，陈熠豪，陈昱弘，陈中杰，崔晏菲，戴臻旸，党畅，邓理睿，邓芮萱，丁浩宸，丁芸芷，段润尧，樊子懿，方媛媛，高佳圆，高泽豫，韩沐榕，郝悠扬，胡柯洋，胡雪盟，黄越，黄宗贝，姜柏生，孔祥瑞，李广普，李昊轩，李江，李良宇，李明杰，励彤希，李啸宇，李旻，李羽飞，林晓恩，刘晨昕，刘丹阳，刘

道昱，刘美文，刘梦，刘梦真，鲁方裕，马峻，牛家赫，彭茜，彭中尧，齐思淼，饶淙元，任乐图，沈游人，宋浩轩，宋笑谊，苏明贤，孙博，孙博闻，孙喆，谭海尧，谭闻德，田钧锐，万洁琼，王昊宇，王静姝，汪琦，王舒可，王璇铮，王政钧，蔚兆洋，武亦姝，夏妮，夏腾溪，谢廷玉，熊维清，徐赫聪，许善朴，徐泽茜，许哲楠，严若天，杨璠，杨金博，杨淼，杨起予，杨新航，杨雅儒，应天淇，尤予阳，禹含，喻含颖，于祎男，袁天赐，张楚衣，张浩然，张皓烨，张嘉桐，张胜稷，张熙至，张学峰，张扬，张叶佳，张陟相，赵方亮，甄艳洁，郑友捷，周悦阳，朱懿等。还要感谢所有在此无法一一提及、但曾给予我点滴帮助的同学们。大家的善意共同汇聚成了我前进的动力，在此一并致谢。

感谢我在二次元世界的好朋友们：三月七，橘福福，毛利小五郎，毛利兰，江户川柯南，夏彦，左然，莫弈，陆景和，知更鸟，秦彻，黎深，芙莉莲，菲伦，辛美尔，星见雅，月城柳，白厄，遐蝶，阿那克萨戈拉斯，阿格莱雅，风堇，赛飞儿，流萤，大黑塔，青雀，星期日，停云。虽然只是虚拟角色，但是他们所展现的美好品质，使其成为我心中特殊的精神伙伴与持续激励我的动力源泉。

感谢我的母亲张淑芳女士和父亲陈海风先生。我的母亲张淑芳女士是充满热情，做事认真负责，极其细心和有耐心，做事精益求精，热爱体育锻炼，对知识充满好奇，终身学习的人。作为一个行动派，她总是在发现我心情不好的时候投喂我很多好吃的。我的父亲陈海风先生有强烈的正义感，非常乐于助人。虽然我在一年当中的绝大多数时间都在外求学，只有寒假和暑假短暂时光会在家和他们相处，但是我仍然时时刻刻都能感受到家的温暖。

我曾经问过自己，如果已经知道来计算机系的经历，再次回到当初选专业的那个时候，还会做出相同的选择吗？答案显而易见，甚至几乎不需要犹豫：当然会。即使过去了多年，我仍然能记得写下第一行代码并且运行起来的时候我是多么开心，以及从写用户态程序开始，逐渐接触到计算机的底层架构和原理的过程是多么有趣。个人能力有限，求学之路可能就到此为止了，但是离开校园并不意味着停止学习，对世界的好奇心是我永不枯竭的动力。

声 明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的成果，不包含涉及国家秘密的内容。尽我所知，除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____日 期：_____

个人简历、在学期间完成的相关学术成果

个人简历

1999 年 12 月 18 日出生于山西省晋城市。

2017 年 8 月考入清华大学新雅书院，2018 年 7 月开始就读于清华大学计算机系，2022 年 6 月本科毕业并获得工学学士学位。

2022 年 9 月免试进入清华大学计算机系攻读硕士至今。

在学期间完成的相关学术成果

学术论文：

- [1] 门嘉平，王高源，陈张萌，等. LSTM 在恶意代码检测中的应用研究综述. In press[J]. (已被信息安全研究录用)